

CPE371

1/2569

Big Data Engineering

IoTs for Big Data

Aung Myo Htut

aung.myo1@kmutt.ac.th



Computer Engineering



King Mongkut's
University of
Technology
Thonburi

What we will learn...

WHAT ARE IOTS

WHY ARE IOTS IMPACT TO BIG DATA

HOW DO IOTS SEND DATA COLLECTION TO INTERNET

PRACTICE WITH SOME EXAMPLES

“data is the new oil”

Clive Humby, British mathematician, 2006

- More than 150 zettabytes (150 trillion gigabytes) of data will need analysis by 2025, *and...*
- 87% of companies surveyed believe that data is an asset for their organization. *But...*
- Nearly 36% of companies don't use all the data they possess, *and...*
- 75% said they don't feel prepared to make use of data.

<https://www.iotacommunications.com/blog/iot-big-data/>

What are IoTs?

Internet of Things

IIoTs : Industrial Internet of Things

AloTs : Artificial Intelligent + IoTs

The internet-able things that are embedded with sensor and software to collect and exchange data.

By 2021, there will be an estimated 35 billion IoT devices worldwide (@SecurToday).
Data generated by IoT devices is forecasted to generate 79.4 zettabytes of data by 2025 (@cxociety)

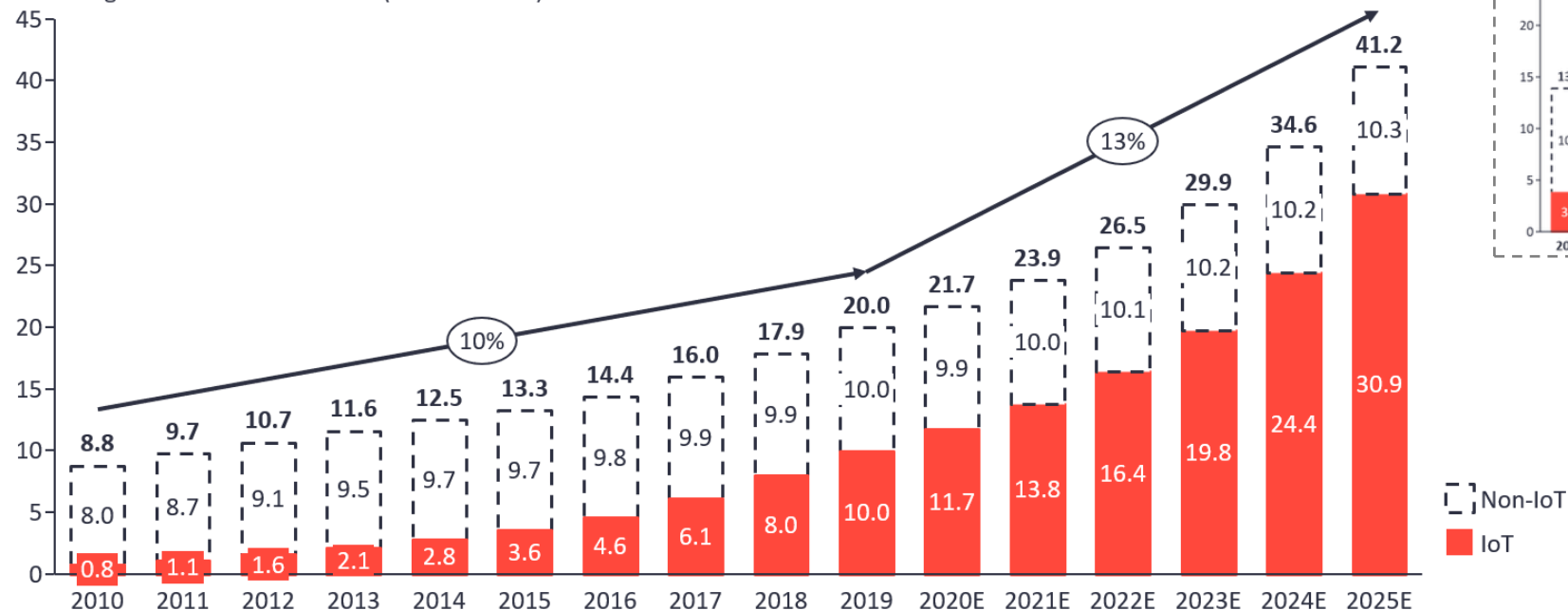
<https://www.iotacommunications.com/blog/iot-big-data/>

Total number of device connections (incl. Non-IoT)

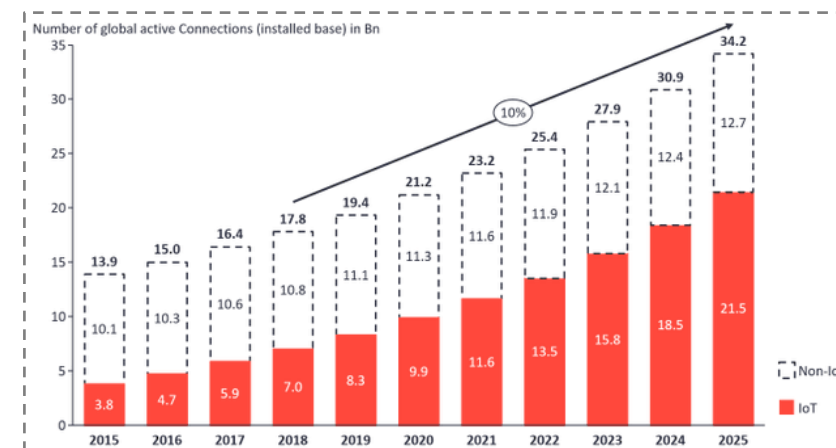
20.0Bn in 2019– expected to grow 13% to 41.2Bn in 2025

Data as of Sep 2018

Number of global active Connections (installed base) in Bn



Xx% = Compound Annual Growth Rate (CAGR)

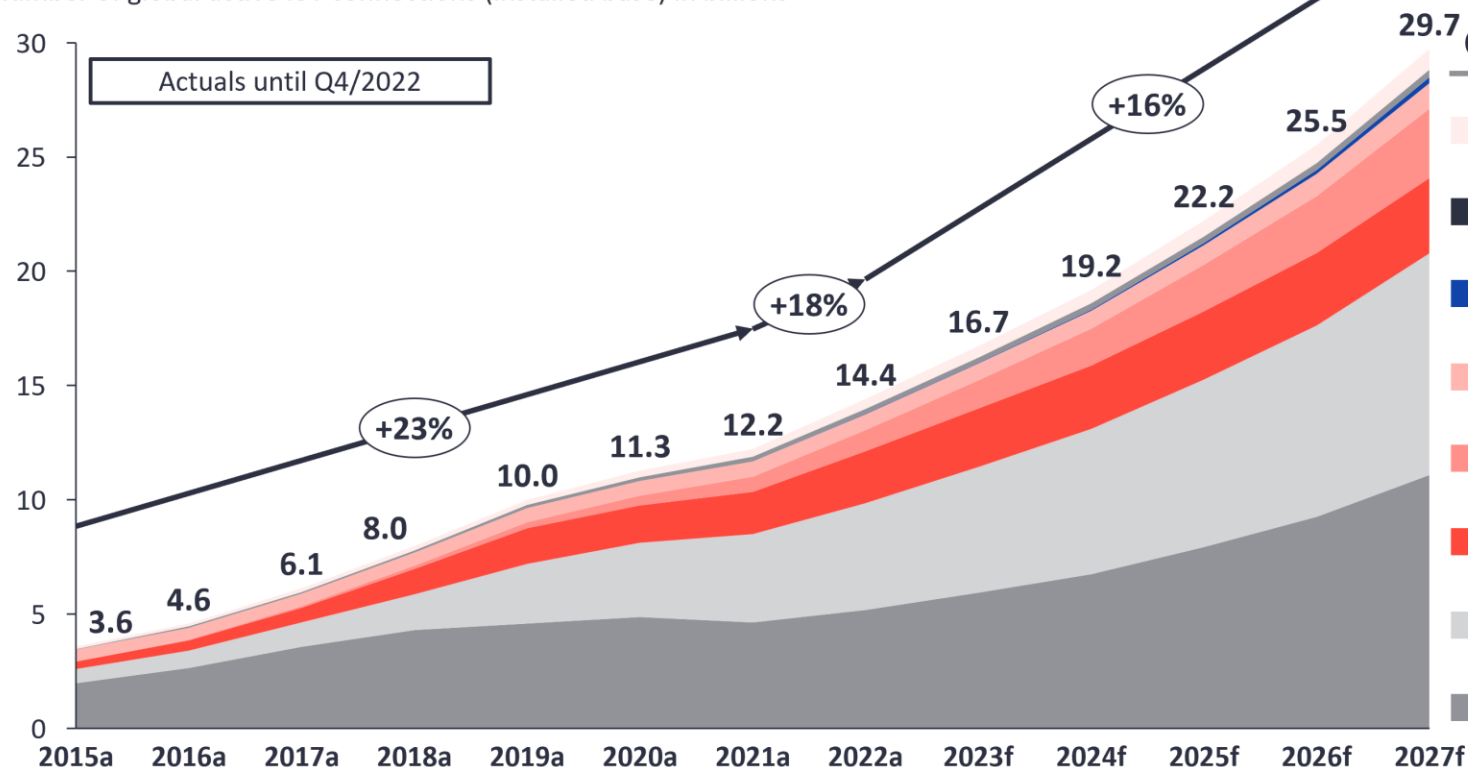


Note: Non-IoT includes all mobile phones, tablets, PCs, laptops, and fixed line phones. IoT includes all consumer and B2B devices connected – see IoT break-down for further details

Source(s): IoT Analytics - Cellular IoT & LPWA Connectivity Market Tracker 2010-25

Global IoT market forecast (in billions of connected IoT devices)

Number of global active IoT connections (installed base) in billions



Connectivity type	CAGR 21–22	CAGR 22–27
Other	21%	17%
Wireless Neighborhood Area Networks (WNAN)	15%	8%
Cellular 5G IoT	200%	87%
Wired IoT	5%	10%
LPWA	38%	27%
Cellular IoT (excl. 5G, LPWA)	22%	8%
Wireless Local Area Networks (WLAN)	21%	16%
Wireless Personal Area Networks (WPAN)	12%	16%

xx% = CAGR

Note: IoT connections do not include any computers, laptops, fixed phones, cellphones, or consumers tablets. Counted are active nodes/devices or gateways that concentrate the end-sensors, not every sensor/actuator. Simple one-directional communications technology not considered (e.g., RFID, NFC). Wired includes ethernet and fieldbuses (e.g., connected industrial PLCs or I/O modules); Cellular includes 2G, 3G, 4G, 5G; LPWA includes unlicensed and licensed low-power networks; WPAN includes Bluetooth, Zigbee, Z-Wave or similar; WLAN includes Wi-Fi and related protocols; WNAN includes non-short-range mesh, such as Wi-SUN; Other includes satellite and unclassified proprietary networks with any range.

Source: IoT Analytics Research 2023. We welcome republishing of images but ask for source citation with a link to the original post and company website.

Why is IoT data collection impacting big data ?

- The data generated by IoT devices is, in many ways, much than other types of data
- Most data is wasted without automatically aggregating and analyzing
- Most IoT platforms need to collect and analyze data in real-time



IoT and Big Data are

two independent technologies that are **inseparable** from each other, to enable well-known technological advances.

While the IoT would largely collect data from physical objects through different sensors, **Big Data** would allow faster and more efficient storage and processing of this data.







Easily connect your system
to a home network



Know when doors and windows
are opened and closed



Remotely monitor movement
inside your home



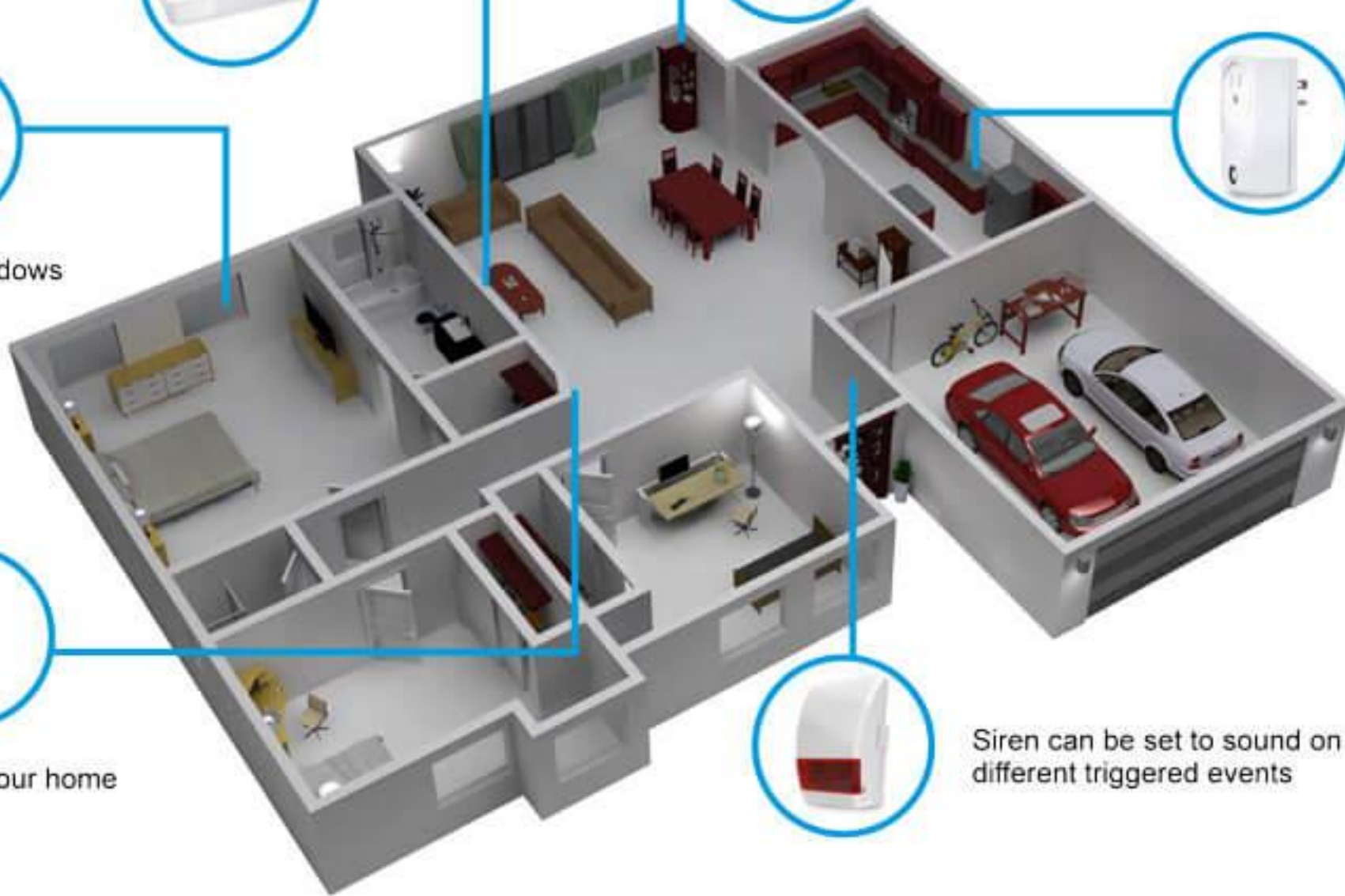
Turn devices on and
off from your smart
phone / tablet



Keep an eye on your home
while you're away

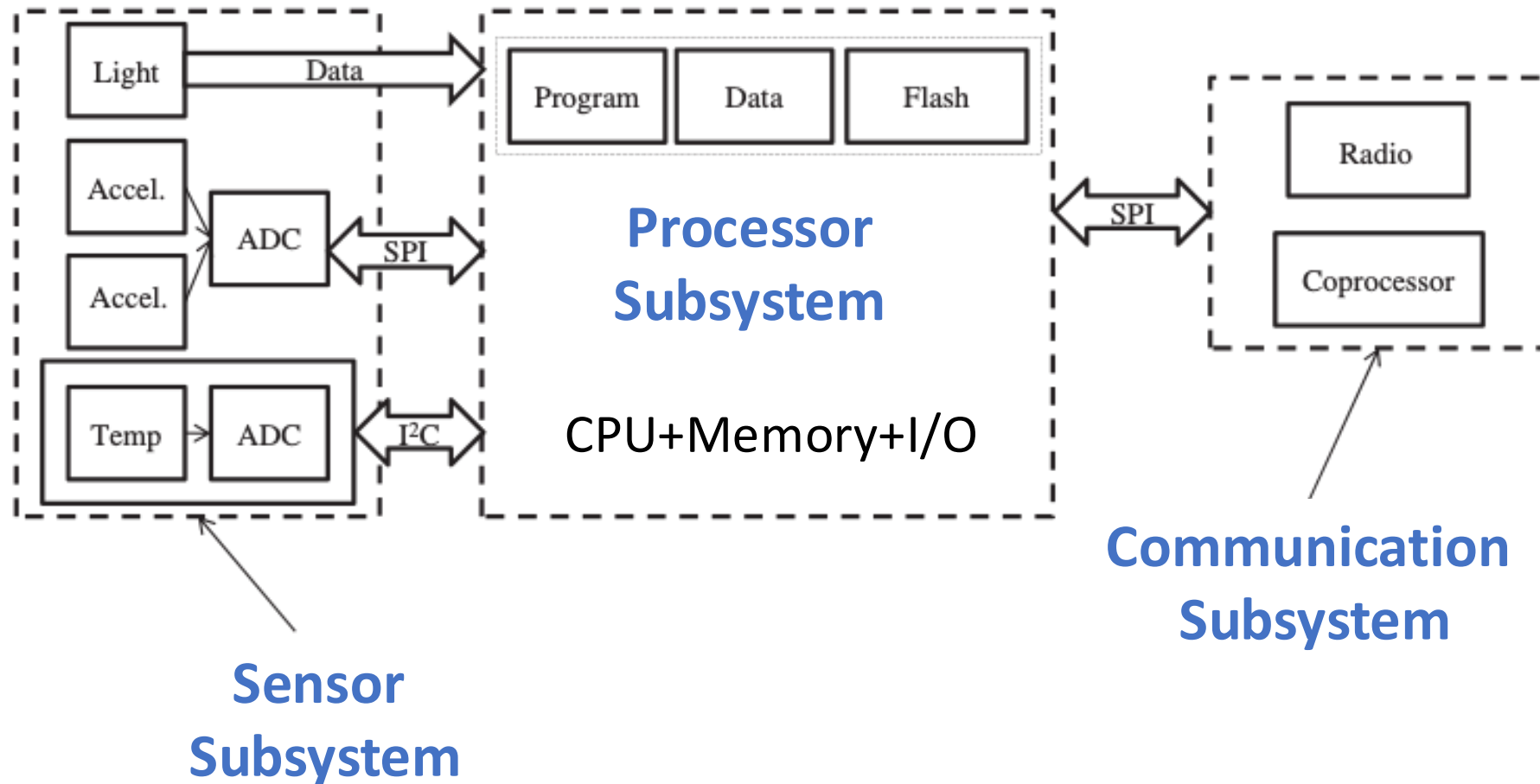


Siren can be set to sound on
different triggered events

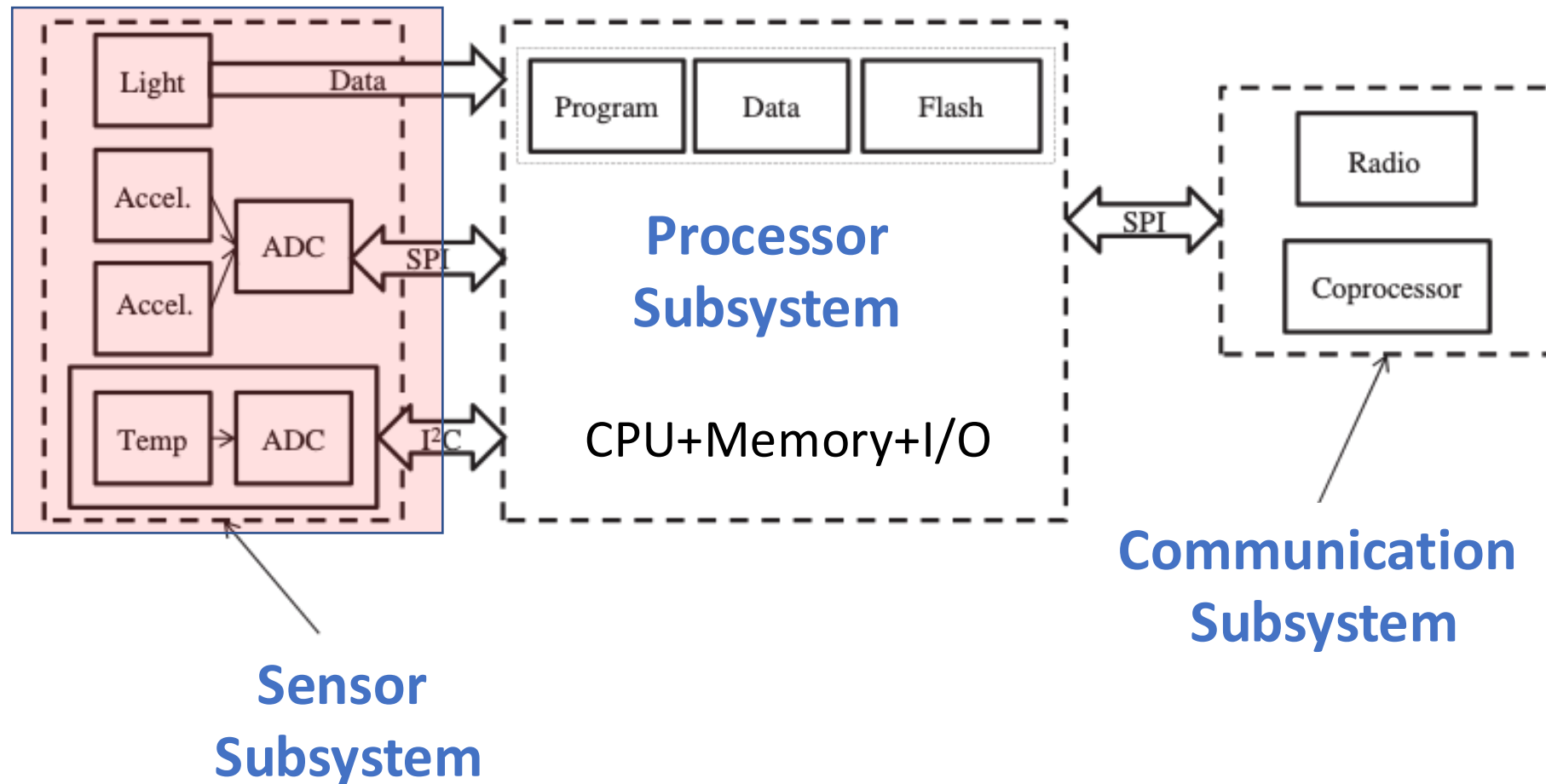




Basic IoT Sensor Node Architecture

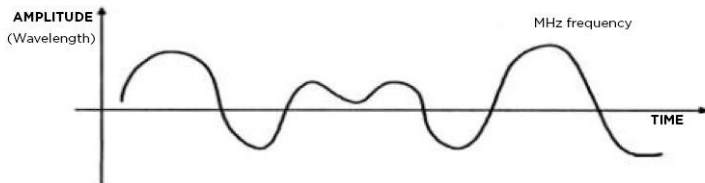


Basic IoT Sensor Node Architecture



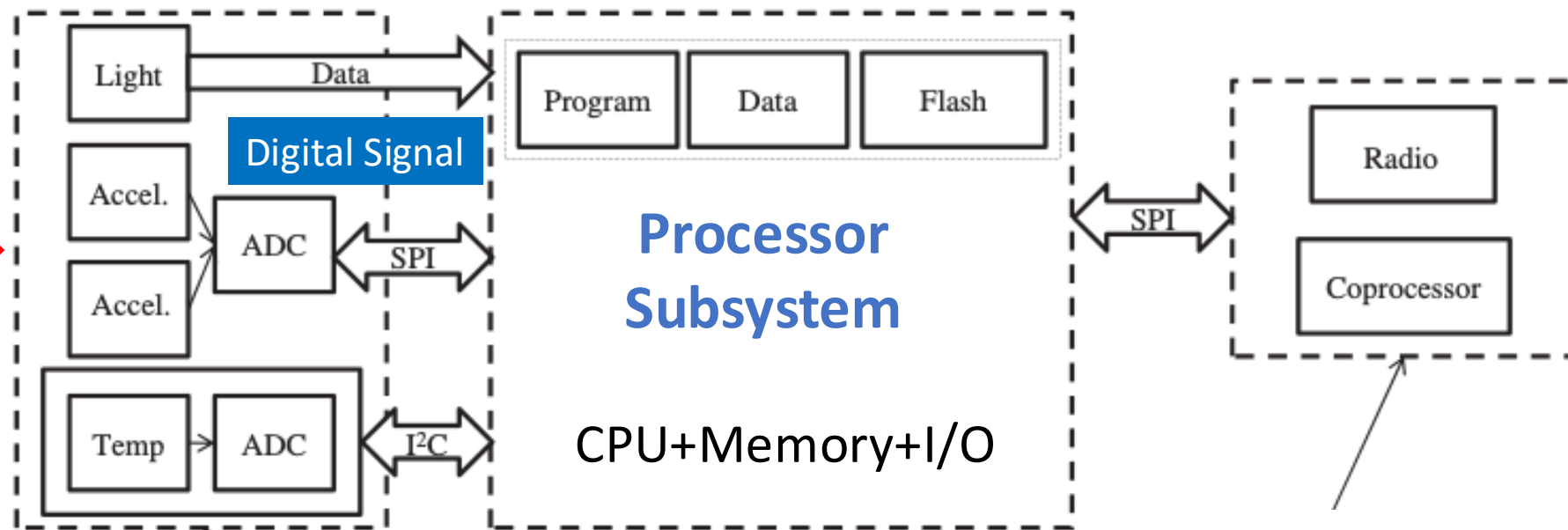
Basic IoT Sensor Node Architecture

ANALOG SIGNAL

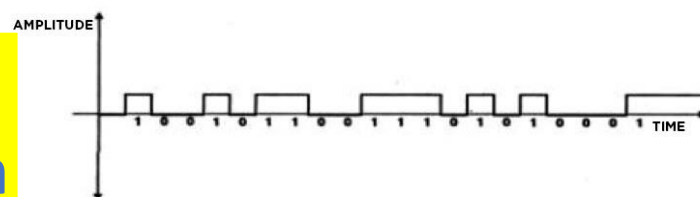


Physical Environment

Analog signal



DIGITAL SIGNAL



Sensor Subsystem

Communication Subsystem

Temperature sensor



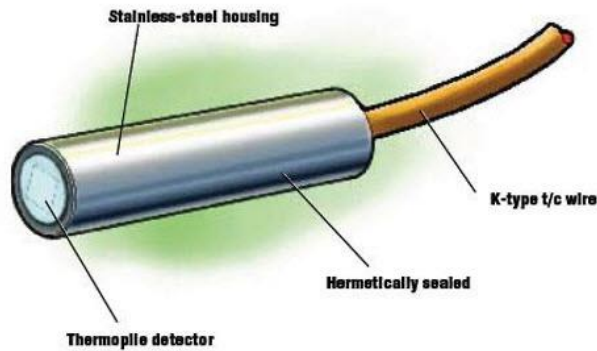
35

°C | °F

โอกาสฝนตก: 1%
ความชื้น: 48%
ลม: 10 กม./ชม.

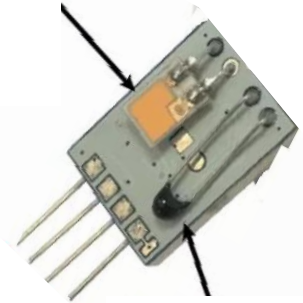


DS18B20
(a)

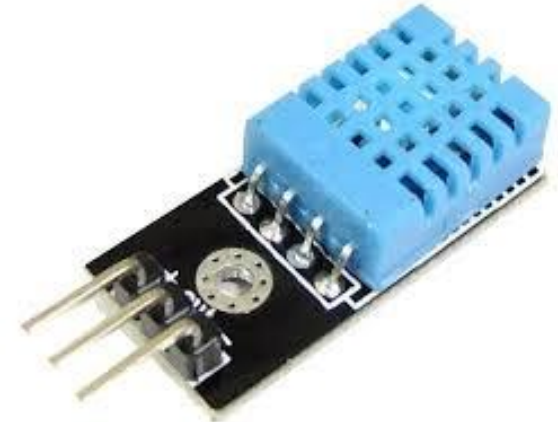


LM35
(b)

Humidity Sensing
component

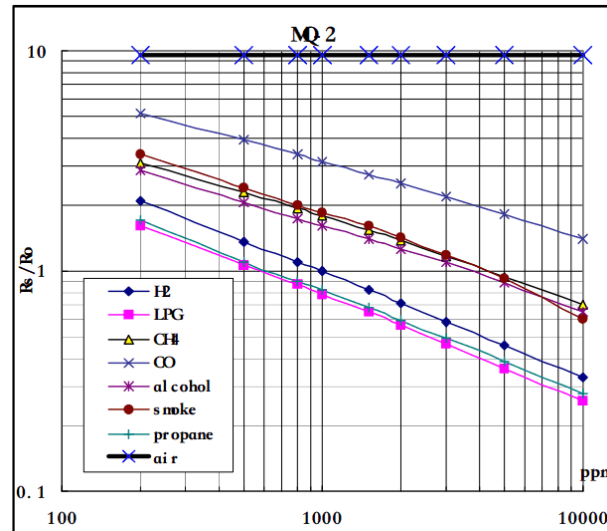


Temperature Sensing
component
(thermistor)



DHT11
(c)

Air Quality Sensor



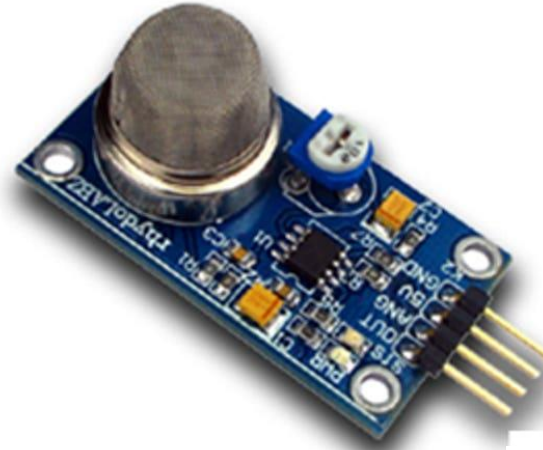
MQ2 Gas Sensor Module

MQ6 carbon monoxide

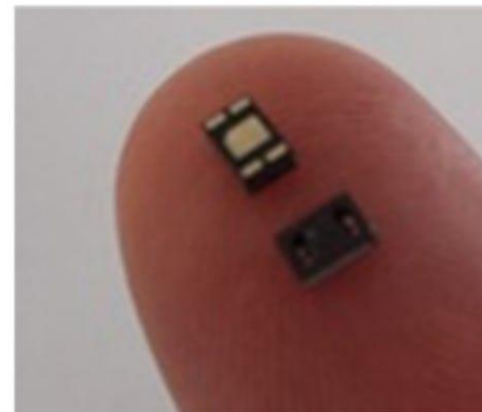
MQ135 Air Quality Sensor

EC – Electrical Conductivity Sensor

Traditional MOS sensor



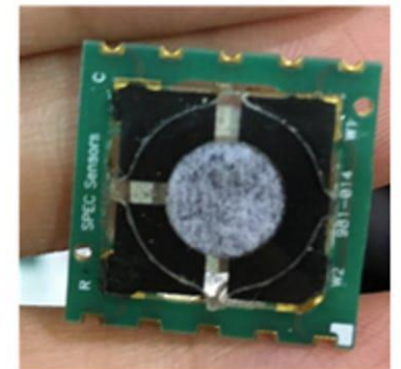
Miniaturized MOS sensor



Traditional EC sensor



Miniaturized EC sensor

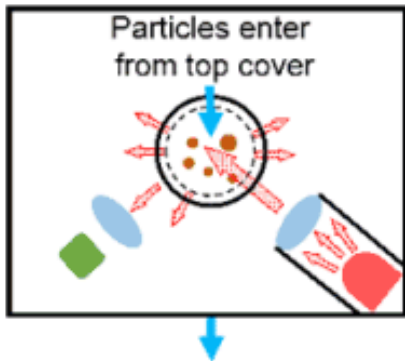


Laser Dust Sensor – measure the particle concentration

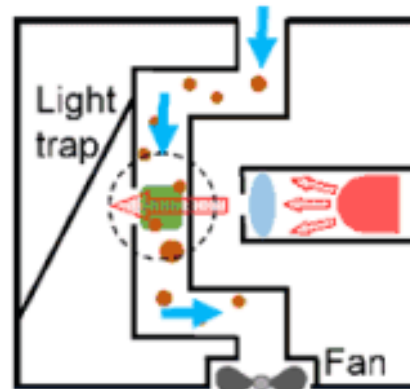
PM2.5 Particle Sensor



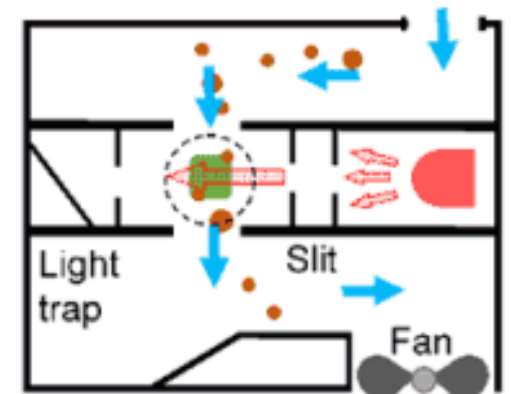
Sharp
GP2Y1014AU



Honeywell
PM2.5 Particle Sensor Module



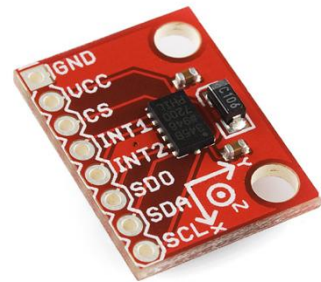
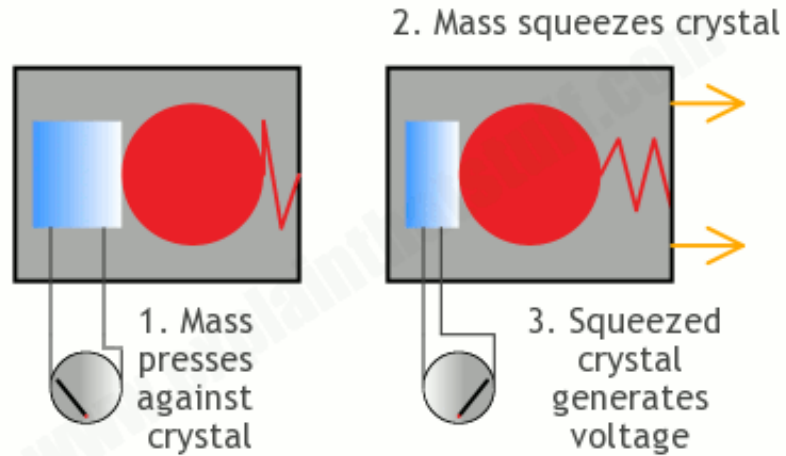
Plantower
PMS1003



Accelerometer

Piezoelectric accelerometer

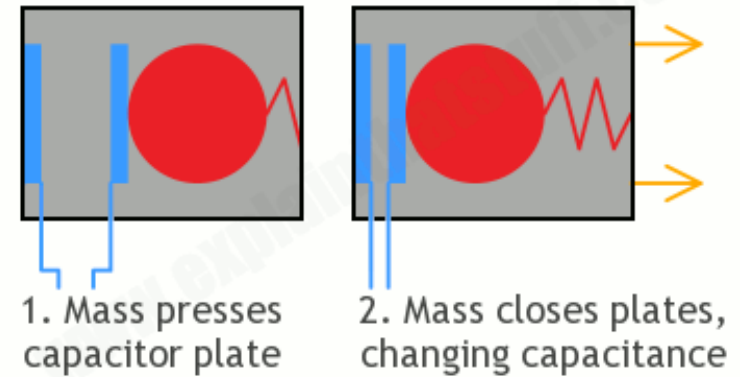
www.explainthatstuff.com



SparkFun's MMA8452Q
Triple Axis Accelerometer

Capacitive accelerometer

www.explainthatstuff.com

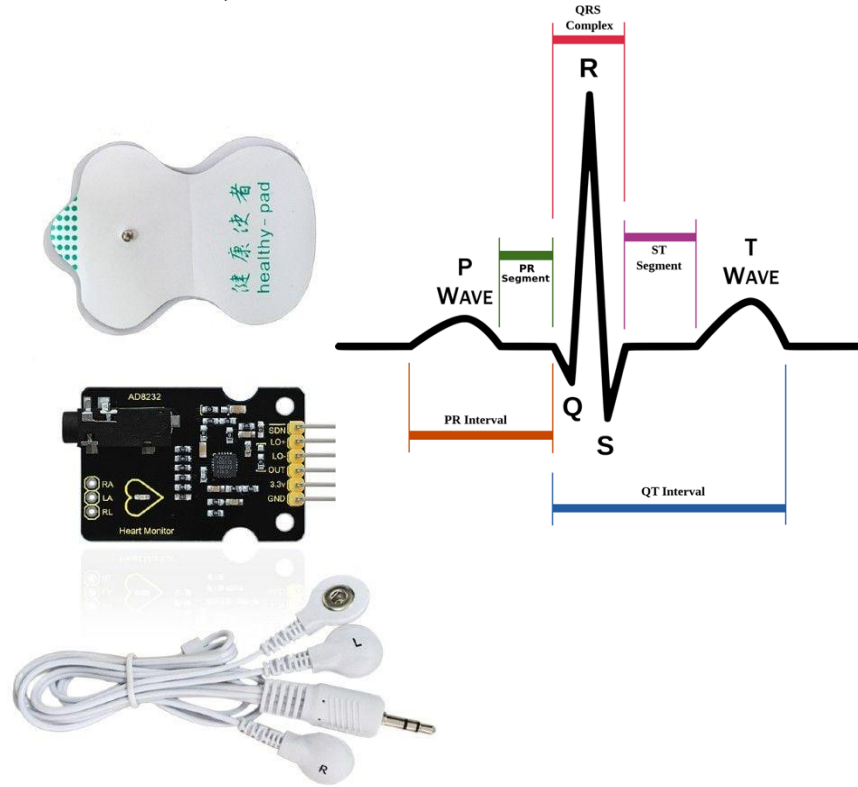


WitMotion BWT61 Bluetooth
MPU6050 Angle+Gyro+Acceleration

Heart Rate Monitor Sensor

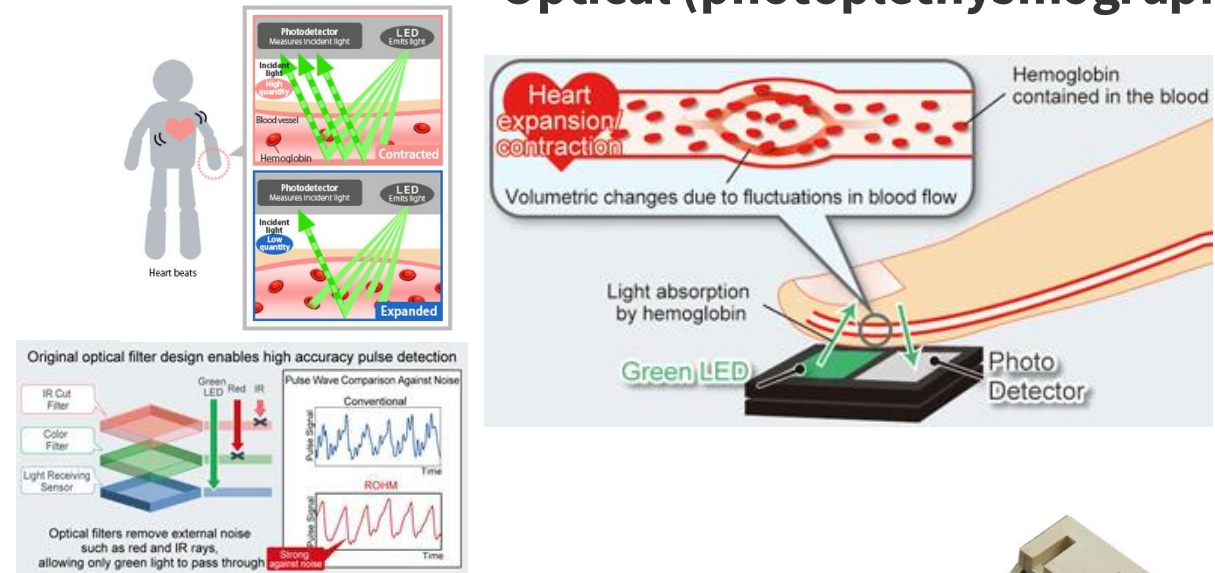


Electrical (electrocardiography)



ECG AD8232

Optical (photoplethysmography)



Pulse Sensor's
Heart rate Monitor

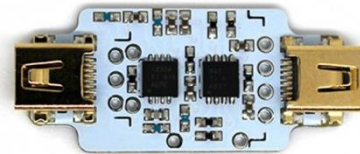


Gravity's
Optical Heart Rate
Monitor Sensor

Brainwave Sensor



MikroElektronika MIKROE-3359



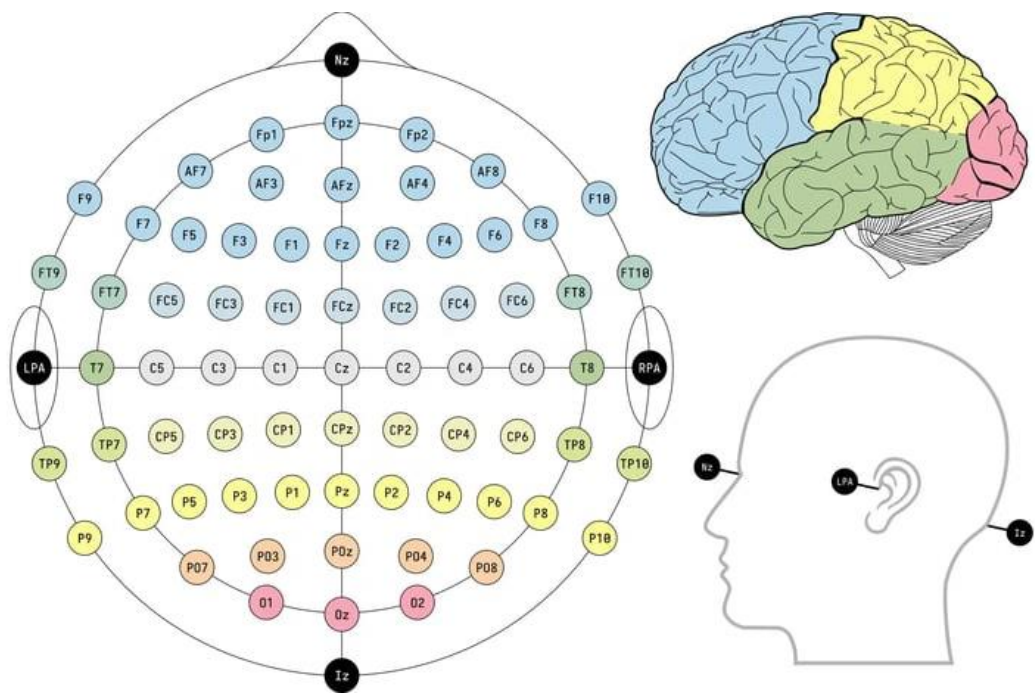
Human Brainwaves

Gamma 35+ Hz		Heightened Awareness
Beta 13 - 35 Hz		Problem-Solving
Alpha 8 - 13 Hz		Relaxed Reflection
Theta 4 - 8 Hz		Meditation & Creativity
Delta 0.5 - 4 Hz		Deep Sleep



10 μ V ~ 100 μ V and 0.1 ~ more than 100 Hz

BITalino - Electroencephalography (EEG) Sensor



The 10-20 system for EEG



GNSS receiver

Tracks GPS, GLONASS, Beidou, Galileo and SBAS satellites.
External antenna with MCX connector

RC I/O co-processor

Accepts PPM/SBUS input and provides 14 PWM output channels for motors and servos

Dual IMU

Accelerometers, gyroscopes and magnetometers for orientation and motion sensing



Navio2 Drone controller with raspberry pi

Triple redundant power supply

With overvoltage protection and power module port for voltage and current sensing

Extension ports

Exposed ADC, I2C and UART interfaces for sensors and radios

High resolution barometer

Senses altitude with 10 cm resolution

Examples of Sensors and Applications

Sensor	Application Area	Sensed Event	Explanation
Accelerometer	Active volcano monitor AVM	2D and 3D acceleration of movements of people and objects	Volcano activities
	SHM Structural health monitor		Stiffness of a structure
	Health care		Stiffness of bones, limbs, joints; Motor fluctuation in Parkinson's disease
	Transportation		Irregularities in rail, axle box or wheels of a train system
	SCM supply chain management		Defect of fragile objects during transportation
Acoustic emission sensor	SHM	Elastic waves generated by the energy released during crack propagation	Measures micro-structural changes or displacements
Acoustic sensor	Transportation & Pipelines	Acoustic pressure vibration	Vehicle detection; Measure structural irregularities; Gas contamination
Capacitance sensor	PA	Solute concentration	Measure the water content of a soil

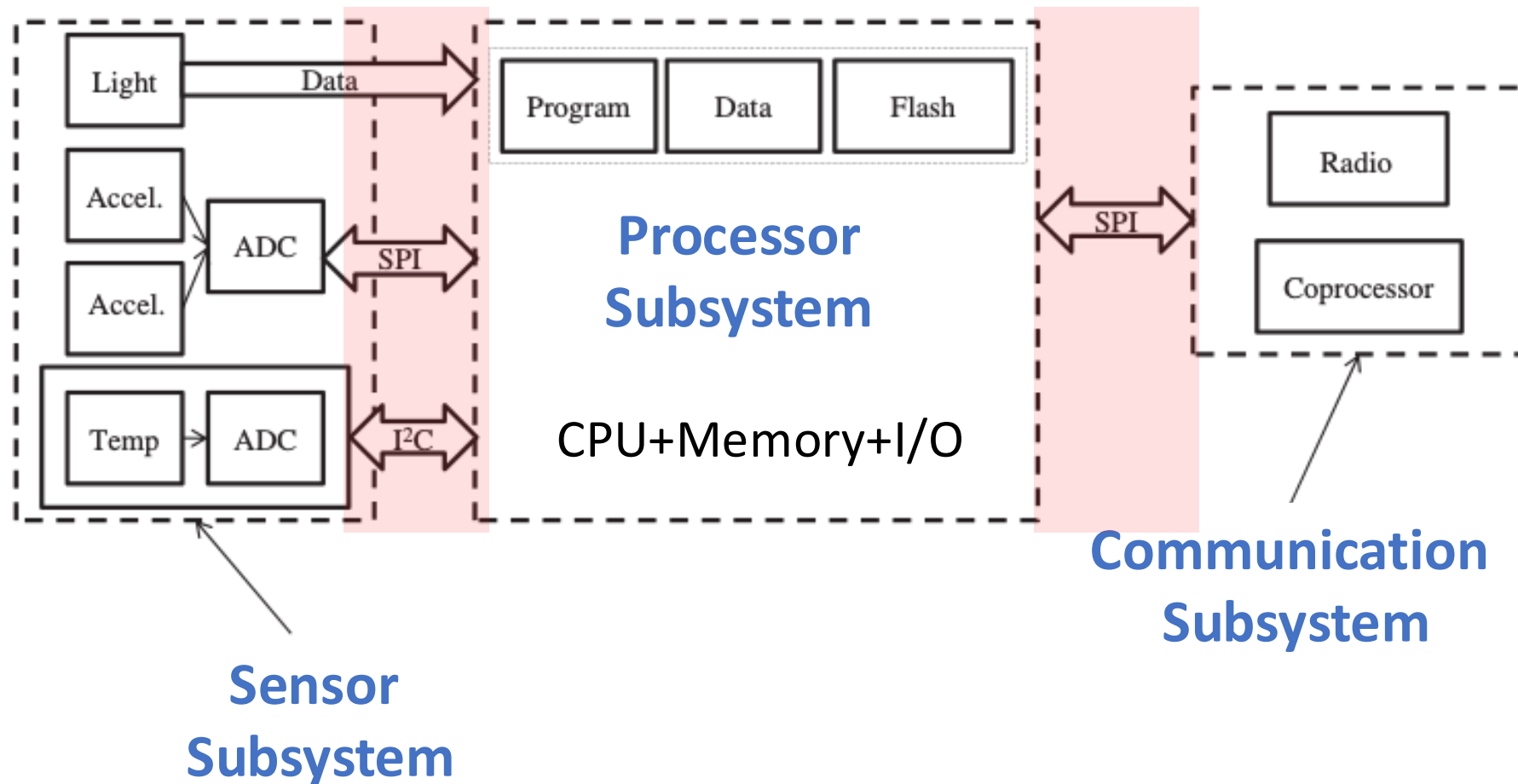
Examples of Sensors and Applications

Sensor	Application Area	Sensed Event	Explanation
ECG	Health care	Heart rate	
EEG		Brain electrical activity	
EMG		Muscle activity	
Electrical sensors	PA	Electrical capacitance or inductance affected by the composition of tested soil	Measure of nutrient contents and distribution
Gyroscope	Health care	Angular velocity	Detection of gait phases
Humidity sensor	PA & HM	Relative and absolute humidity	
Infrasonic sensor	AVM	Concussive acoustic waves – earth quake or volcanic eruption	
Magnetic sensor	Transportation	Presence, intensity, direction, rotation and variation of a magnetic field	Presence, speed and density of a vehicle on a street; congestion
Oximeter	Health care	Blood oxygenation of patient's hemoglobin	Cardiovascular exertion and trending of exertion relative to activity
pH sensor	Pipeline (water)	Concentration of hydrogen ions	Indicates the acid and alkaline content of a water measure of cleanliness

Examples of Sensors and Applications

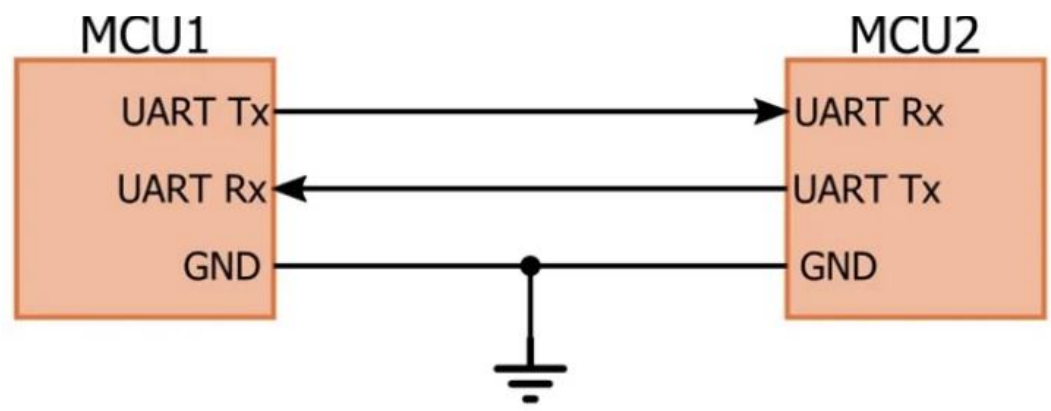
Sensor	Application Area	Sensed Event	Explanation
Photo acoustic spectroscopy	Pipeline	Gas sensing	Detects gas leak in a pipeline
Piezoelectric cylinder	Pipeline	Gas velocity	A leak produces a high frequency noise that produces a high frequency noise that produces vibration
Soil moisture sensor	PA	Soil moisture	Fertilizer and water management
Temperature sensor	PA & HM	Pressure exerted on a fluid	
Passive infrared sensor	Health care & HM	Infrared radiation from objects	Motion detection
Seismic sensor	AVM	Measure primary and secondary seismic waves (Body wave, ambient vibration)	Detection of earth quake
Oxygen sensor	Health care	Amount and proportion of oxygen in the blood	
Blood flow sensor	Health care	The Doppler shift of a reflected ultrasonic wave in the blood	

Communication Protocol



1.Universal Asynchronous Receiver/Transmitter (UART)

UART is a serial communication protocol between MCU/MCU or MCU/device by using 2 crossed wires as transmitter line and receiver line (full duplex)



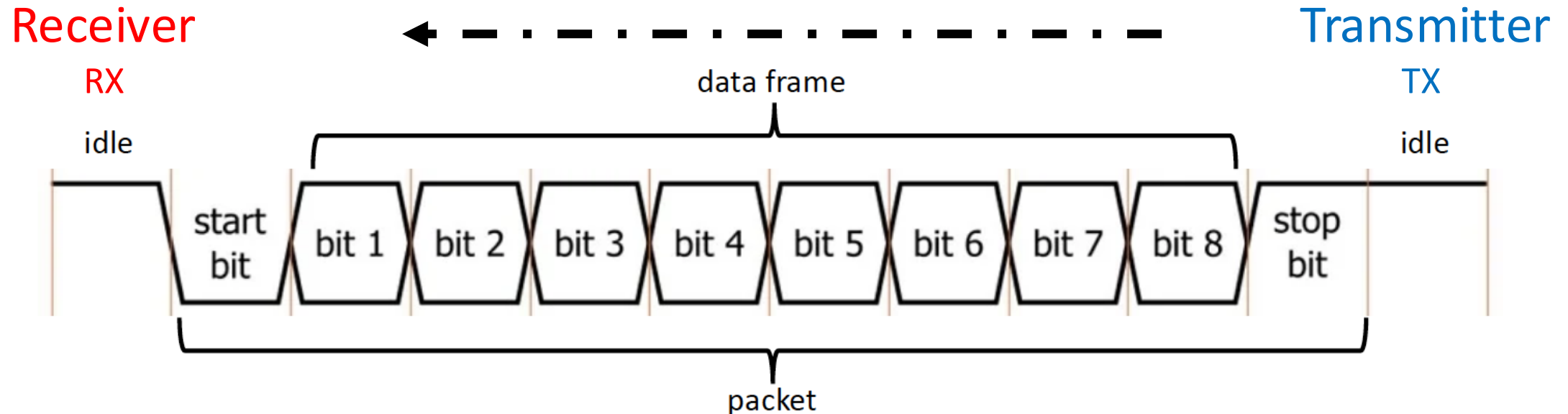
Wires Used	2
Maximum Speed	Any speed up to 115200 baud, usually 9600 baud
Synchronous or Asynchronous?	Asynchronous
Serial or Parallel?	Serial
Max # of Masters	1
Max # of Slaves	1

How does UART communicate

Start bit: The first bit that is overhead bit for a one-byte UART transmission. It indicates that the data line is leaving its idle state (Ready to send).

The idle state is typically logic high, so the start bit is logic low.

Stop bit: The last bit that is overhead bit for a one-byte UART transmission. Its logic level is the same as the signal's idle state

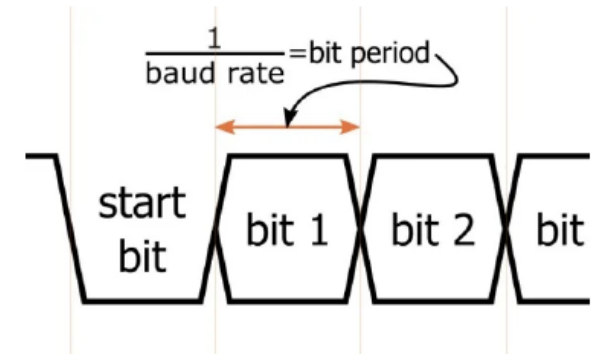


Baud Rate: The rate at which the **signal** changes.

Bit Rate: The number of **bits** transmitted per second.

Since UART uses 1 bit per symbol, so **baud rate $\approx$ bit rate**.

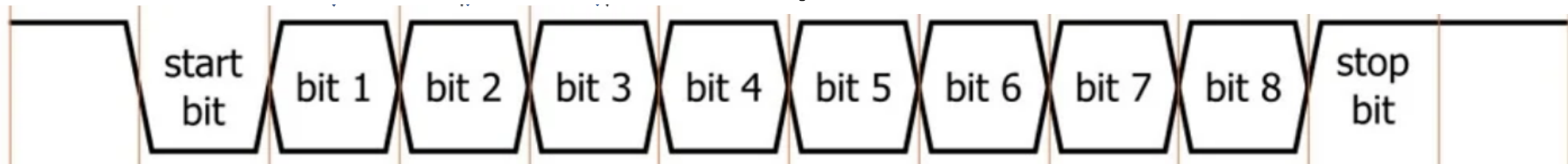
For example, a 9600-baud transmission means one bit requires $1/(9600 \text{ bps}) \approx 104.2 \mu\text{s}$



Actually, baud rate is the approximate rate because there is an additional time for overhead bits and perhaps for delays between one-byte transmissions

With framing overhead (1 start + 8 data + 1 stop = 10 bits):

Effective throughput = $9600 \times (8/10) = 7680$ bps of useful data.



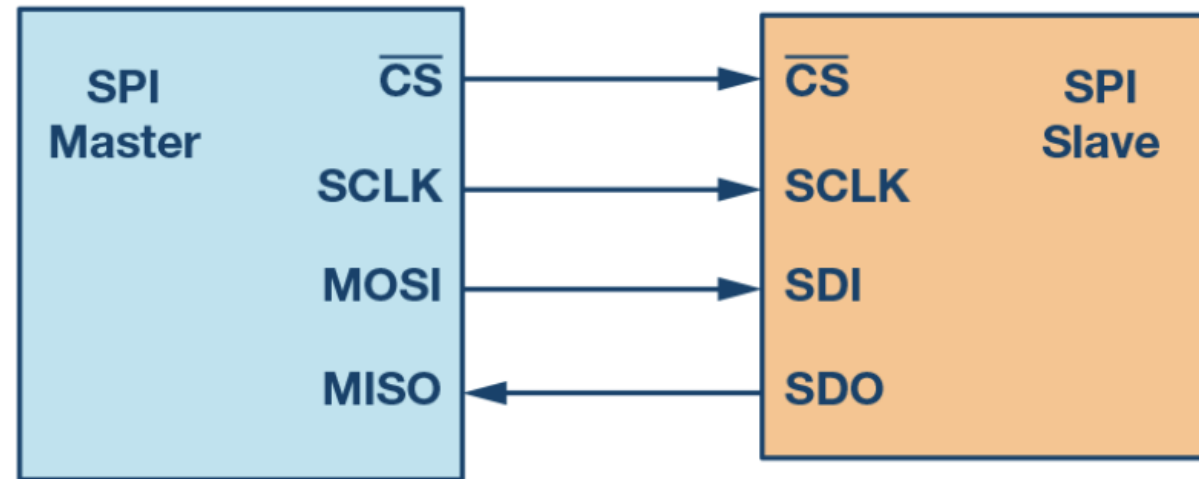
2. Serial Peripheral Interface (SPI)

SPI is one of the most widely used serial-communications protocol specially designed for interfacing between microcontroller and peripheral ICs e.g., sensors, ADCs, DACs.

SPI is a synchronous, full duplex master-slave-based communication that allows a single master device with multiple slave devices.

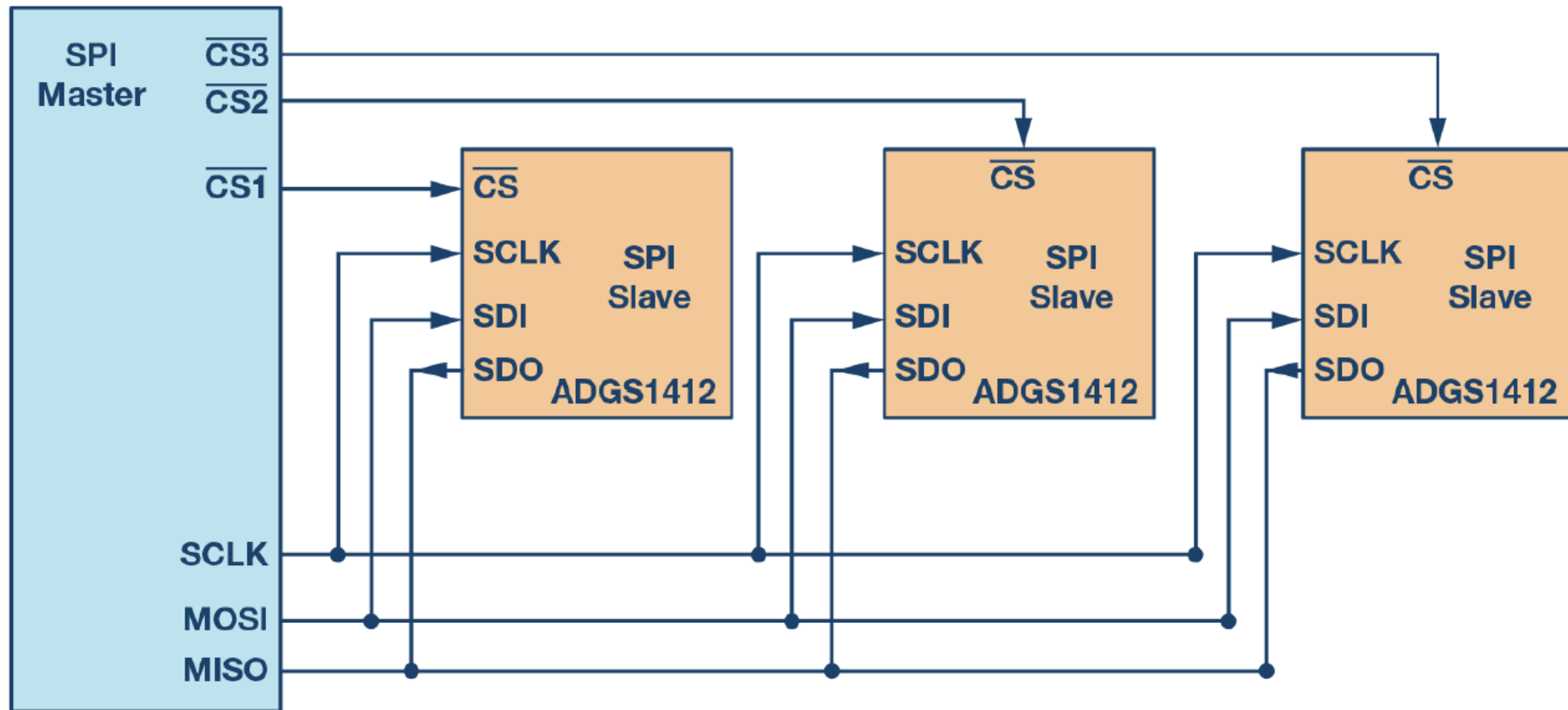
4-wire SPI devices have four signals:

- Clock (SPI CLK, SCLK)
- Chip select (CS)
- Master out, slave in (MOSI)
- Master in, slave out (MISO)

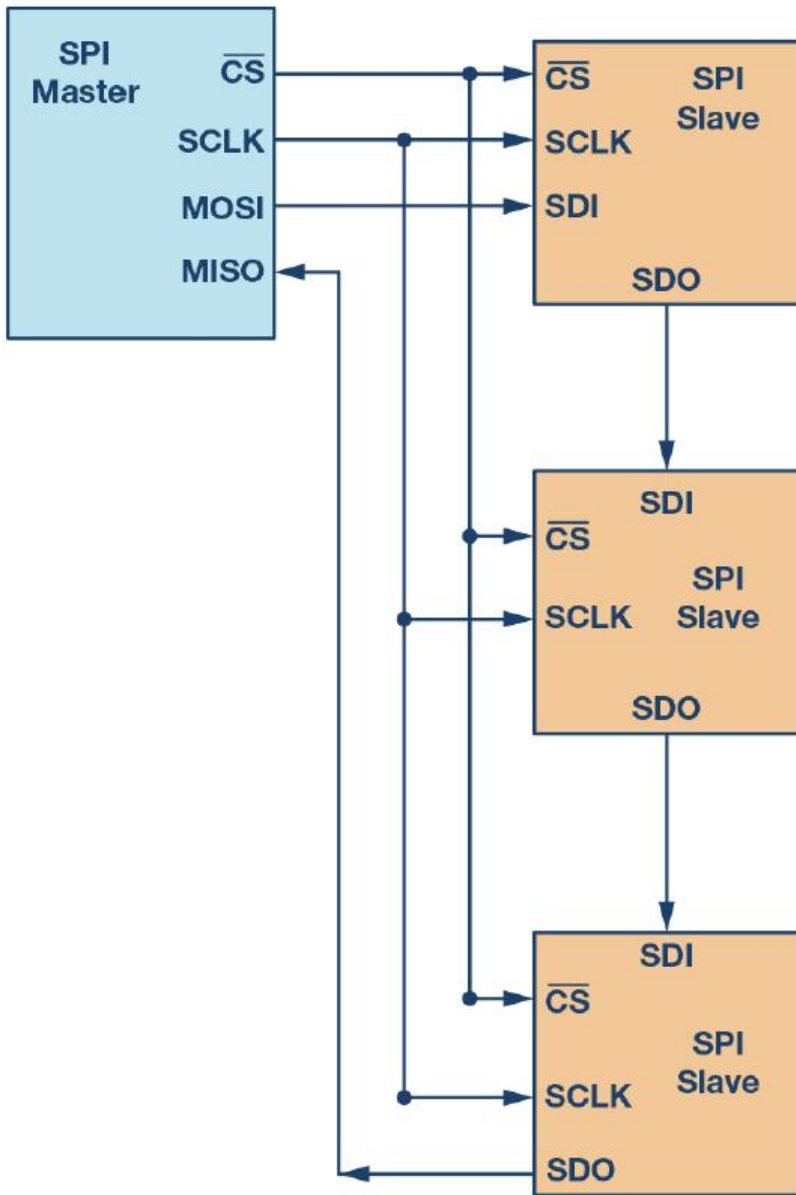


SPI configuration with master and a slave

SPI is typically much faster than I²C due to the simple protocol and, data/clock lines are synchronous, each device requires a unique address wire (CS) .
SPI is basically used in application that speed is needed e.g.; SD cards and display modules, or when information updates and changes quickly.



Multislave SPI configuration



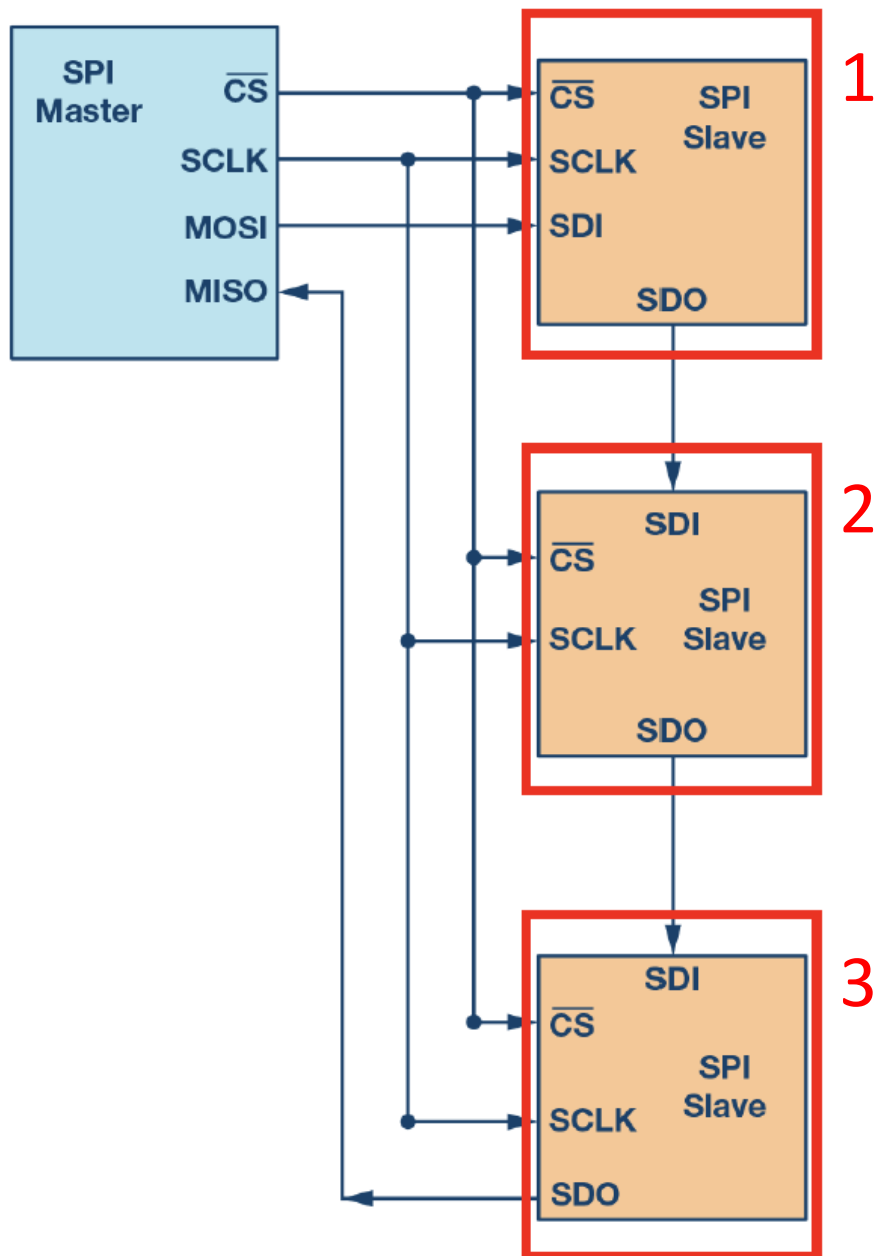
Multislave SPI daisy-chain configuration

Wires Used	4
Maximum Speed	Up to 10 Mbps
Synchronous or Asynchronous?	Synchronous
Serial or Parallel?	Serial
Max # of Masters	1
Max # of Slaves	Theoretically unlimited*

*In practice, the number of slaves is limited by the load capacitance of the system, which reduces the ability of the master to accurately switch between voltage levels.

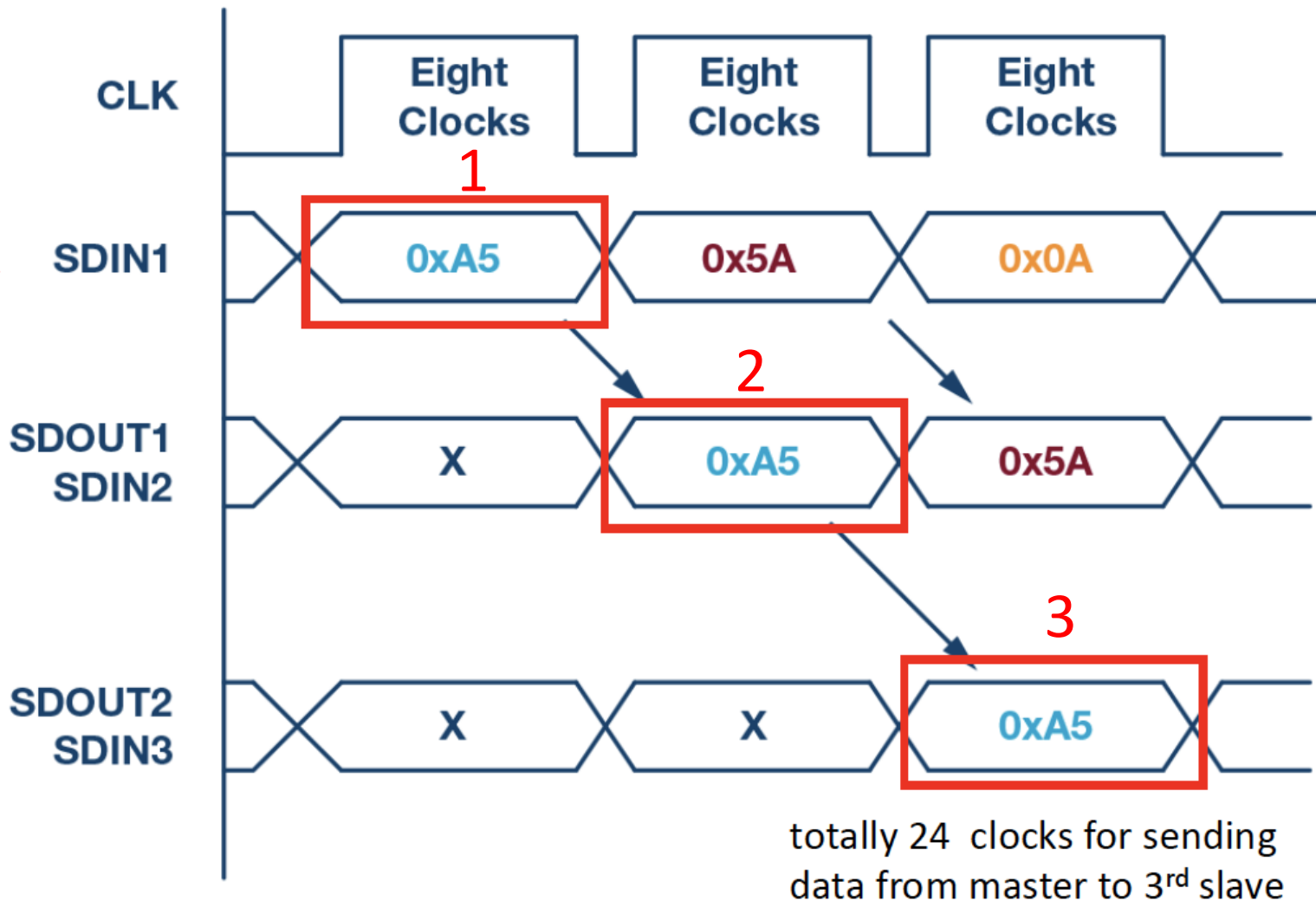
<https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/>

<https://www.analog.com/en/analog-dialogue/articles/introduction-to-spi-interface.html>



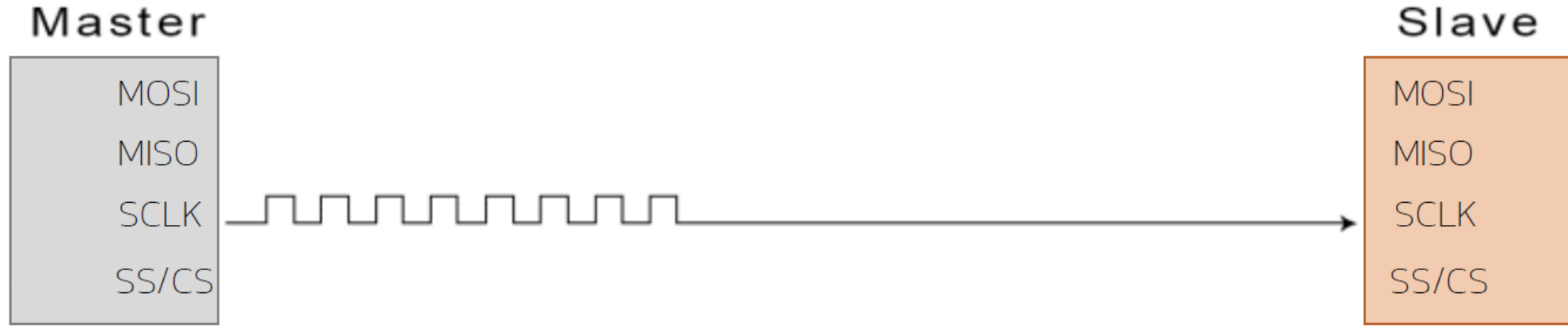
Multislave SPI daisy-chain configuration

But we must wait the signal from the first slave finishing before passing on to the next one.

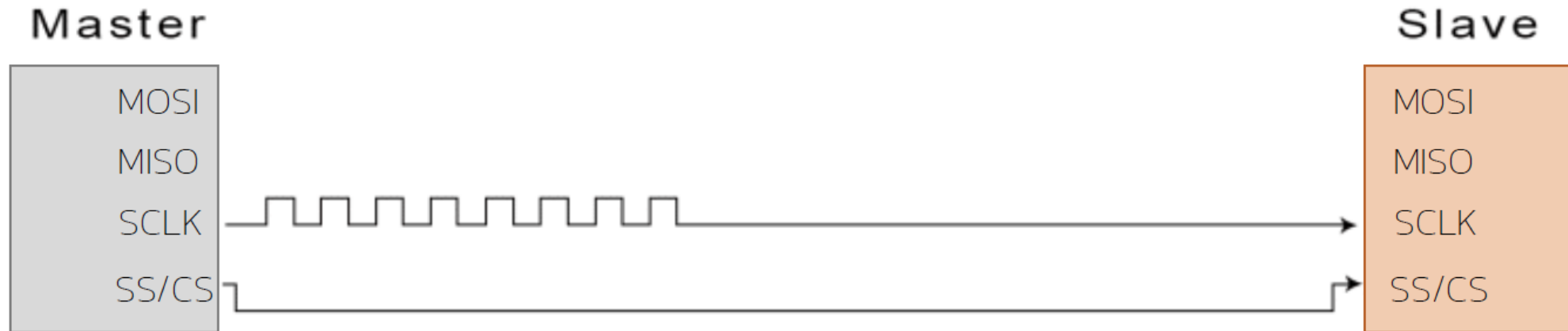


How does SPI communicate

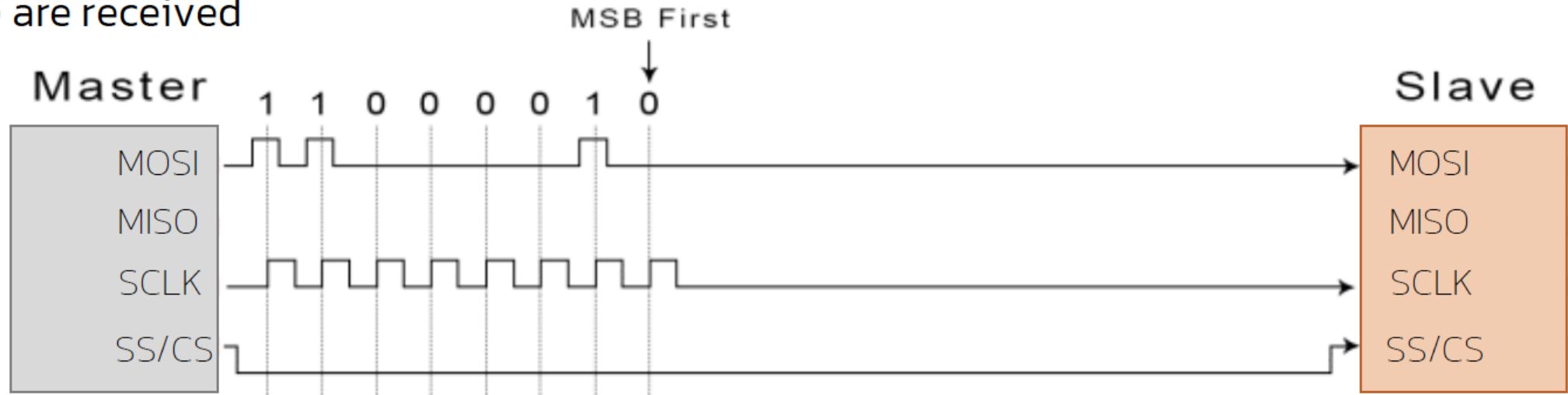
1. The master outputs the clock signal



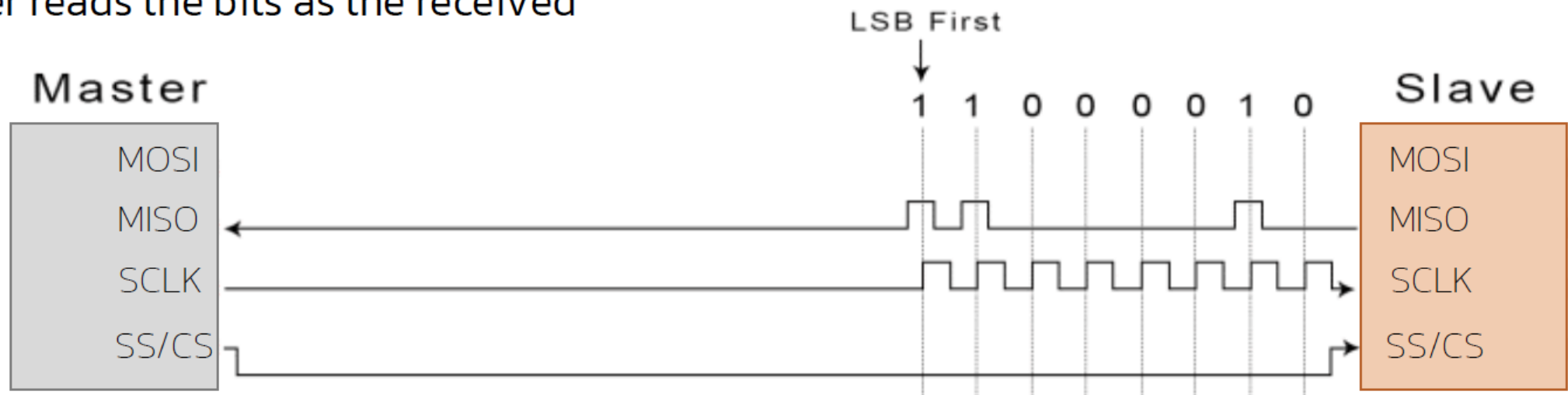
2. The master switches the SS/CS pin to low voltage, which activate the slave



3. the master sends the data one bit at a time to the slave along the MOSI line. The slave reads the bits as the are received



4. If a response is needed, the slave returns data one bit a time to the master along the MISO line. The master reads the bits as the received



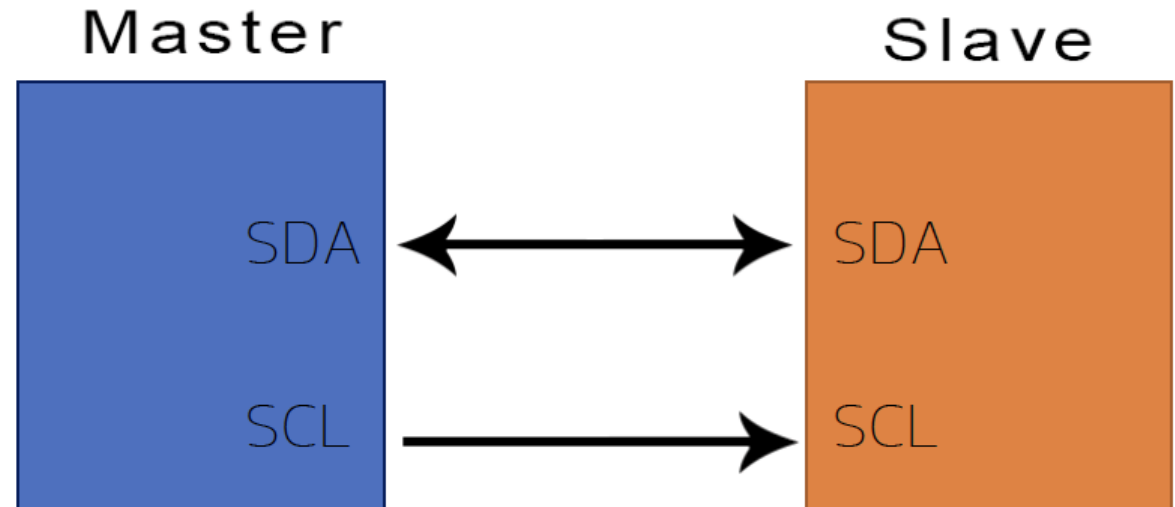
3. Inter-Integrated- Circuit (I²C)

I²C combines the best features of SPI and UARTs. By using only 2-wire communication as UART and additional to multiple slaves to a single master as SPI. Moreover, I²C also provide multiple masters controlling single, or multiple slaves.

2-wire required

SDA (Serial Data) – Data line for the master and slave to send and receive data.

SCL (Serial Clock) – Clock line for control signal.



I²C doesn't need slave select lines like SPI, It select the slave device by addressing in the the first frame after the start bit in a new message.

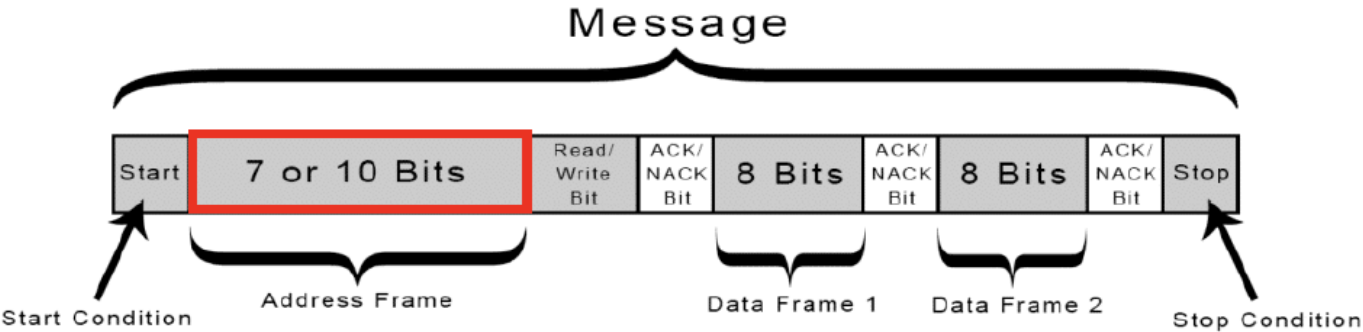
Start Condition: The SDA switches from high to low before the SCL switches from high to low.

Stop Condition: The SDA switches from low to high after the SCL switches from low to high.

Address Frame: A 7 or 10 bit sequence unique to each slave that identifies the slave.

Read/Write Bit: A single bit that be low when the master is sending data to the slave and high when the master read data from devices

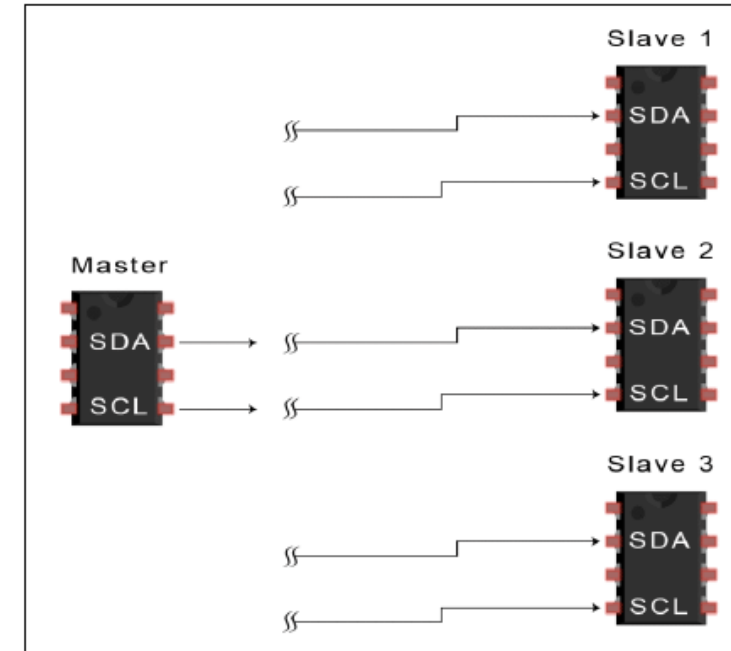
ACK/NACK Bit: Each address/data frame is followed by an acknowledge/no-acknowledge bit. If an address frame or data frame was successfully received, an ACK bit is returned from the receiving device to the sender.



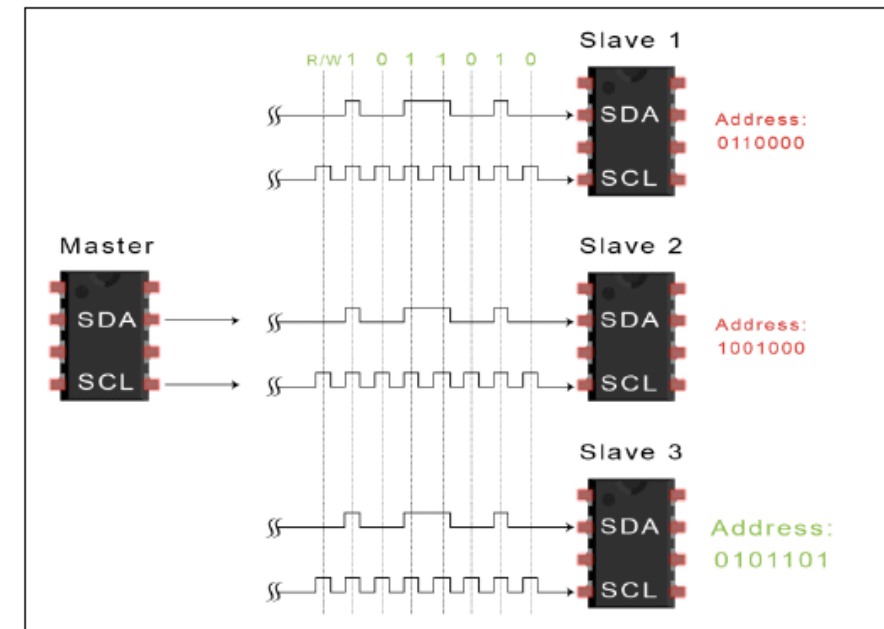
Wires Used	2
Maximum Speed	Standard mode= 100 kbps
	Fast mode= 400 kbps
	High speed mode= 3.4 Mbps
	Ultra fast mode= 5 Mbps
Synchronous or Asynchronous?	Synchronous
Serial or Parallel?	Serial
Max # of Masters	Unlimited
Max # of Slaves	1008

How does I²C communicate

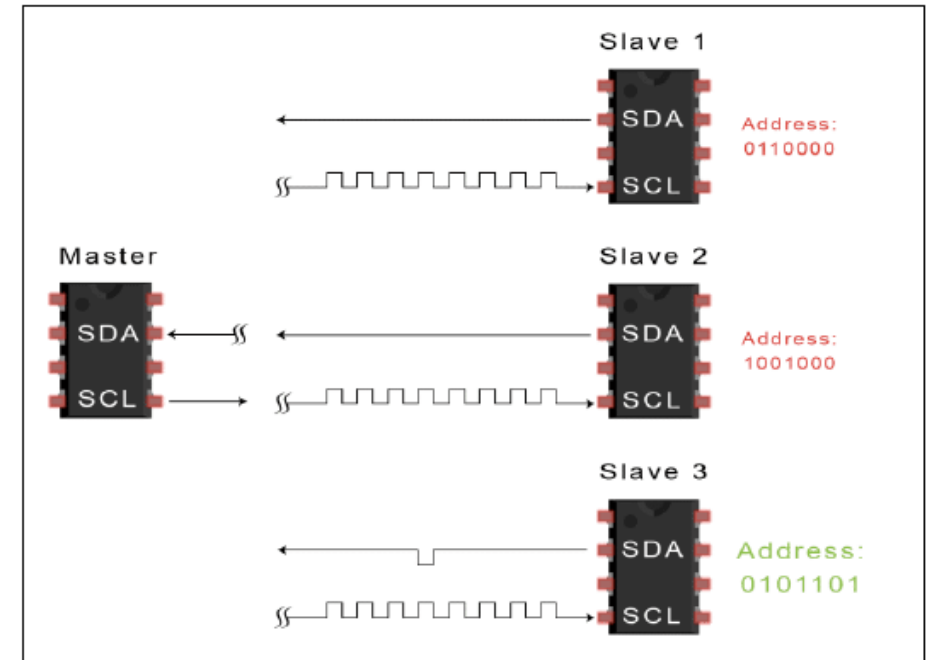
1. The master sends the start condition to every connected slave by switching the SDA line from a high voltage level to a low voltage level *before* switching the SCL line from high to low:



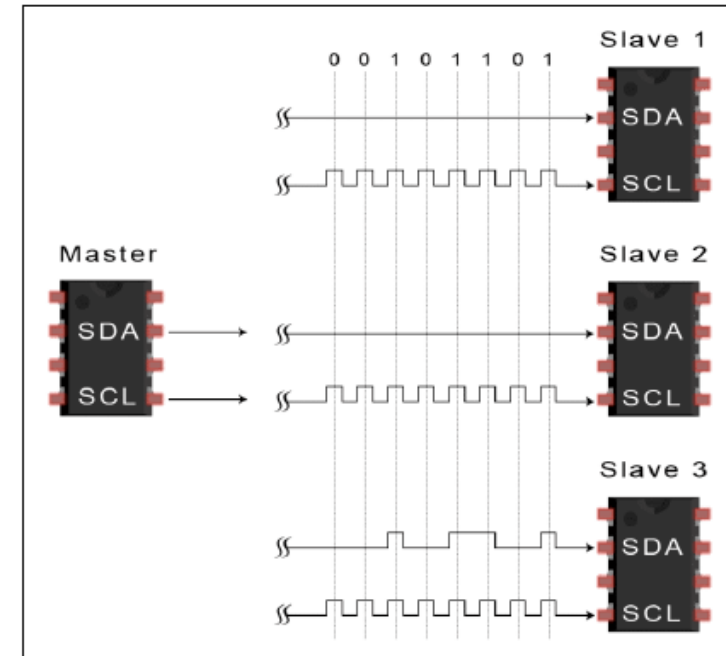
2. The master sends each slave the 7 or 10 bit address of the slave it wants to communicate with, along with the read/write bit:



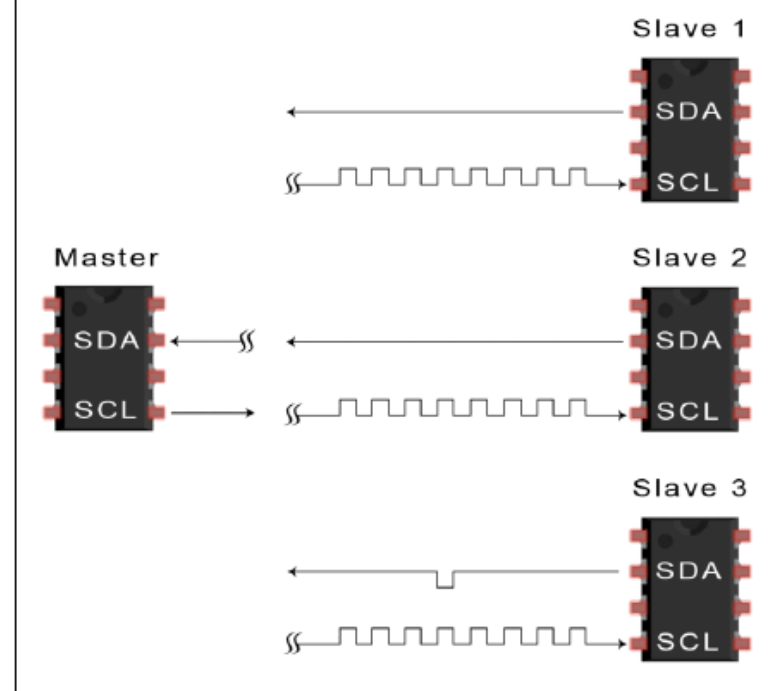
3. Each slave compares the address sent from the master to its own address. If the address matches, the slave returns an ACK bit by pulling the SDA line low for one bit. If the address from the master does not match the slave's own address, the slave leaves the SDA line high



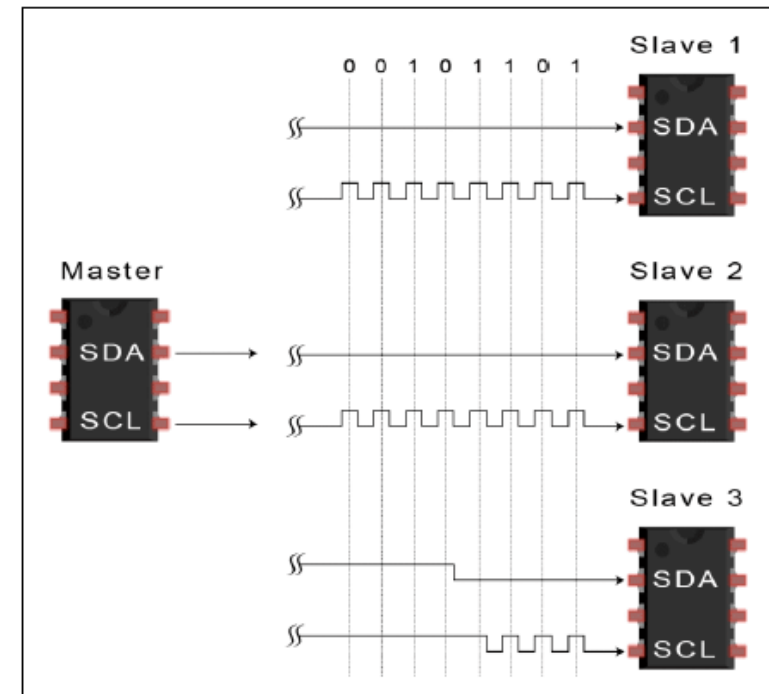
4. The master sends or receives the data frame:



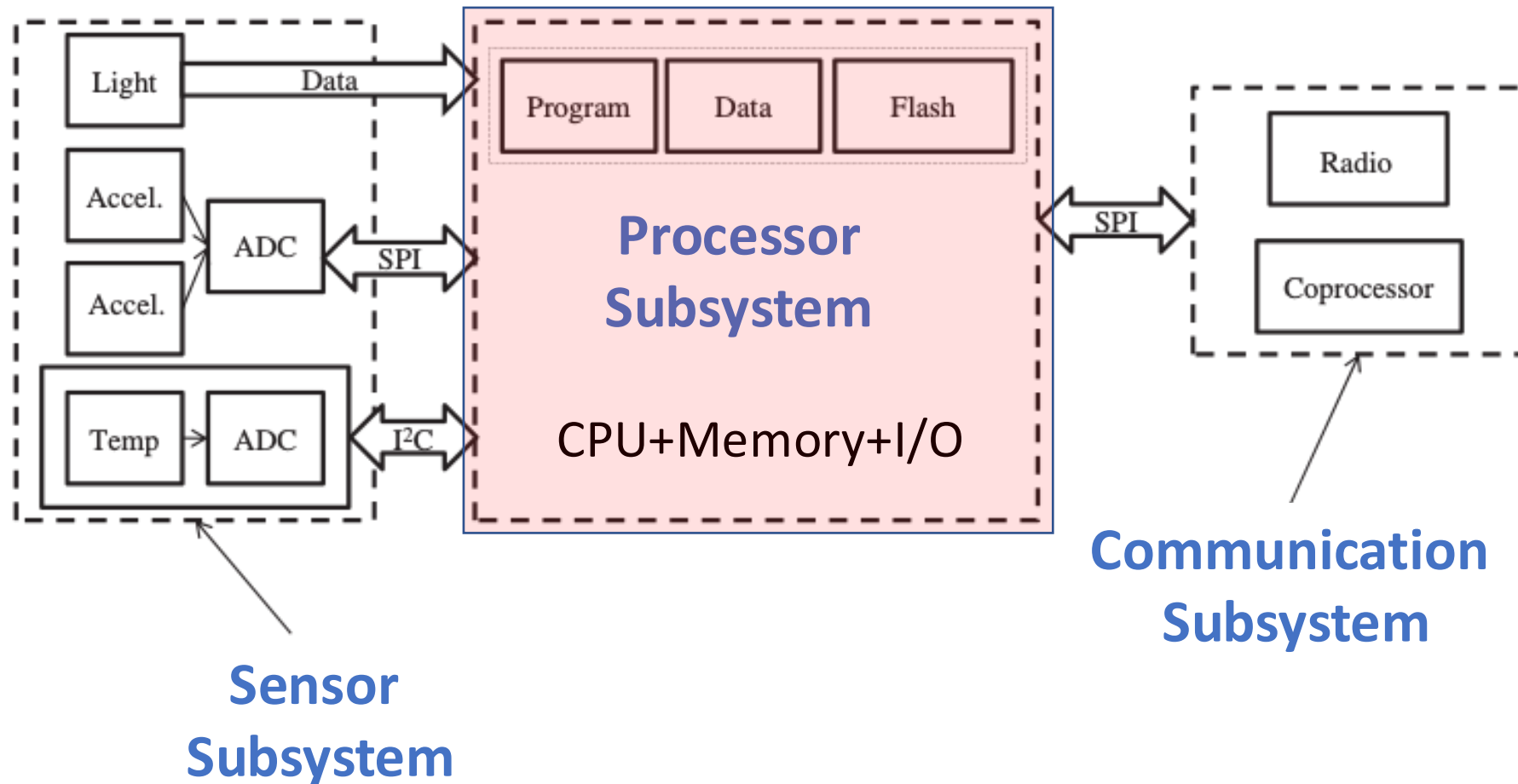
5. After each data frame has been transferred, the receiving device returns another ACK bit to the sender to acknowledge successful receipt of the frame:

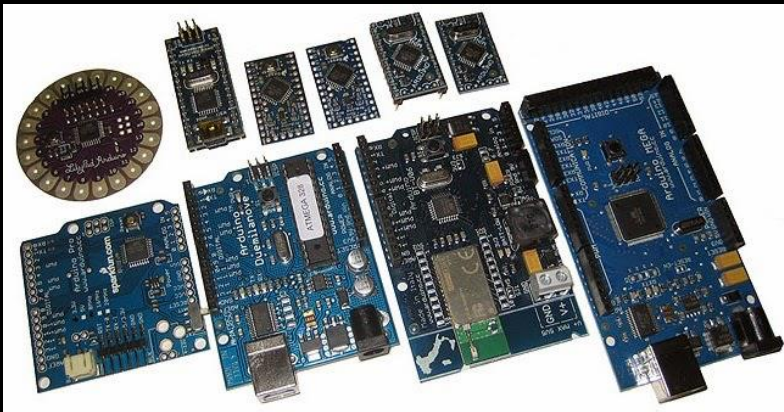


6. To stop the data transmission, the master sends a stop condition to the slave by switching SCL high before switching SDA high:



Basic IoT Sensor Node Architecture





Arduino

- Open-source *microcontroller* platform based on flexible, easy-to-use hardware and software.
 - C-programmable to interact with various forms of I/O to read/write sensor data.
 - I/O functionalities extensible via "*shields*".
- Comes with IDE to facilitate application development

```

Blink | Arduino 1.8.5

Blink $

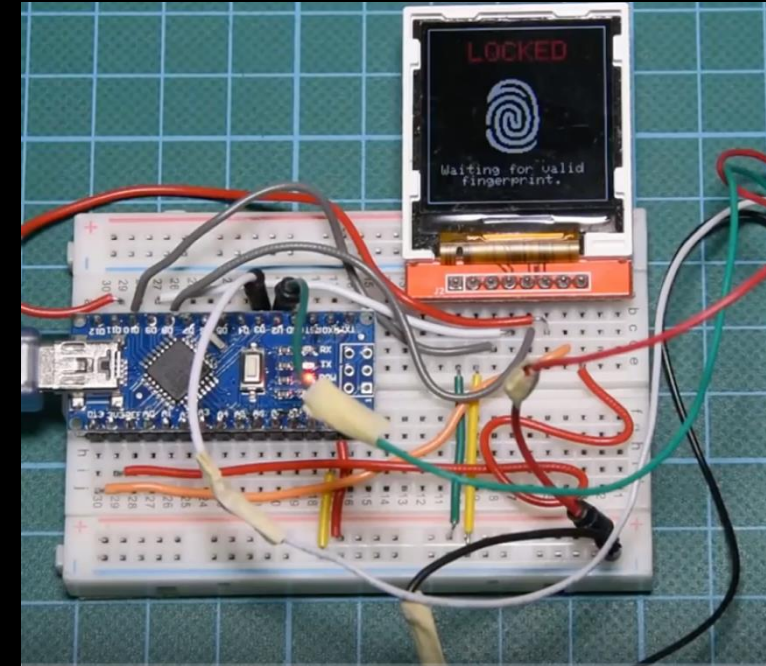
This example code is in the public domain.

http://www.arduino.cc/en/Tutorial/Blink
*/

// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {$
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(1000); // wait for a second
  digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the voltage LOW
  delay(1000); // wait for a second
}

```

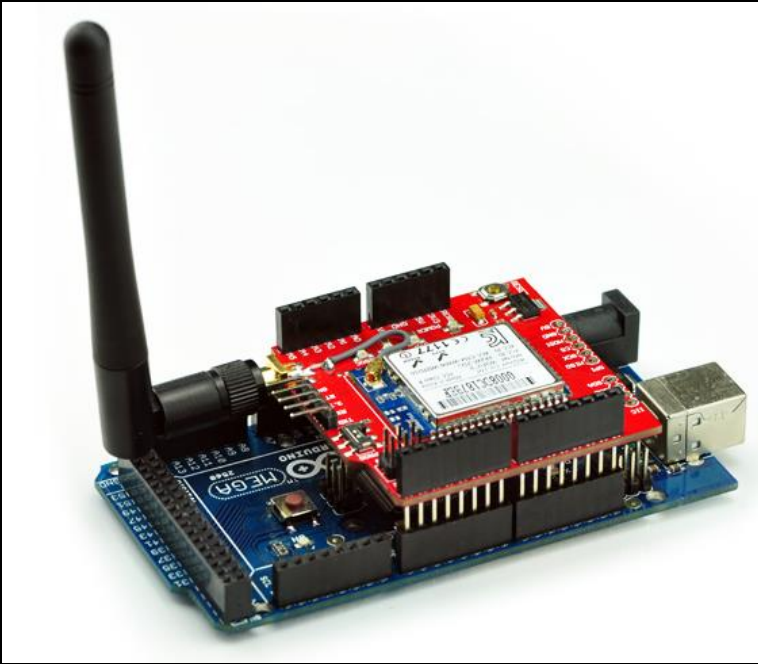


<https://www.youtube.com/watch?v=nxjLx5RdaSQ>

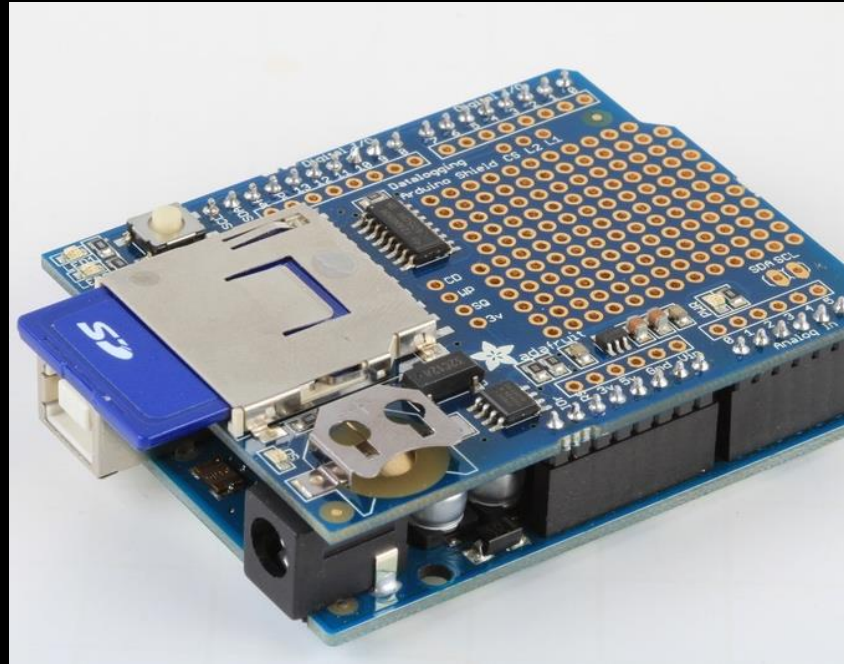
Arduino Models

- Different in terms of speed, size, resource, power consumption, ports, networking, etc.
- Share codes unless using different PIN connections.

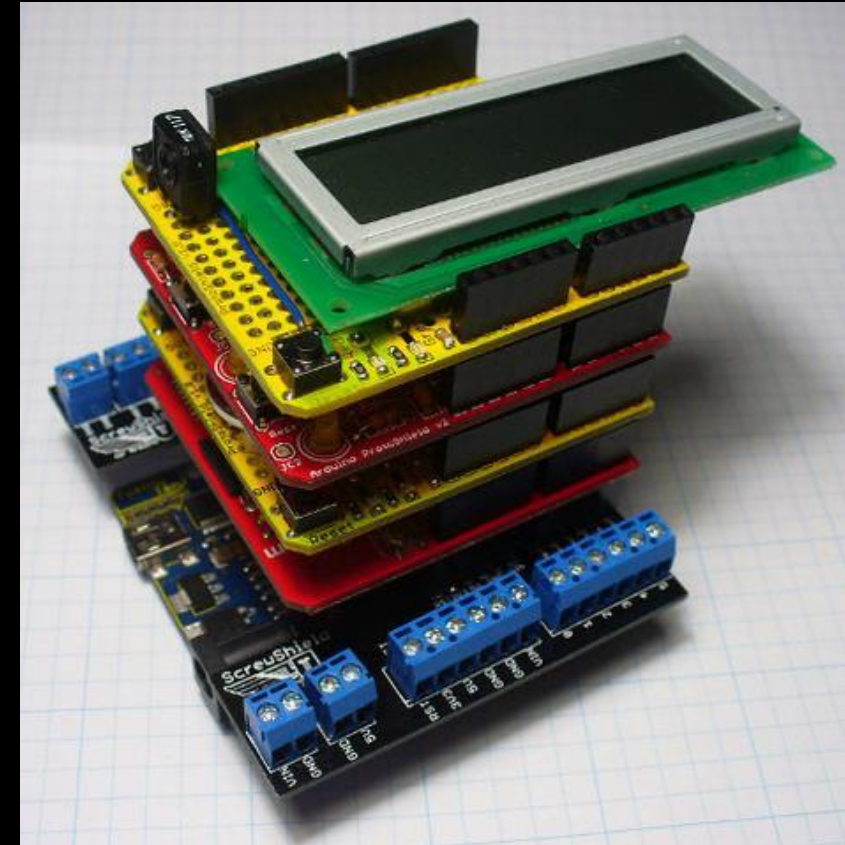
Arduino Board	Family	MEMORY			Clock	UART	PWM	Digital	Analog	VCC	Vin Range	USB-Serial
		SRAM	FLASH	EEPROM								
Duemilanove (328)	ATmega328	2K	32k	1kB	16MHz	1	6	14	6	5V	7-12V	ATmega16U2
Uno	ATmega328	2k	32k	1kB	16MHz	1	6	14	6	5V	7-12V	ATmega16U2
Arduino Mega 2560	ATmega2560	8k	256k	1kB	16MHz	4	14	54	16	5V	7-18V	ATmega16U2
Arduino Mega ADK	ATmega2560	8k	256k	1kB	16MHz	4	14	50	16	5V	7-18V	ATmega16U2
Arduino Ethernet	ATmega328	2k	32k	1kB	16MHz	1	4	9	6	5V	6-18V	N/A
Arduino BT	ATmega328	2k	32k	1kB	16MHz	1	6	14	6	5.5V	1.2V-5.5V	Bluegiga WT1
Arduino Pro Mini 328 5V	ATmega328	2k	32k	1kB	16MHz	1	6	14	6	5V	5-12V	N/A
Arduino Nano 3.0	ATmega328	2k	32k	1kB	16MHz	1	6	14	8	5V	7-12V	FTDI FT232RL
Arduino Mini	ATmega328	2k	32k	1kB	16MHz	1	6	14	8	5V	7V-9V	N/A
Arduino Pro 3.3V	ATmega328P	2k	32k	1kB	8MHz	1	6	14	6	3.3V	3.35-12V	N/A
Arduino Pro 5V	ATmega328P	2k	32k	1kB	16MHz	1	6	14	6	5V	5-12V	N/A
Arduino Fio	ATmega328P	2k	32k	1kB	8MHz	1	6	14	8	3.3V	3.35-12V	N/A
LilyPad Simple Board	ATmega168P	1k	16k	512B	8MHz	1	5	9	4	2.7-5.5V		N/A
LilyPad 328 Main Board	ATmega328P	2k	32k	1kB	8MHz	1	6	14	6	2.7-5.5V		N/A



WiFi Shield (802.11 b/g/n)



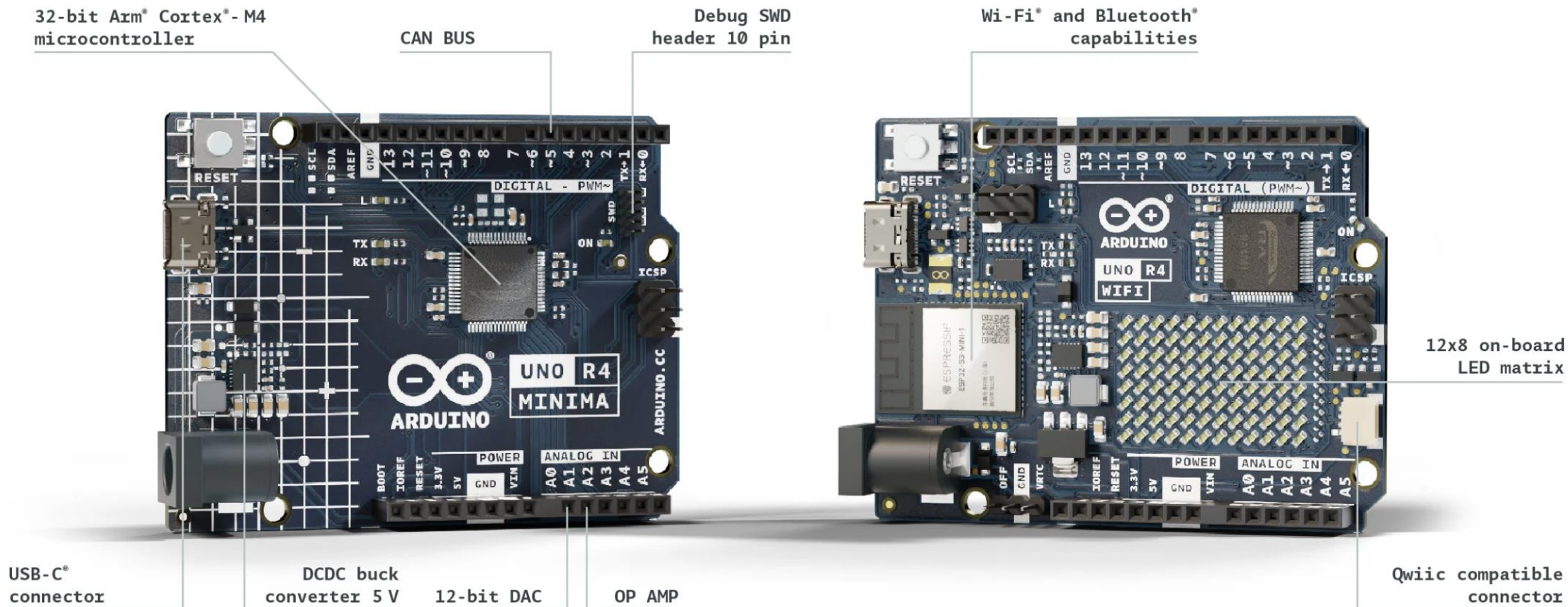
Adafruit Data Logger Shield



Stacking Arduino Shields

Arduino's Shields

- Plug-in boards on top of Arduino to provide functionalities such as Wi-Fi/BLE/XBee networking, GPS, SD card readers, etc.



UNO R4 Minima

Focused on simplicity, it offers essential functions that make it ideal for prototyping and basic learning.

UNO R4 WiFi

Perfect for IoT and wireless projects with its ESP32-S3 for WiFi/BLE and a 12x8 onboard LED matrix for stunning visual displays.

Specification of Raspberry Pi 4 Model B

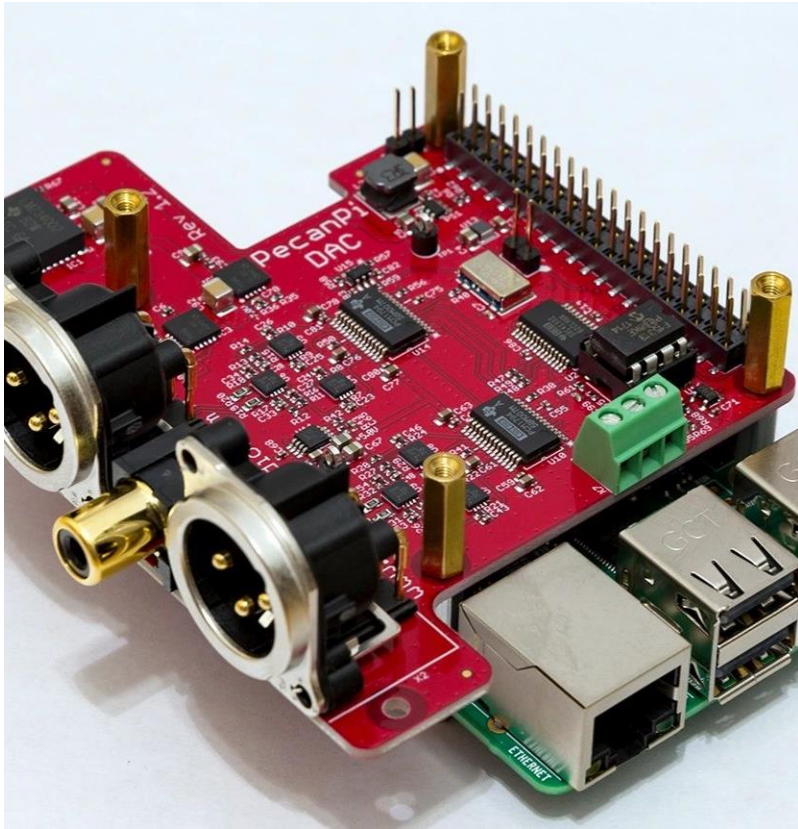
- Broadcom BCM2711, Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.5GHz
- 1GB, 2GB or 4GB LPDDR4-2400 SDRAM (depending on model)
- 2.4 GHz and 5.0 GHz IEEE 802.11ac wireless, Bluetooth 5.0, BLE
- Gigabit Ethernet
- 2 USB 3.0 ports; 2 USB 2.0 ports.
- 2 × micro-HDMI ports (up to 4kp60 supported)
- 2-lane MIPI DSI display port
- 2-lane MIPI CSI camera port
- 4-pole stereo audio and composite video port
- H.265 (4kp60 decode), H264 (1080p60 decode, 1080p30 encode)
- OpenGL ES 3.0 graphics
- 5V DC

Raspberry Pi 4

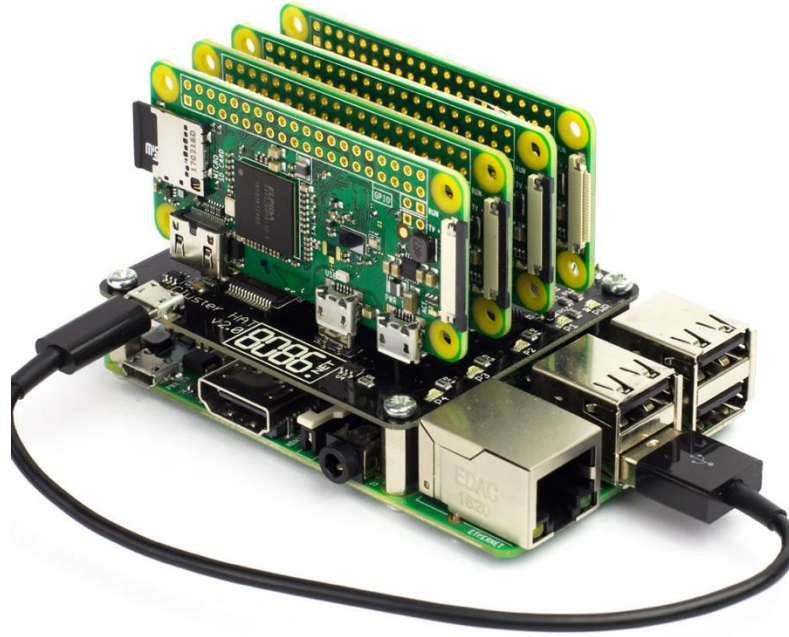
Your tiny, dual-display, desktop computer

...and robot brains, smart home hub, media centre,
networked AI core, factory controller, and much more

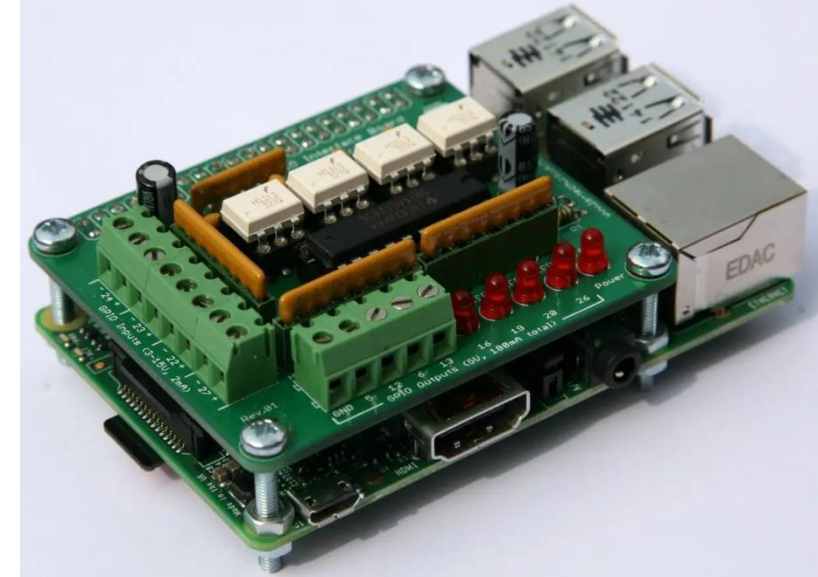




PecanPI DAC



Cluster HAT with 4 RSP-Zeros



GPIO Interface Board

Raspberry Pi's HATs

Same as Arduino, Raspberry pi' HATs (Hardware Attached on Top) are add-ons boards for Raspberry Pi family

BCM2712- a quad-core Cortex-A76 processor clocked at 2.4 GHz

FASTER CPU
& DEDICATED
GRAPHICS CHIP

FAN CONNECTOR

PI I/O CHIP

FASTER USB

HAT & GPIO
INTERFACE

GIGABIT
ETHERNET

PCIe 2.0 x1
INTERFACE

2 x 4 LANE
CAMERA/DISPLAY
TRANSCIEVERS

POWER BUTTON

2 x MICRO-HDMI® PORTS

USB-C POWER SUPPLY



Raspberry Pi **5**



NVIDIA Jetson Modules



Nano



TX1



TX2



Xavier



nvidia®



JETSON NANO SPECIFICATIONS



GPU	128 Core Maxwell 472 GFLOPs (FP16)
CPU	4 core ARM A57 @ 1.43 GHz
Memory	4 GB 64 bit LPDDR4 25.6 GB/s
Storage	16 GB eMMC
Video Encode	4K @ 30 4x 1080p @ 30 8x 720p @ 30 (H.264/H.265)
Video Decode	4K @ 60 2x 4K @ 30 8x 1080p @ 30 16x 720p @ 30 (H.264/H.265)
Camera	12 (3x4 or 4x2) MIPI CSI-2 DPHY 1.1 lanes (1.5 Gbps)
Display	HDMI 2.0 or DP1.2 eDP 1.4 DSI (1 x2) 2 simultaneous
UPHY	1 x1/2/4 PCIE 1 USB 3.0
SDIO/SPI/SysIOs/GPIOs/I2C	1x SDIO / 2x SPI / 5x SysIO / 13x GPIOs / 6x I2C

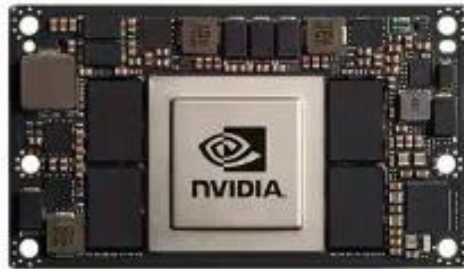
THE JETSON FAMILY

From AI at the Edge to Autonomous Machines



JETSON NANO

5–10W
0.5 TFLOPS (FP16)
45mm x 70mm
\$129 / \$99 (Devkit)



JETSON TX1 → JETSON TX2 4GB

7–15W
1–1.3 TFLOPS (FP16)
50mm x 87mm
\$299



JETSON TX2 8GB | Industrial

7–15W
1.3 TFLOPS (FP16)
50mm x 87mm
\$399–\$749



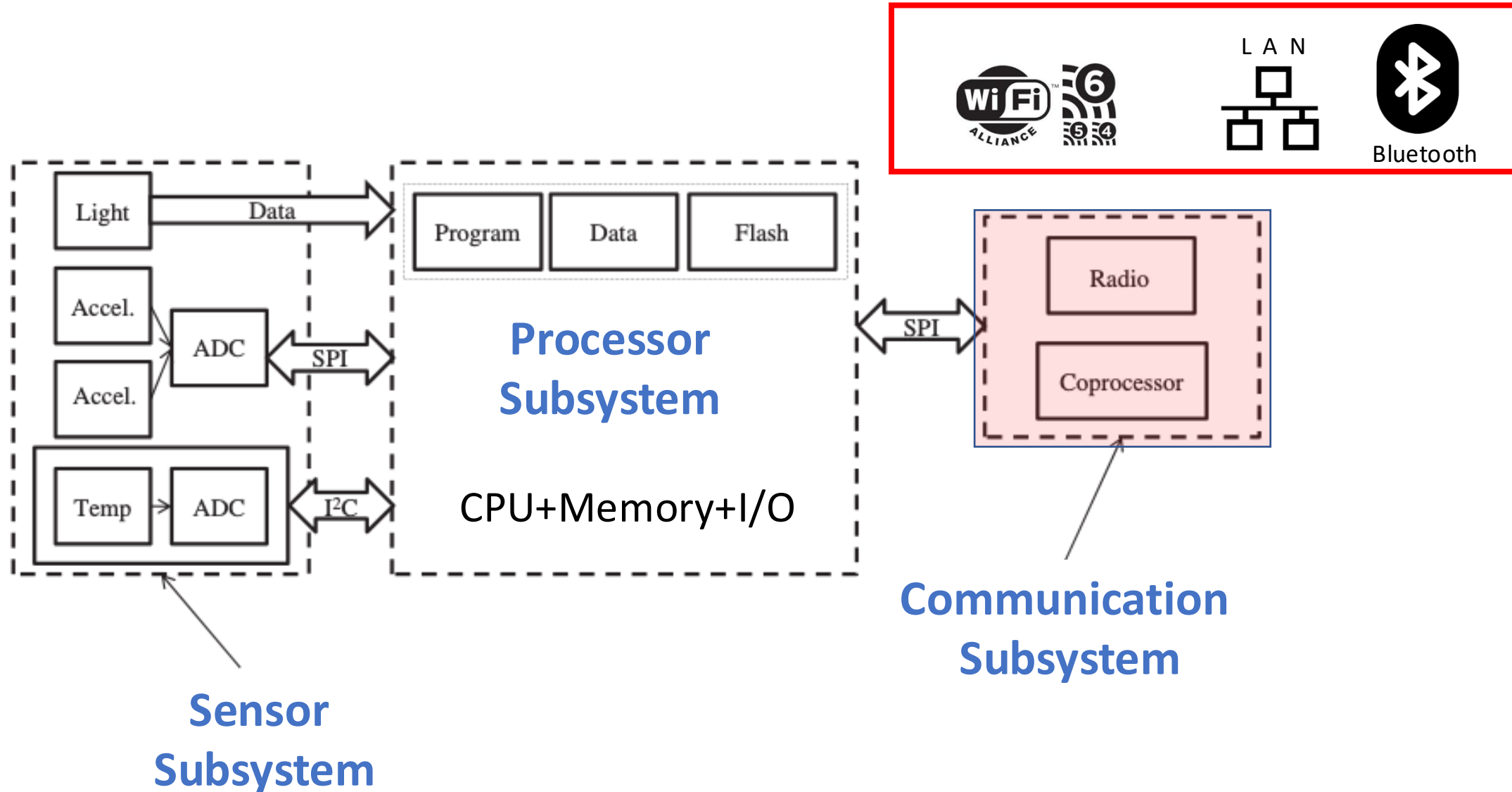
JETSON AGX XAVIER

10–30W
11 TFLOPS (FP16) | 32 TOPS (INT8)
100mm x 87mm
\$1099

AI at the Edge

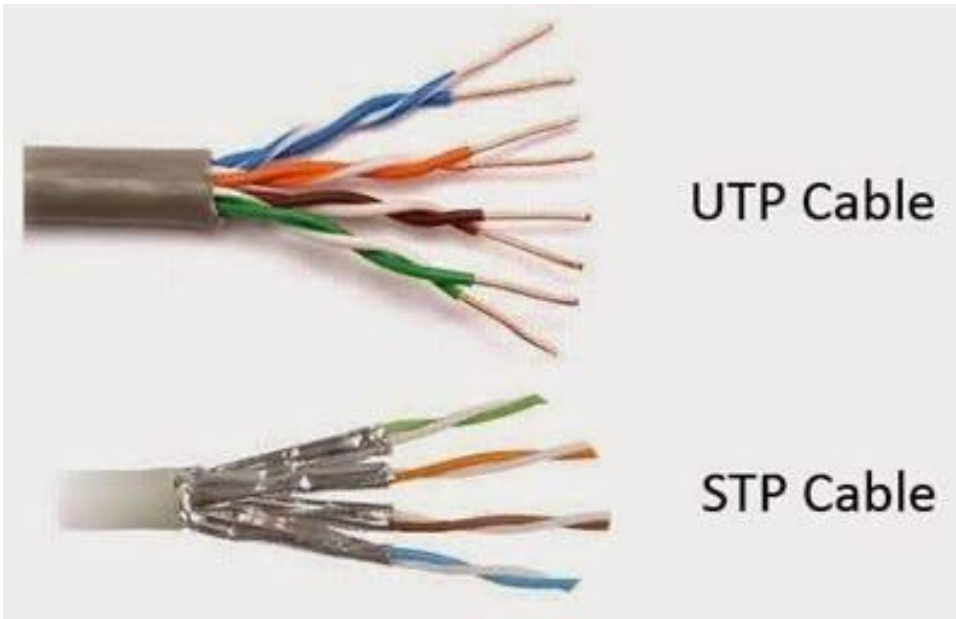
Fully Autonomous Machines

Basic IoT Sensor Node Architecture



Wired- Network

Ethernet Standards			
Standard	Speed	Segment Length	Cable
10Base5	10Mbps	500m / 164ft.	RG-8 or RG-11 coaxial
10Base2	10Mbps	185m / 606ft.	RG 58 A/U or RG 58 C/U coaxial
10Base-T	10Mbps	100m / 328ft.	Category 3 or better UTP
100Base-T	100Mbps	100m / 328ft.	Cat 5 UTP or STP
100Base-TX	100Mbps	100m / 328ft.	Cat 5 UTP or STP
100Base-FX	100Mbps	2 km	2-pair 850 nm multimode optic fibers
1000Base-T	1Gbps	100m / 328ft.	4-pair, CAT5 or CAT5e
1000Base-SX	1Gbps	550m (multimode)	2-pair fiber optic



cat 5 vs cat 6



Cat5e VS Cat6		
Product Name	Cat5e UTP Cable	Cat6 UTP Cable
Speed	10BASE-T, 100BASE-TX(Fast Ethernet), 1000BASE-T (Gigabit Ethernet)	10BASE-T, 100BASE-TX(Fast Ethernet), 1000BASE-T (Gigabit Ethernet), 10G BASE-T (10-Gigabit Ethernet)
Frequency	100 MHz	250 MHz
Performance	Good	Better

Easy way to tell the difference between a Cat5e and Cat6 Patch Cable



Cat6



Cat5e

Wired- Network

Name	Code	Standard	Data	Distance	Cable
Ethernet	10BASE-T	802.3i-1990	10 Mbits	100 m	Copper
Fast Ethernet	100BASE-T	802.3u-1995	100 Mbits	100 m	Copper
Fast Ethernet	100BASE-SX	802.3u-1995	100 Mbits	2000 m	Fibre
Giga Ethernet	1000BASE-T	802.3ab-1999	1000 Mbits	100 m	Copper
Giga Ethernet	1000BASE-LX	802.3z-1998	1000 Mbits	5 km	Fibre
10 Gigabit Ethernet	10GBASE-T	802.3an-2006	10 Gbits	100 m	Copper
10 Gigabit Ethernet	10GBASE-LR	802.3ae-2002	10 Gbits	10 km	Fibre
100 Gigabit Ethernet	100GBASE-LR4	802.3ba-2010	100 Gbits	10 km	Fibre
Terabit Ethernet	400GBASE-LR8	802.3bs-2017	400 Gbits	10 km	Fibre

Reference เพิ่มเติม

<https://www.fiber-optic-components.com/1000base-sx-sfp-multimode-vs-1000base-lx-sfp-single-mode.html>

11AX

Wireless- Network

THE PATH TO TRULY BRILLIANT WI-FI

IEEE802.xxx

WiFi-1

11Mbps



11b

1999

WiFi-2/3

54Mbps



11a/g

2003

WiFi-4

600Mbps



11n

2009

WiFi-5

6.93Gbps



11ac

2013

9.6 Gbps



11ax

2019



4x

**BETTER IN DENSE
ENVIRONMENTS**

Improve average
throughput per user by
at least four times in
dense or congested
environments



FASTER THROUGHPUT

Deliver up to 40 percent
higher peak data rates
for a single client device



**INCREASE NETWORK
EFFICIENCY**

By more than four times

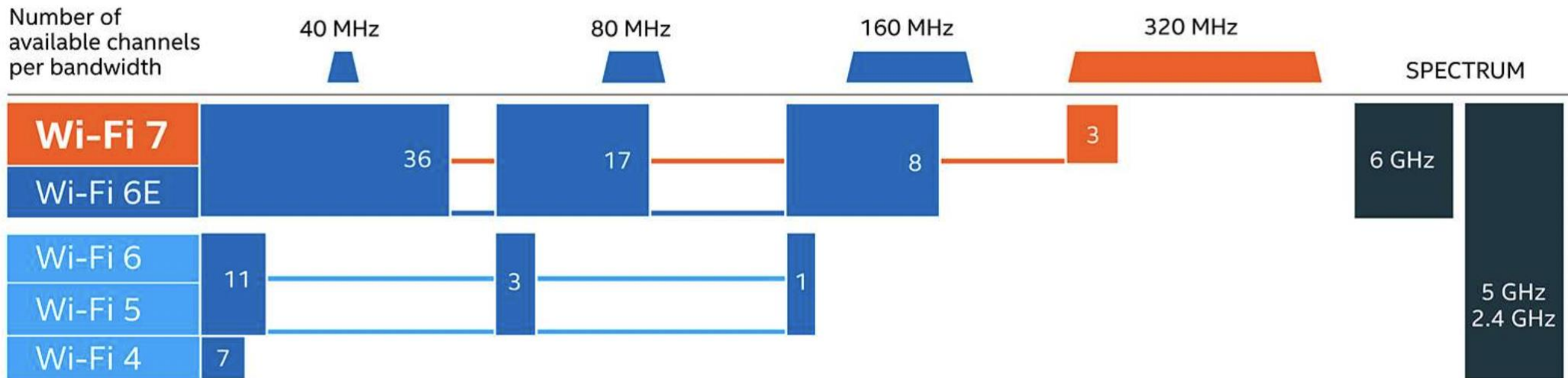


EXTEND BATTERY LIFE

Of client devices

Wi-Fi 7 – More lanes and a wider VIP highway

Wi-Fi 7 **doubles available bandwidth** compared to Wi-Fi 6E, with three super-wide 320 MHz channels on the dedicated 6 GHz band, in addition to all of the channels on the legacy 5 GHz and 2.4 GHz bands.



More densely packed cargo

When combined with the new 320 MHz channel bandwidths, 4K QAM delivers 2.4X faster speeds than Wi-Fi 6, with PC users experiencing maximum speeds over 5 Gbps**.



** >5 Gbps Wi-Fi 7 2x2 client speed** - is based on the current draft of the 802.11be specification which specifies the theoretical maximum data rate for a 2x2 device that supports 320 MHz channels, 4096 QAM, and Multi-Link Operation is 5.76 Gbps. Based on an industry-standard assumption of 90% efficiency for new Wi-Fi products operating in the exclusive 6 GHz band, the resulting estimated maximum over the air 2x2 client speed would be 5.19 Gbps.

The **Bluetooth Low Energy Protocol** (also known as BLE or Bluetooth Smart), was first introduced by Bluetooth SIG (Bluetooth Special Interest Group) along with the Bluetooth 4.0 version (2010).

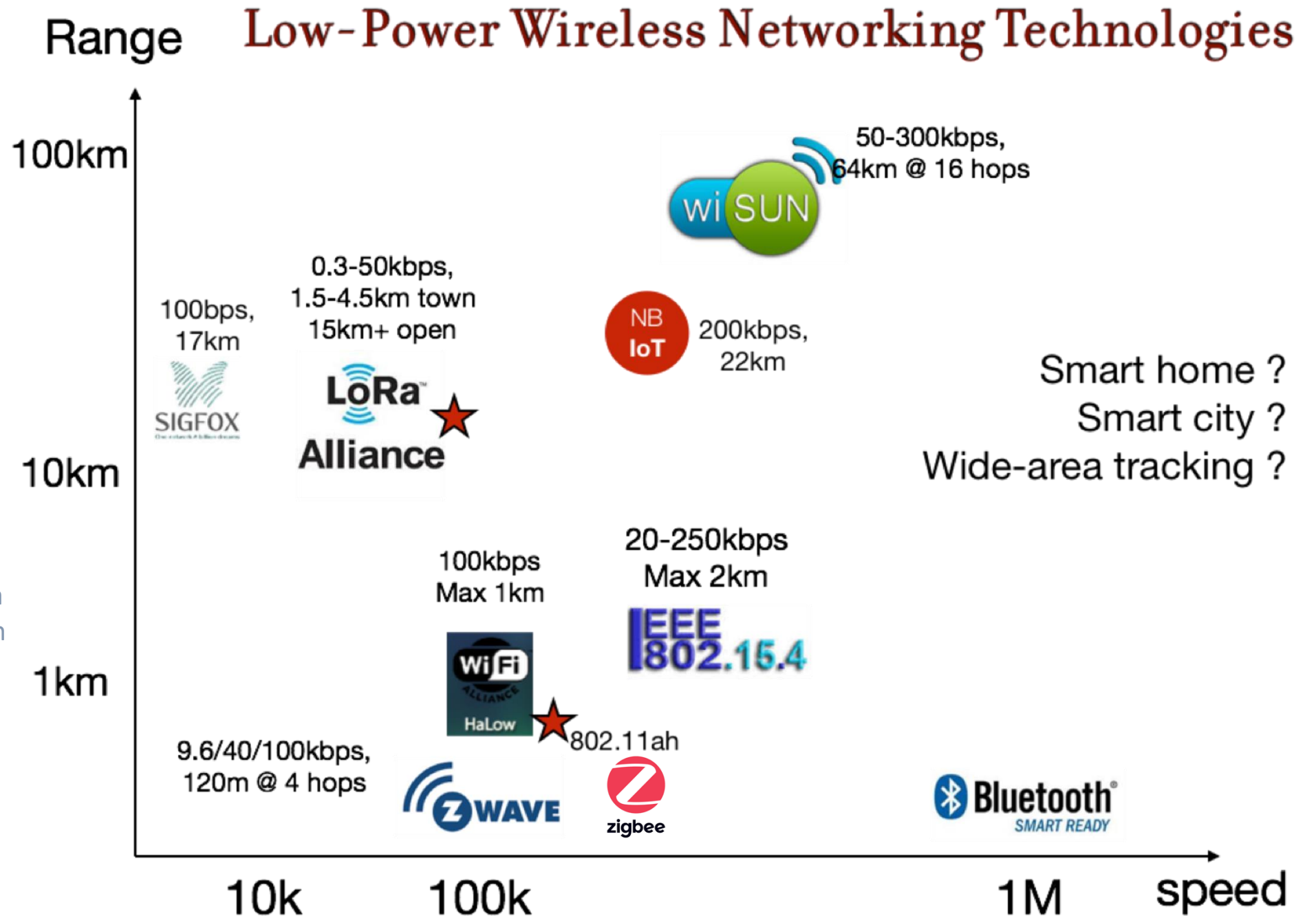
Year Introduced	Bluetooth Version	Feature
2004	2.0	Enhanced Data Rate
2007	2.1	Secure Simple Pairing
2009	3.0	High Speed with 802.11 Wi-Fi Radio
2010	4.0	Low-energy protocol
2013	4.1	Indirect IoT device connection
2014	4.2	IPv6 protocol for direct internet connection
2016	5.0	4x range, 2x speed, 8x message capacity + IoT
2019	5.1	Direction Finding
2020	5.2	EATT, faster, lower power, one-to-many connection
2021	5.3	Feature Enhancement, Security, Easy to connect

BLE

Other Wireless Technology

Challenging

- Addressing & Identification
- Low power communication
- Routing Protocol with low power require & efficient communication protocol
- High speed & loss less



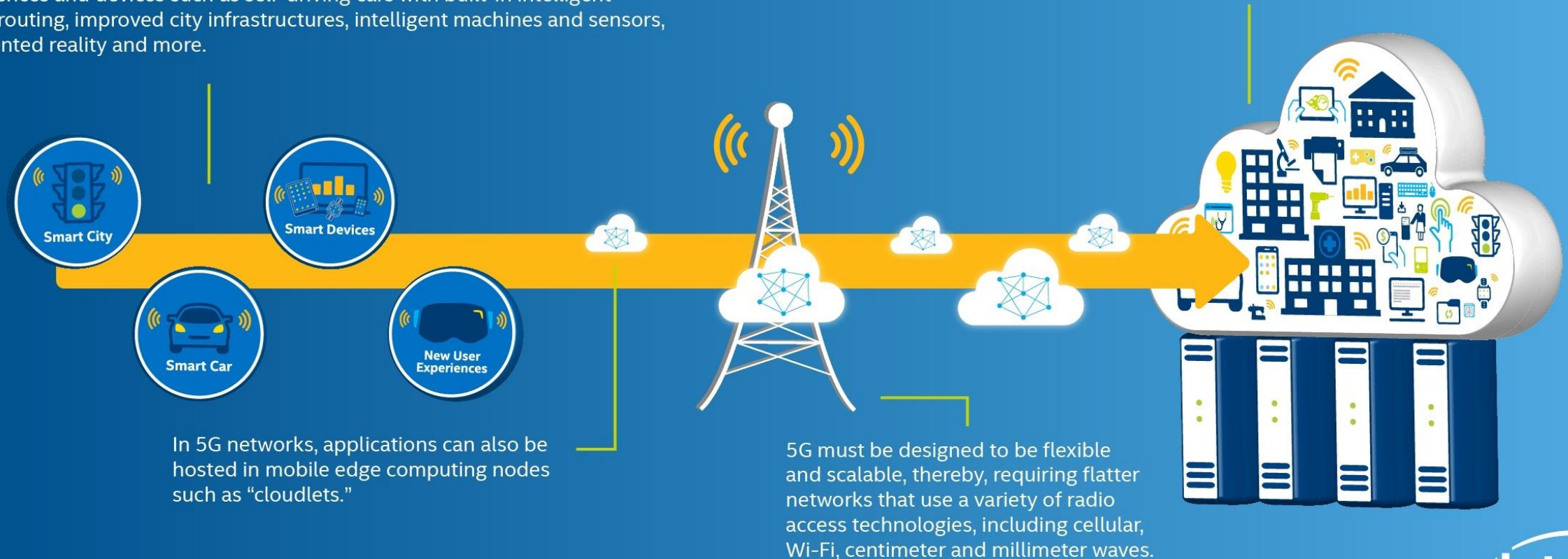
Smart home ?
Smart city ?
Wide-area tracking ?

5G: FROM DEVICE TO DATA CENTER

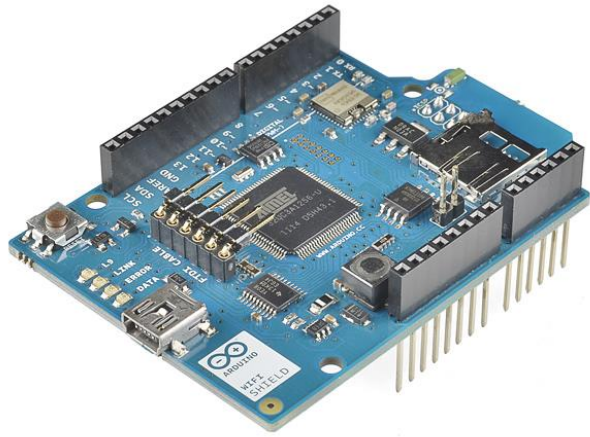
By 2020, 50 billion smart devices are expected to be in use.* 5G will help support the massive growth in the Internet of Things and enable devices to communicate with each other seamlessly through the convergence of mobile communications and computing. 5G networks will also diffuse intelligence across the entire network, from the device to the data center.

Using fast wireless connection to cloud computing and data services, and to other connected devices, 5G will enable a variety of new capabilities, user experiences and devices such as self-driving cars with built-in intelligent traffic routing, improved city infrastructures, intelligent machines and sensors, augmented reality and more.

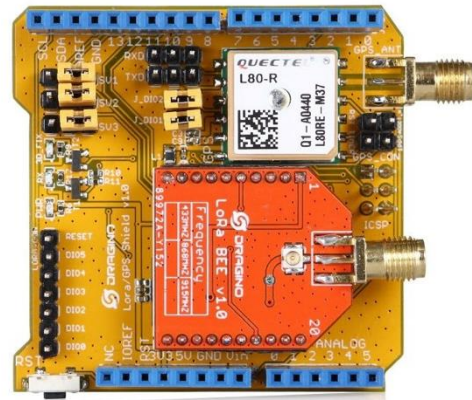
5G's combination of high-speed wireless communications and efficient cloud computing means that even the tiniest devices can access virtually unlimited computing power.



Communication Module



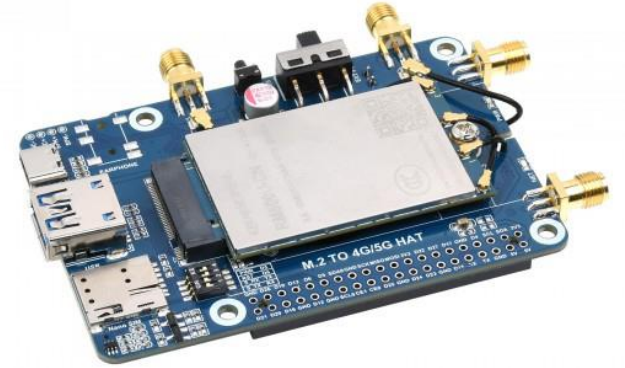
Wifi-shield for Arduino



LoRA-shield for Arduino

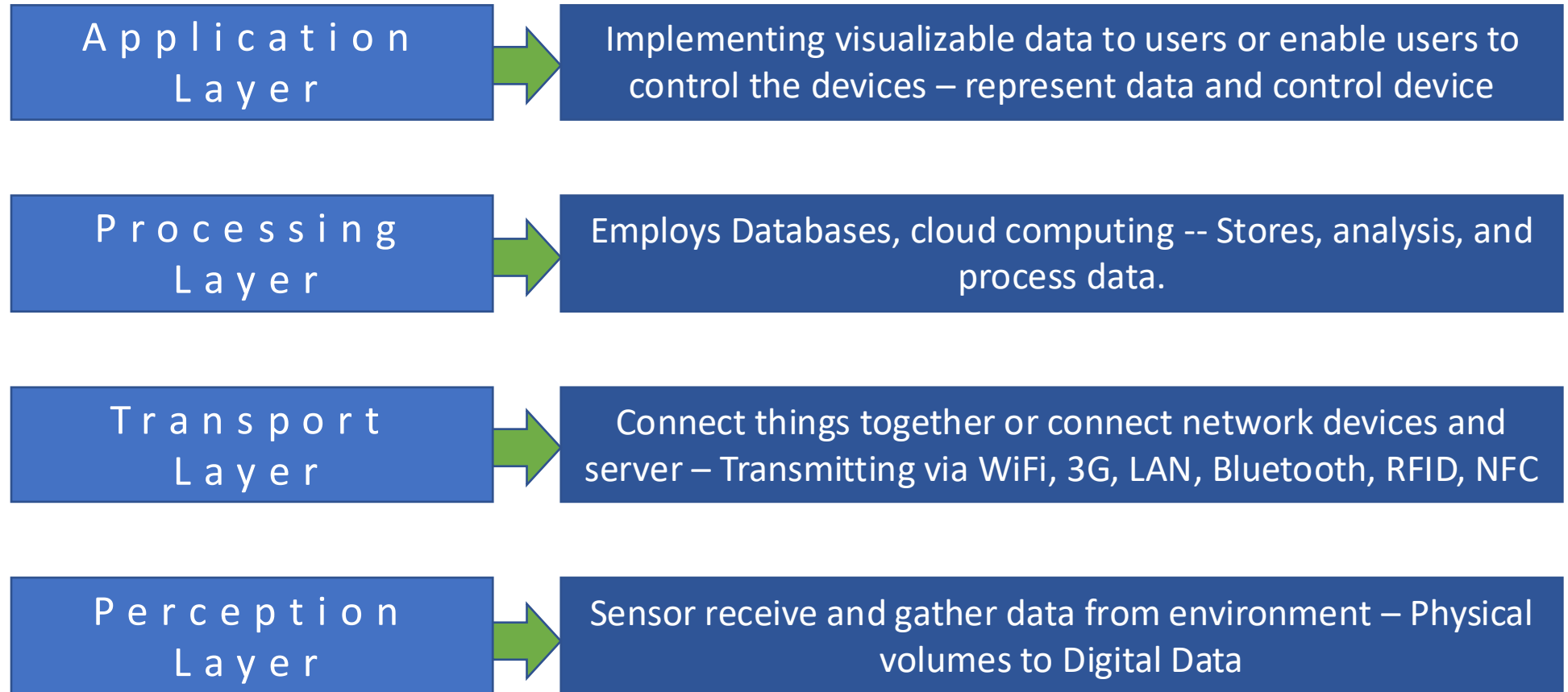


NB-IoT SIM7000E
HAT for raspberry Pi

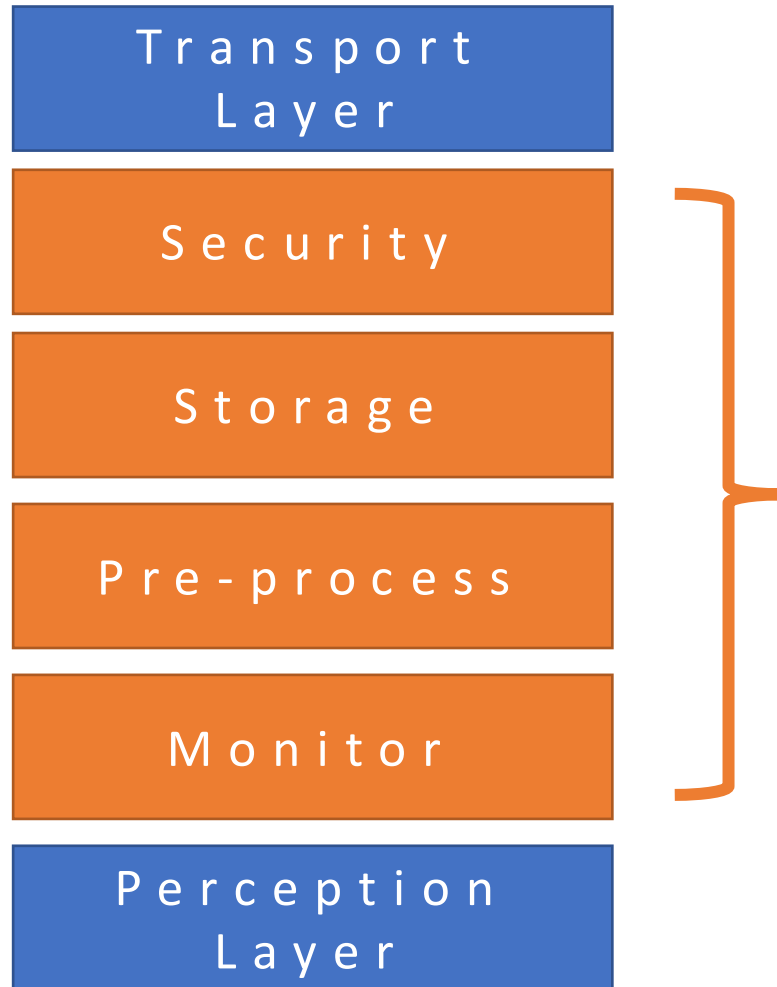


RM500x / RM502x 5G HAT for
Raspberry Pi, quad antennas
LTE-A, multi band, 5G/4G/3G

IoT Architecture



Fog computing Architecture



Fog computing's Feature

- Low latency
- Distribute node
- Mobility
- Real-time response
- Interaction with cloud

EDGE computing

INDUSTRIAL IoT DATA PROCESSING LAYER STACK

CLOUD LAYER

Big Data Processing
Business Logic
Data Warehousing

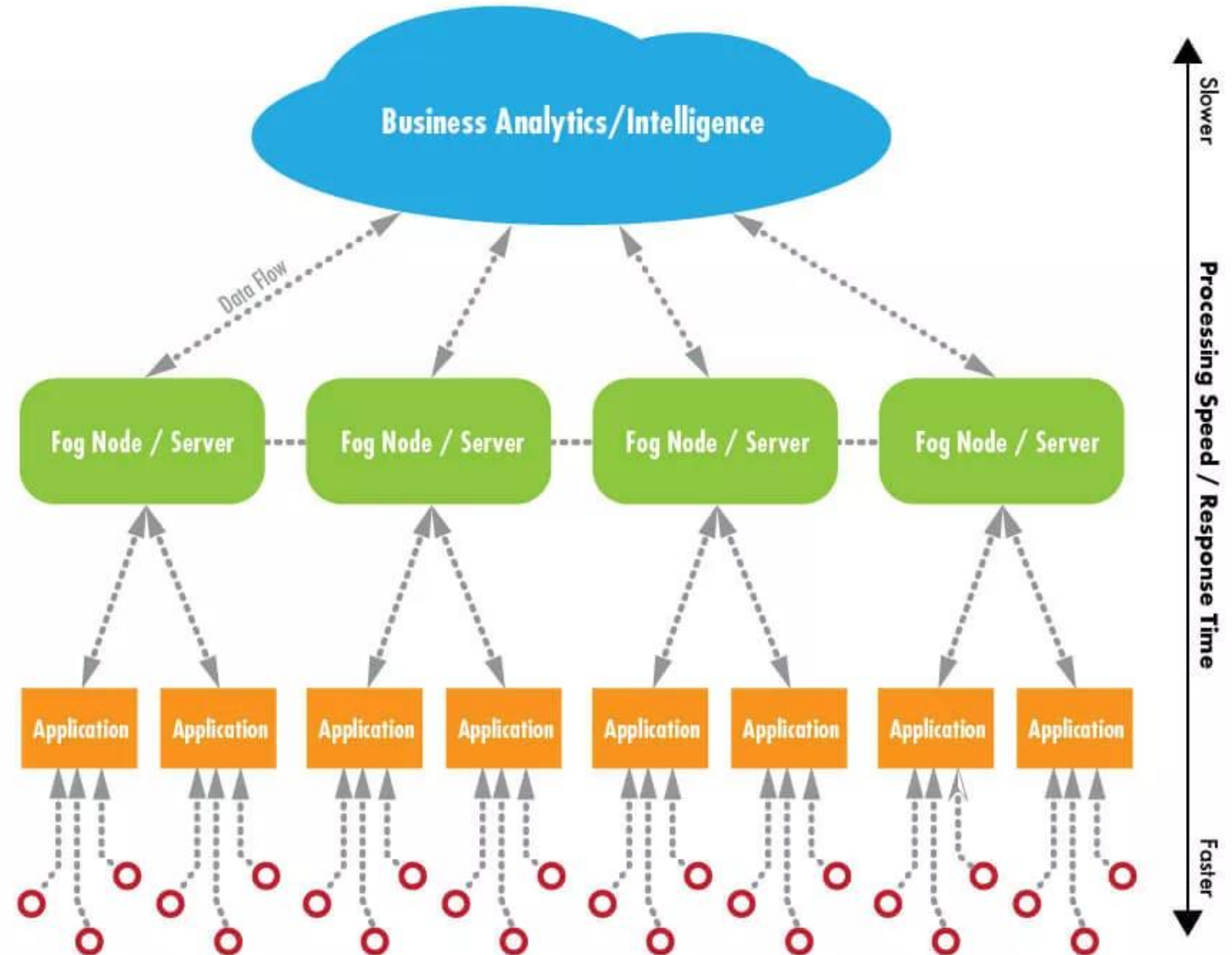
FOG LAYER

Local Network
Data Analysis & Reduction
Control Response
Virtualization/Standardization

EDGE LAYER

Large Volume Real-time Data Processing
At Source/On Premises Data Visualization
Industrial PCs
Embedded Systems
Gateways
Micro Data Storage

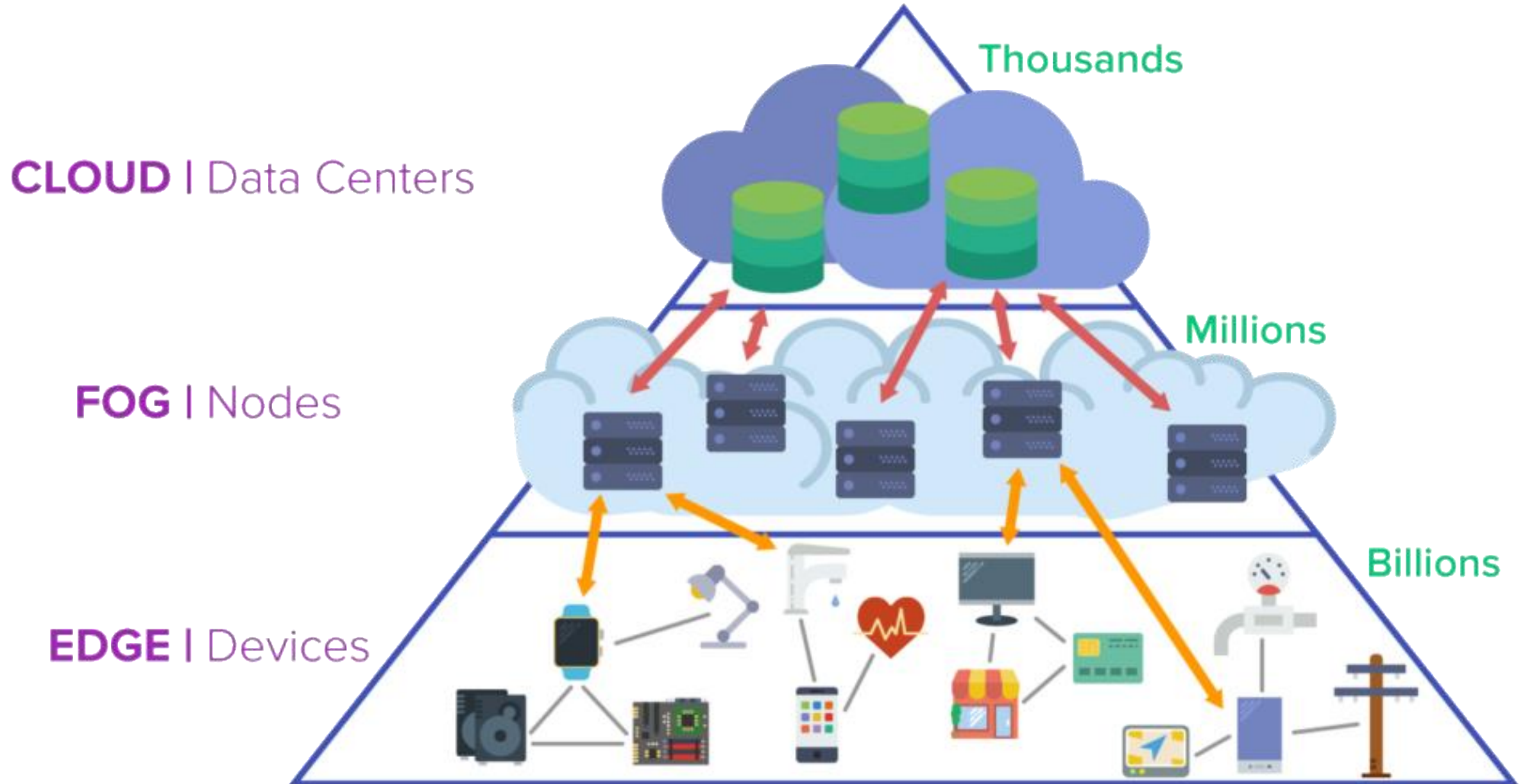
Sensors & Controllers (data origination)



EDGE AI

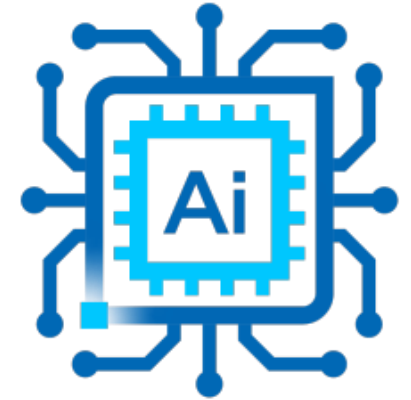
- Latency
- Real-time analytics
- Scalability
- Information security and privacy
- Automated decision-making
- Reduced costs

Cloud/Fog / Edge Computing in IoT



Edge AI

or “Artificial Intelligence on the Edge” the new buzzword in 2019/2020 that successor from IoTs in 2019/2020.



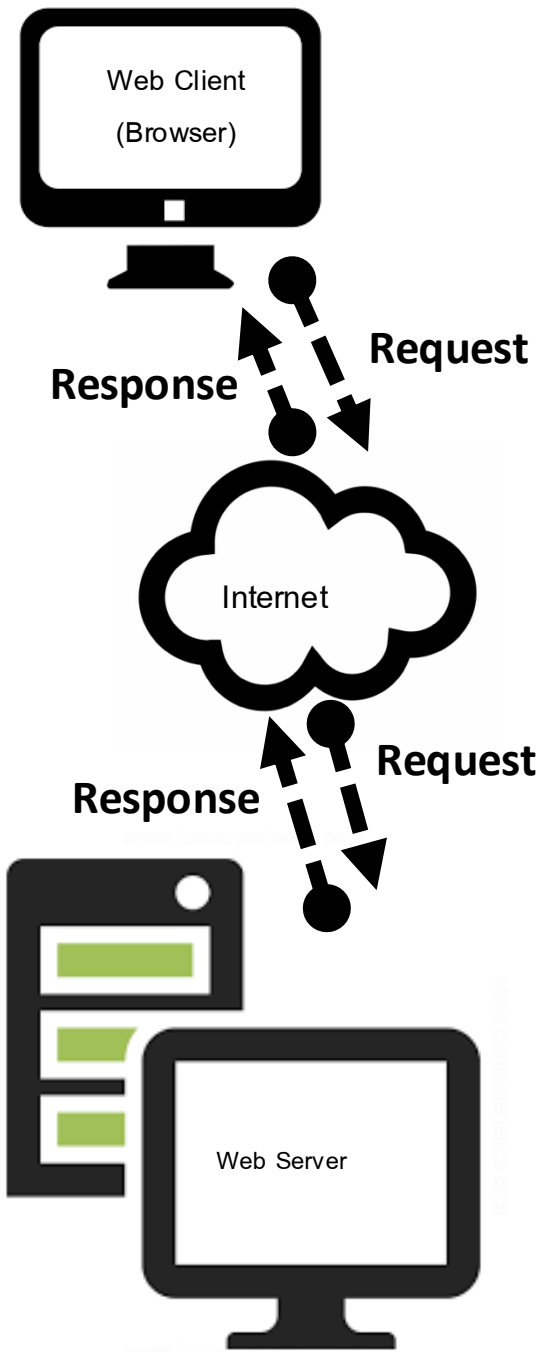
intel® Edge AI

Advantages of Edge AI – Applying Machine Learning on Edge

- **Reduced latency:** No data transferring back & forth from the cloud takes time.
- **Real-time analytics with higher speeds:** Processing Data locally significantly improves speed compared to cloud computing
- **Reduced bandwidth requirement and cost:** Reduce the cost of internet bandwidth and cloud storage.
- **Improved data security:** This greatly reduces the amount of data that is sent to the cloud and other external locations e.g.; iPhone's face detection
- **Scalability:** Edge AI typically processes large amounts of data. If you have to process video image data from many different sources simultaneously, transferring the data to a cloud service is not required.
- **Improved reliability:** Higher levels of security combined with greater speed produce greater the reliability of Edge AI System.
- **Reduced power:** Edge AI processes data at the device level so it saves energy costs

How do IoTs send data collection to internet

- IoT Nodes as Plain HTTP Server
- IoT Nodes as HTTP Client
- IoT Nodes with RESTful Web service
- IoT nodes send data to cloud service via HTTP
- IoT nodes with IoT cloud platform using MQTT



Hypertext Transfer Protocol (HTTP)

The **Hypertext Transfer Protocol (HTTP)** is an application layer protocol that is used to transmit virtually all files and other data on the World Wide Web, whether they're HTML files, image files, query results, or anything else.



A browser is an HTTP client because it sends requests to an HTTP server (Web server), which then sends responses back to the client. The standard (and default) port for HTTP servers to listen on is 80, though they can use any port.



HTTP is based on the TCP/IP protocols and is used commonly on the Internet for transmitting web-pages from servers to browsers.

HTTP Message Format

There are 2 types of messages that HTTP uses:

1. HTTP Request message
2. HTTP Response message

1. HTTP Request Message:

The request line has three parts, separated by spaces:

method name

path of the requested resource

HTTP version

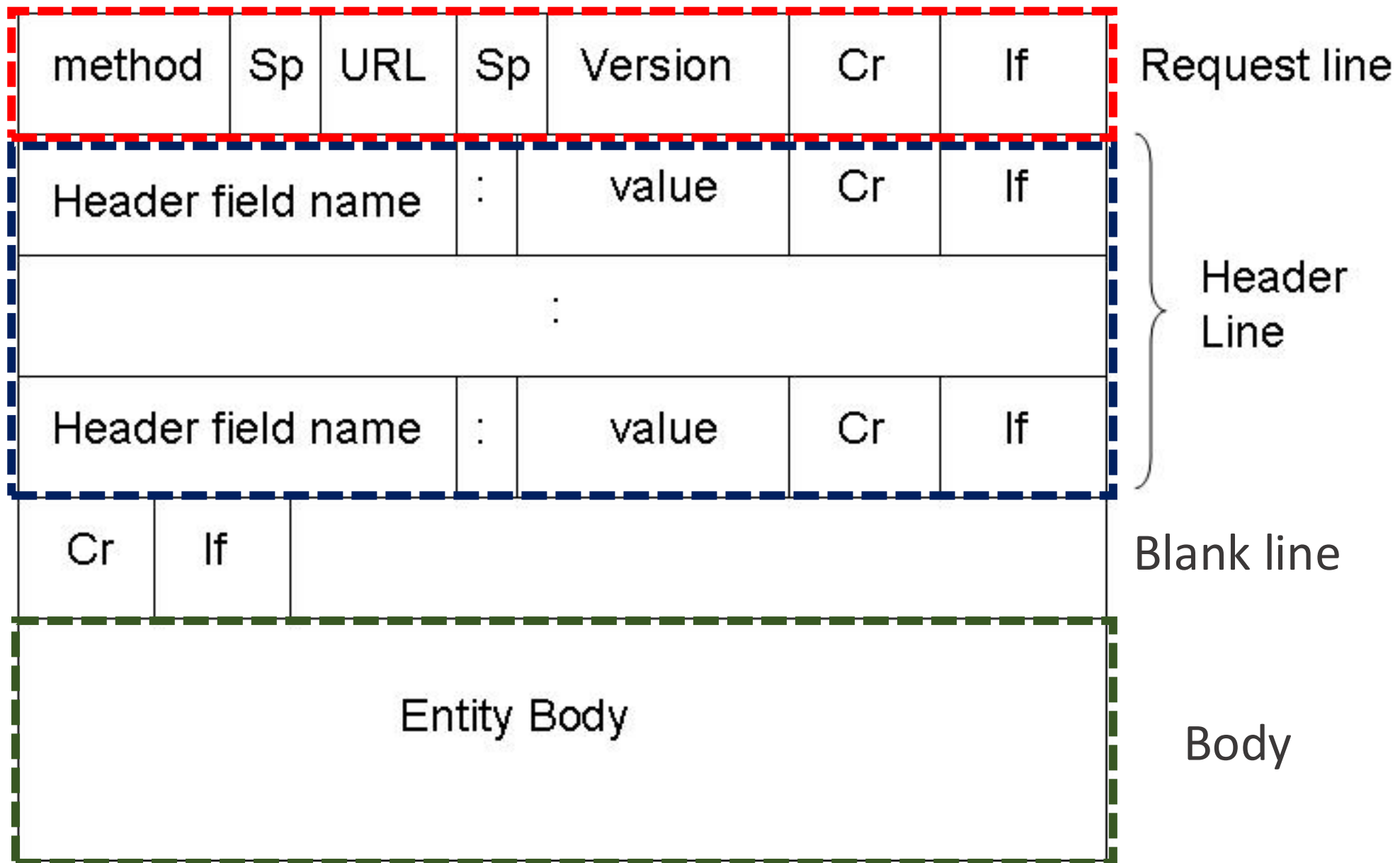
e.g.:

GET

https://www.tutorialspoint.com/http/http_requests.htm

HTTP/1.1

The GET method is the most commonly used. It means that “give me this resource”.



HTTP Request Message format

HTTP POST

Generally – not necessarily – `POST` APIs are used to create a new resource on server. So when you invoke the same `POST` request N times, you will have N new resources on server. So, **POST is not idempotent.**

HTTP GET, HEAD, OPTIONS and TRACE

`GET` , `HEAD` , `OPTIONS` and `TRACE` methods NEVER change the resource state on server. They are purely for retrieving the resource representation or meta data at that point of time. So invoking multiple requests will not have any write operation on server, so **GET, HEAD, OPTIONS and TRACE are idempotent.**

HTTP PUT

Generally – not necessarily – `PUT` APIs are used to update the resource state. If you invoke a `PUT` API N times, very first request will update the resource; then rest N-1 requests will just overwrite the same resource state again and again – effectively not changing anything. Hence, **PUT is idempotent.**

HTTP DELETE

When you invoke N similar `DELETE` requests, first request will delete the resource and response will be `200 (OK)` or `204 (No Content)`. Other N-1 requests will return `404 (Not Found)`. Clearly, the response is different from first request, but there is no change of state for any resource on server side because original resource is already deleted. So, **DELETE is idempotent**.

Please keep in mind if some systems may have DELETE APIs like this:

```
DELETE /item/last
```

In above case, calling operation N times will delete N resources – hence `DELETE` is not idempotent in this case. In this case, a good suggestion might be to change above API to POST – because POST is not idempotent.

```
POST /item/last
```

Now, this is more closer to HTTP spec – hence more REST compliant.

2. HTTP Response Message:

The HTTP message response line also has three parts separated by spaces:

HTTP version	response status code	English phrase of the status code
--------------	----------------------	-----------------------------------

This first line is called as Status line.

HTTP response status codes:

200 OK --- The request succeeded, and the resulting resource

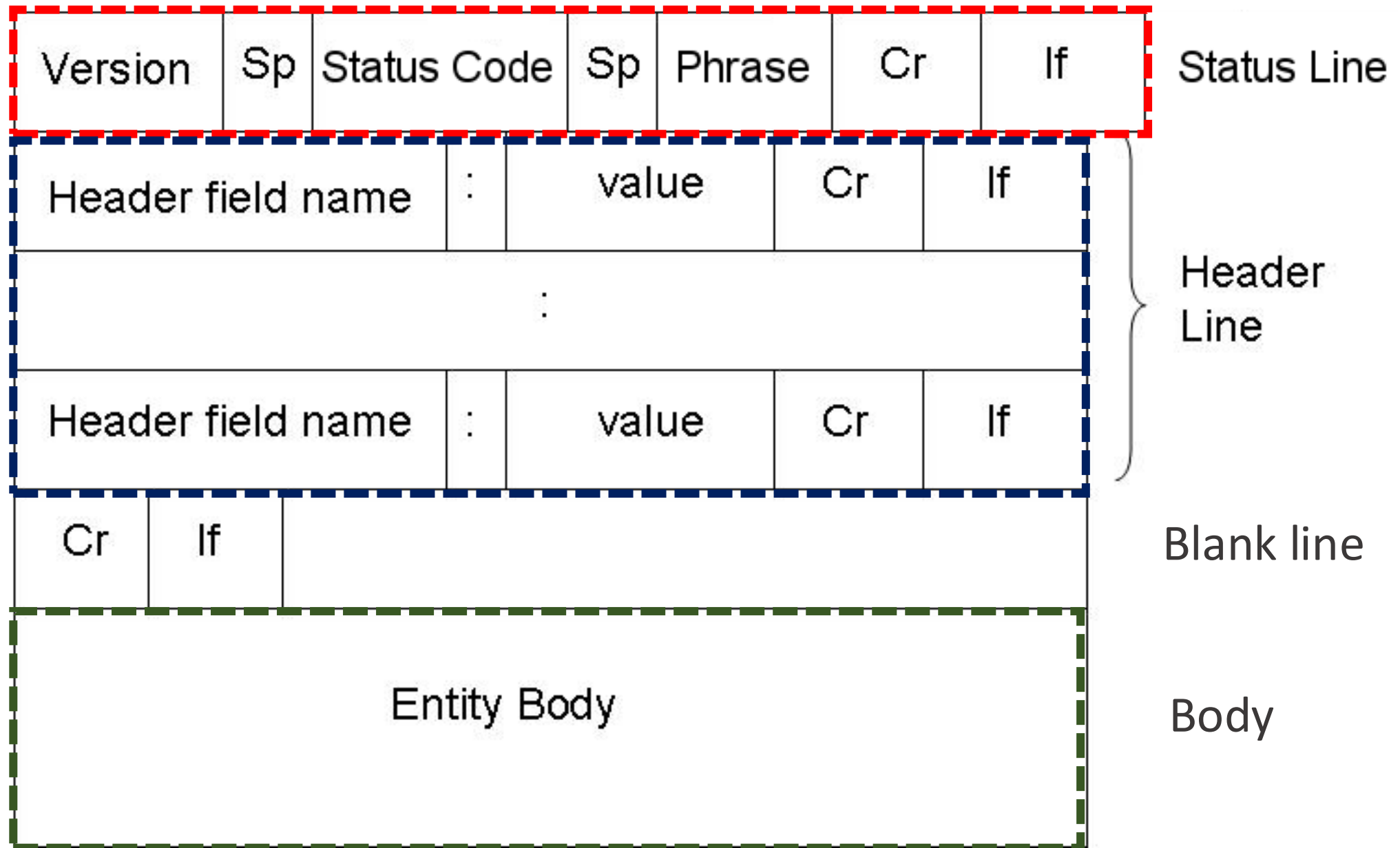
404 Not Found --- The requested resource doesn't exist.

301 Moved Permanently

302 Moved Temporarily

303 See Other (HTTP 1.1 only) ---- The resource has moved to another URL (given by the Location: response header) and should be automatically retrieved by the client. This is often used by a CGI script to redirect the browser to an existing file.

500 Server Error --- An unexpected server error. The most common cause is a server-side script that has bad syntax, fails, or otherwise can't run correctly.



The HTTP Response message format

Requests: HTTP for Humans™

Requests is an elegant and simple HTTP library for Python, built for human beings.



Requests allows you to send HTTP/1.1 requests extremely easily. There's no need to manually add query strings to your URLs, or to form-encode your POST data. Keep-alive and HTTP connection pooling are 100% automatic, thanks to [urllib3](https://urllib3.readthedocs.io/en/latest/).

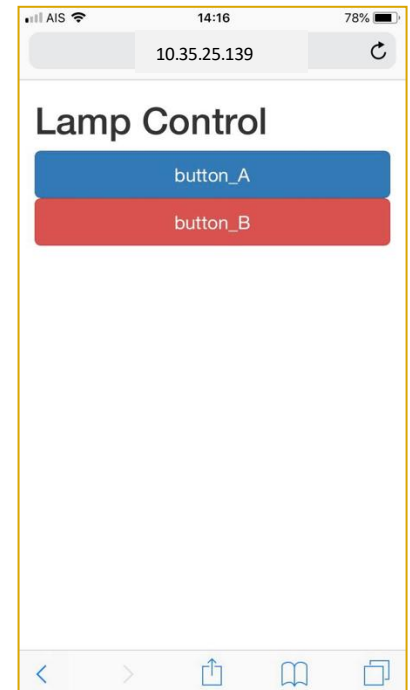
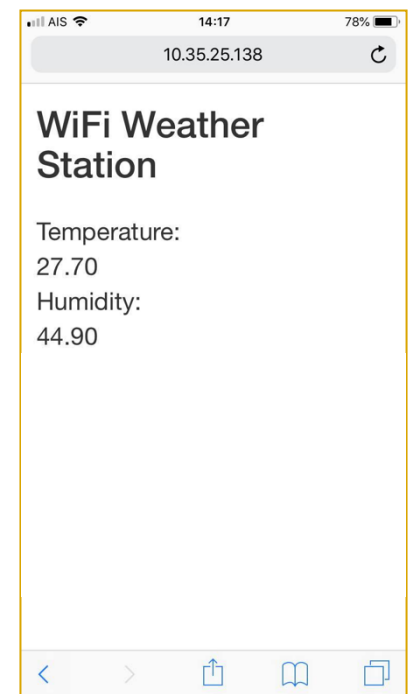
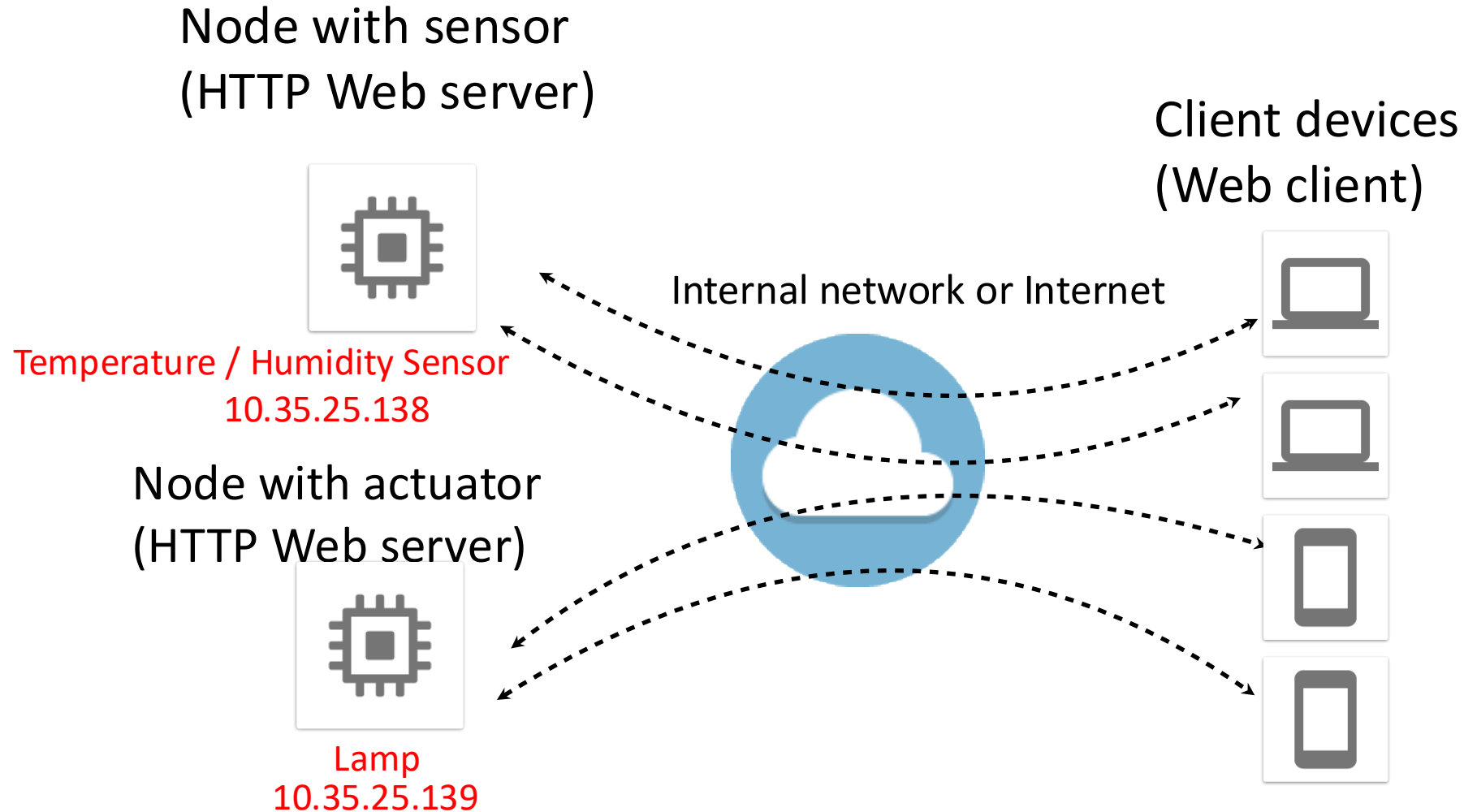
```
1) To install requests  
$ pip install requests
```

```
2) Test get method
```

```
>>>import requests  
>>>r = requests.get('https://api.github.com/')  
>>>print(r.text)
```

Method 1:

IoT Nodes as Plain HTTP Server

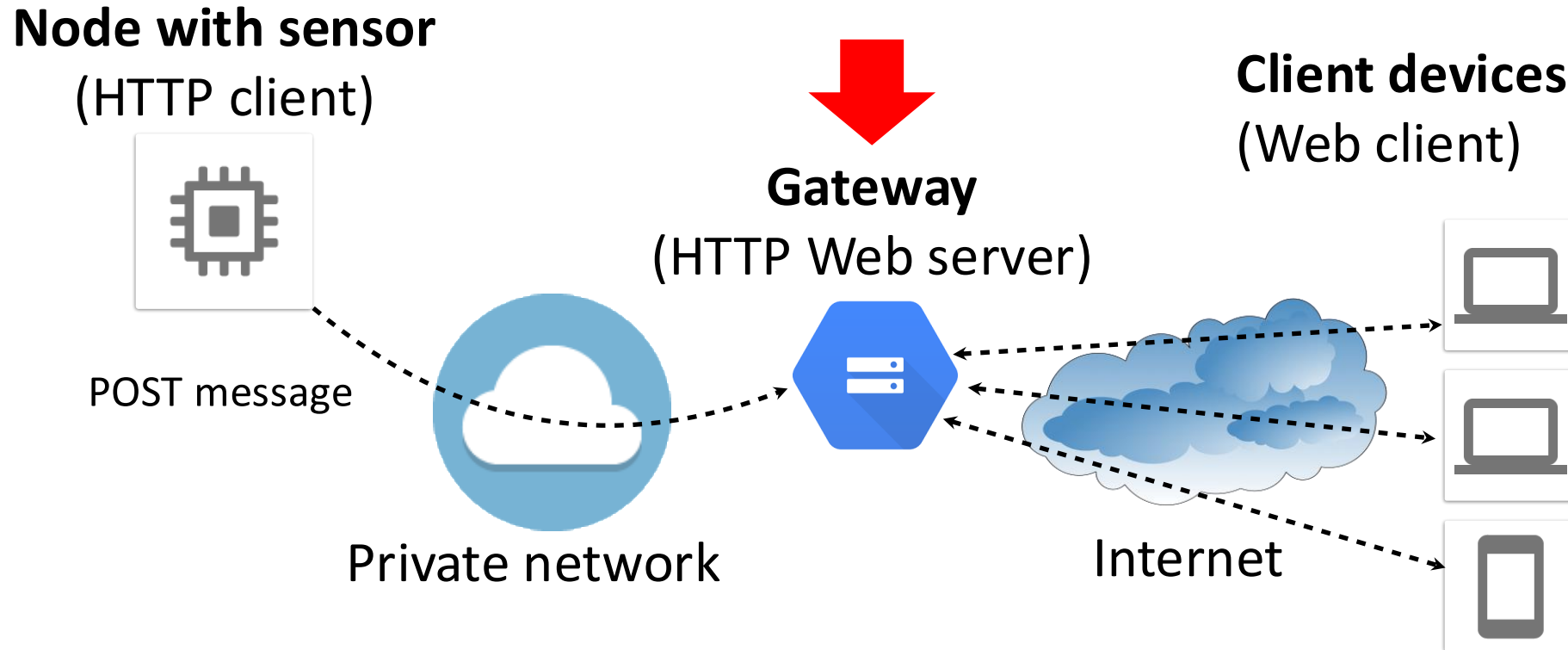


Limitation of Method1

- Simply work, if IoT node doesn't have frequent communication
- a lots of network traffic
 - The whole web page returned as HTTP response.
 - Respond same data many times.
 - High resource consumptions (CPU, bandwidth)
- Very limited GUI.

Method2:

IoT Node as HTTP Client



IoT node:

Sends only one copy of data

Not return the whole web page

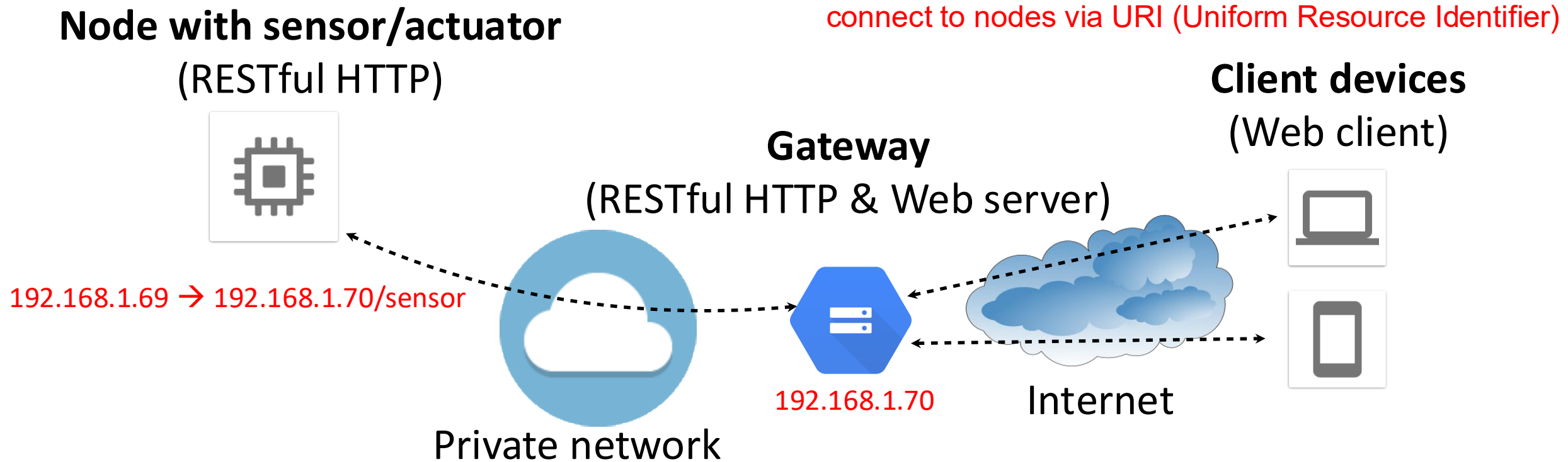
Limitation of Method2

- Node cannot act as an actuator role
 - Cannot receive any HTTP command
 - Not possible to control actuators attached to IoT node

Method3:

IoT Node with RESTful Web service

REST: REpresentational State Transfer [more info--> https://www.tutorialspoint.com/restful/index.htm](https://www.tutorialspoint.com/restful/index.htm)



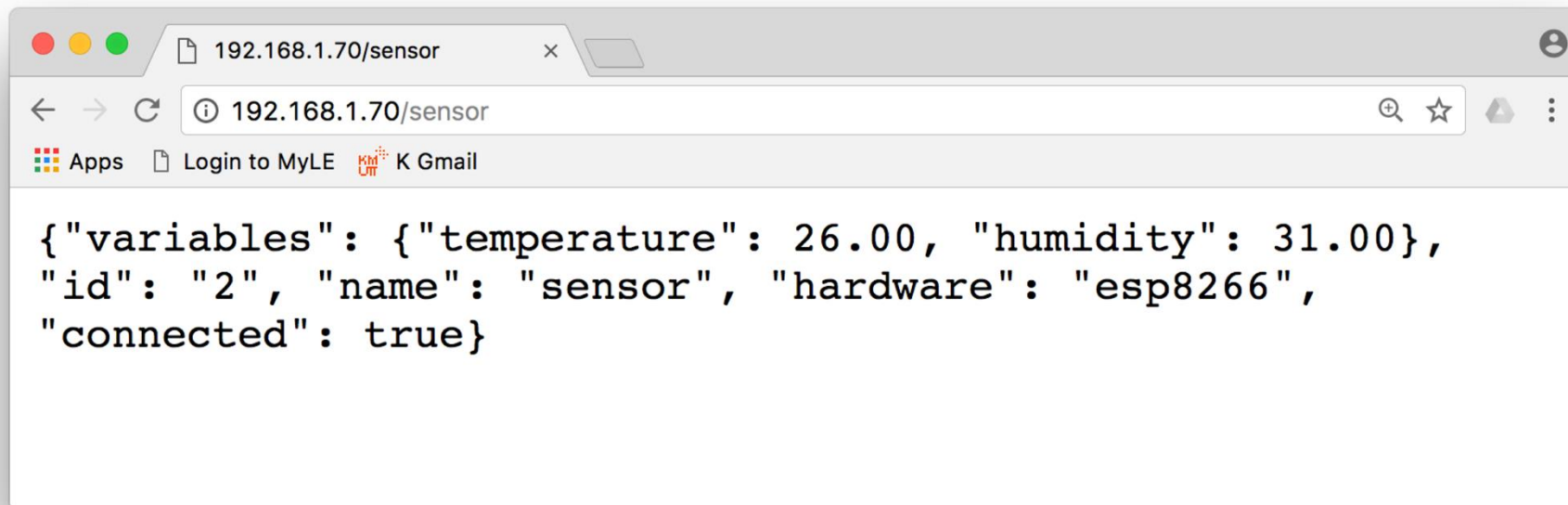
IoT node resources monitored/controlled via HTTP commands
IoT node doesn't return the whole web page

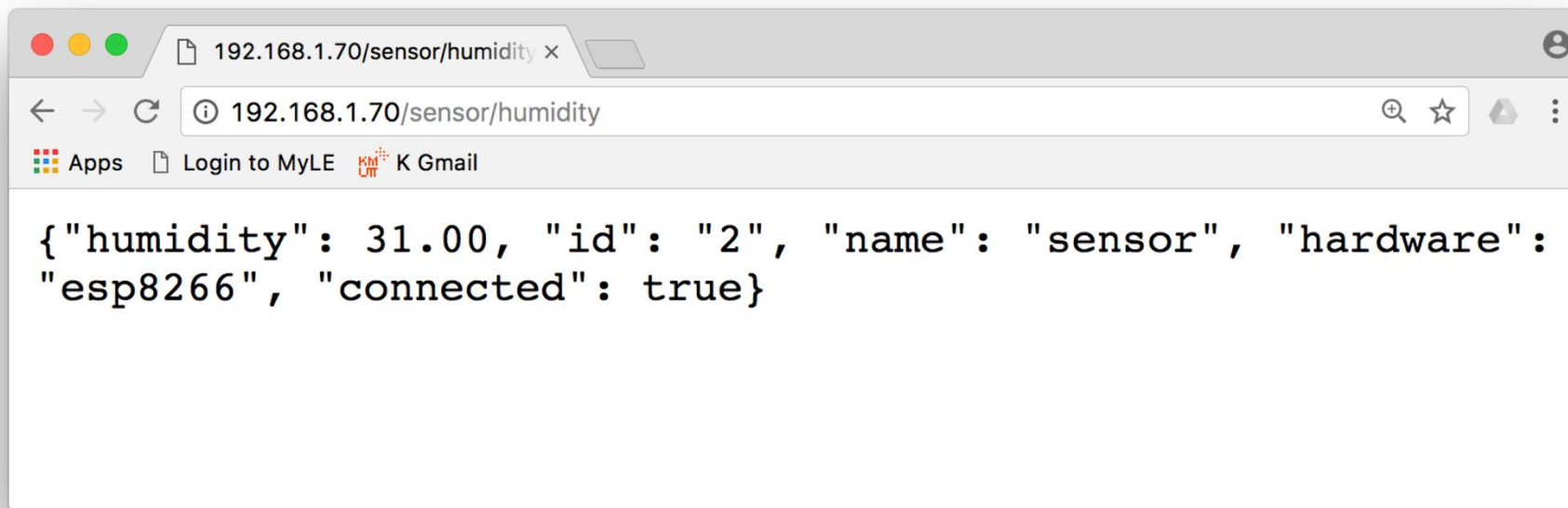
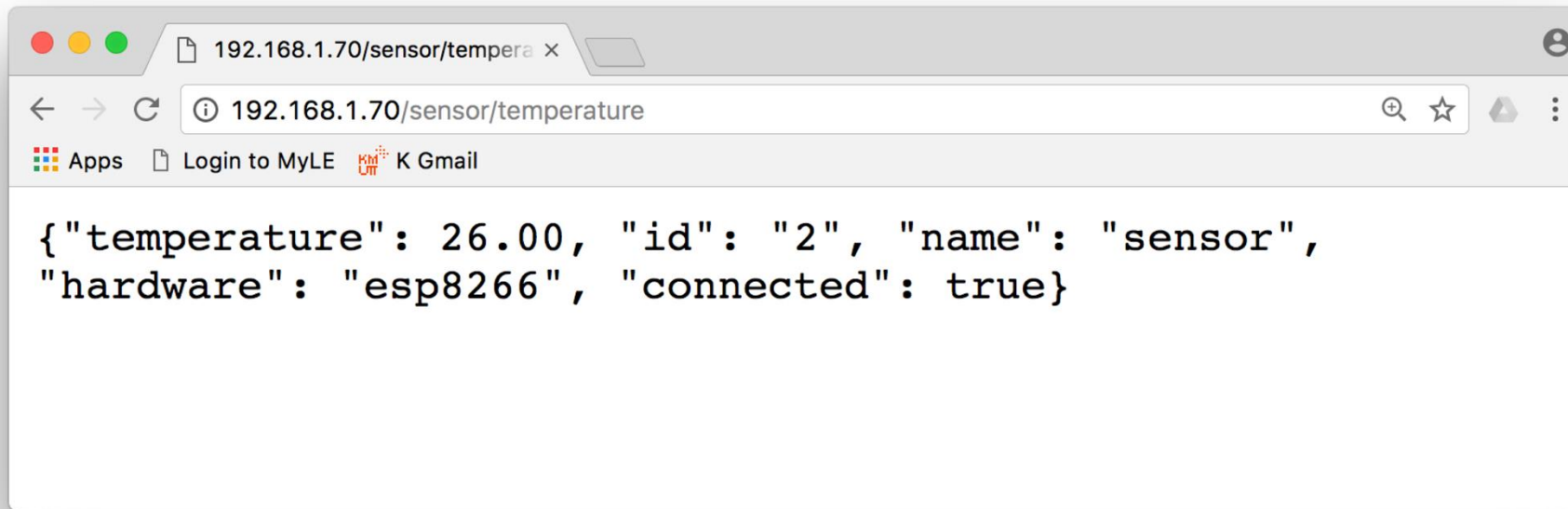
■ RESTful Web service

uses HTTP methods to implement the concept of REST architecture. A RESTful web service usually defines a URI (Uniform Resource Identifier) a service, provides resource representation such as JSON and set of HTTP Methods

Go to a web browser and type the Uniform Resource Identifiers (URIs)

```
http://<board_IP_address>/sensor  
or http://<board_IP_address>/sensor/temperature
```

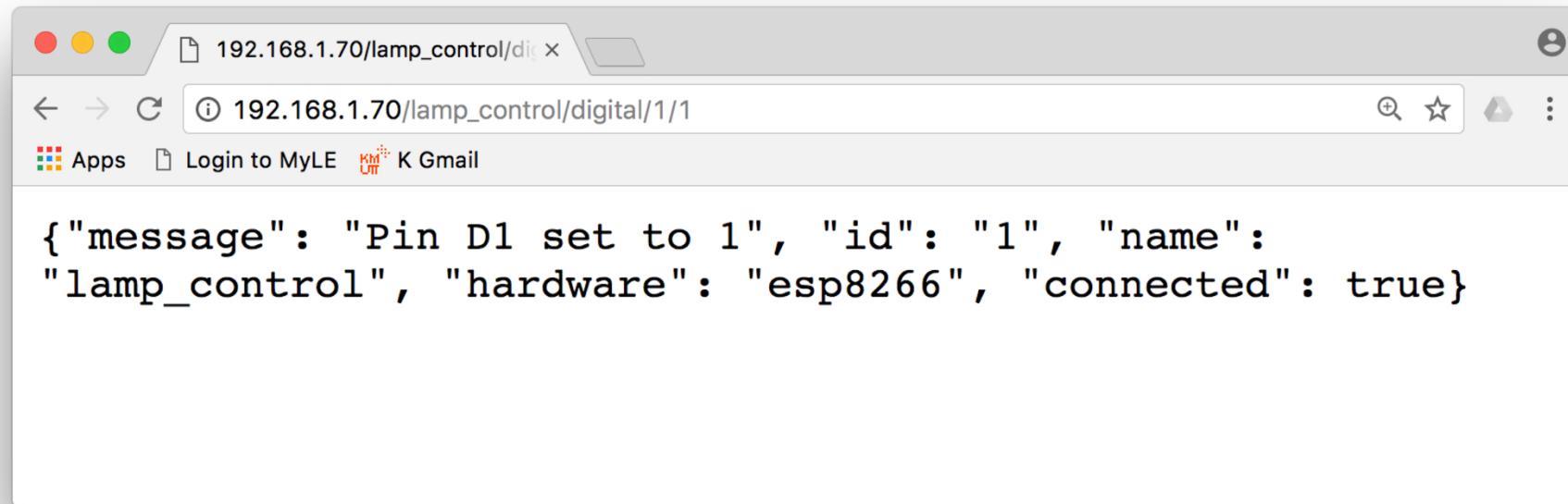




RESTful webservice can also control the actuators

e.g.; To turn on the LED connected to PIN D8, use

```
http://<board_IP_address>/lamp_control/digital/8/1
```

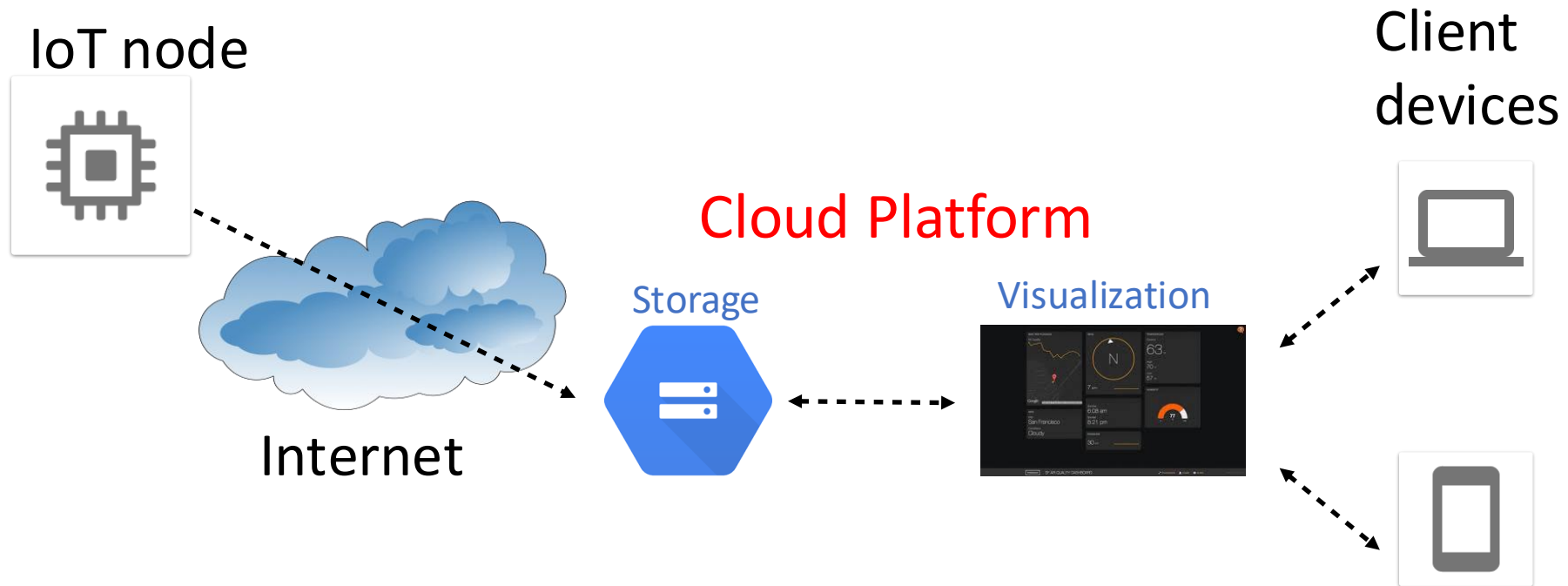


Limitations of method3

- HTTP not efficient for low-resource node, lossy environment
 - Take lots of bytes → More time to send and hence more power.
 - Based on TCP → More processing power and memory footprint. Not good for lossy network due to intermitted connectivity and retransmission.
- Will see alternative protocols taking less bytes in HTTP.
- Still no database to keep data.
- Limited version of user interface for visualization & control.

Method4:

IoT nodes send data to cloud service via HTTP



Public Cloud Platforms



Metabase



zetta



DeviceHub.net



MOZAİK



Home Assistant

Data Exchange Format

■ CSV

A CSV (comma-separated values) file is a **text file that has a specific format which allows data to be saved in a table structured format.**

Example: user.csv

Username; Identifier; First name; Last name

booker12; 9012; Rachel; Booker

grey07; 2070; Laura; Grey

johnson81; 4081; Craig; Johnson

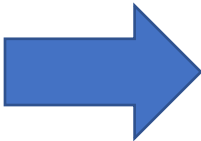
jenkins46; 9346; Mary; Jenkins

smith79; 5079; Jamie; Smith

Delimiter

Header

Data



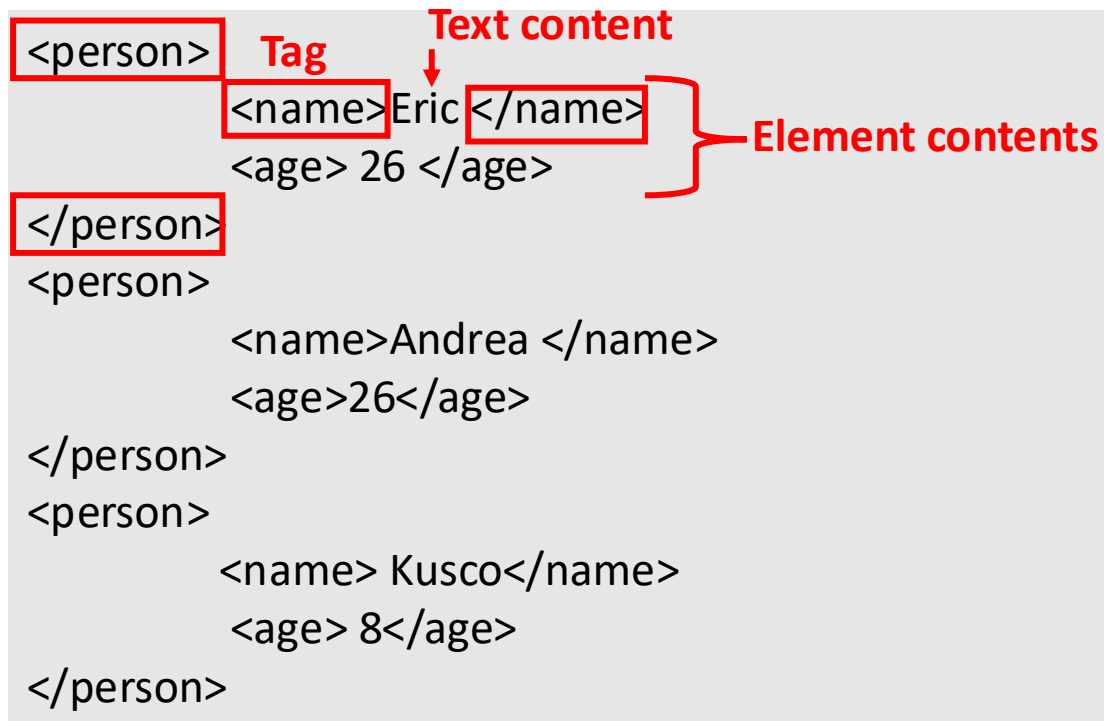
Username	Identifier	First name	Last name
booker12	9012	Rachel	Booker
grey07	2070	Laura	Grey
johnson81	4081	Craig	Johnson
jenkins46	9346	Mary	Jenkins
smith79	5079	Jamie	Smith

Data Exchange Format

■ XML

XML stands for **extensible markup language**. XML was designed in 1996 and officially became a W3C standard in 1998. It was created to better represent data formats with a hierarchical structure. The most famous markup language is hypertext markup language (HTML), which is used to format Web pages

Example:



The image shows an XML snippet with several annotations. The code is as follows:

```
<person>  
  <name>Eric</name>  
  <age> 26 </age>  
</person>  
<person>  
  <name>Andrea </name>  
  <age>26</age>  
</person>  
<person>  
  <name> Kusco</name>  
  <age> 8</age>  
</person>
```

Annotations in red:

- A red box around the opening tag `<person>`.
- A red box around the opening tag `<name>`, with the label "Tag" and an arrow pointing to it.
- A red box around the closing tag `</name>`, with the label "Text content" and an arrow pointing to it.
- A red bracket on the right side grouping the `<name>`, `Eric`, and `</name>` elements, with the label "Element contents".
- A red box around the closing tag `</person>`.

Data Exchange Format

■ JSON

JSON stands for “Javascript Object Notation” is a generic data format with a minimal number of value types: **strings, numbers, booleans, lists, objects, and null**. It was invented in 2001 and became popularized by Yahoo and Google in 2005 and 2006. It represents hierarchical data with the use of commas, curly braces and brackets

Example:

```
{“name”:“Eric”,“age”:“26”},  
{“name”:“Andrea”,“age”:“26”},  
{“name”:“Kusco”,“age”:“8”}
```

hierarchical data

```
{  
  "items": [{  
    "id": 3201,  
    "status": "",  
    "calibrated": 1521135120000,  
    "location": {  
      "latitude": -117.125,  
      "longitude": 31.125  
    }  
  },  
  {  
    "id": 5416,  
    "status": "offline",  
    "calibrated": 1521638100000,  
    "location": {  
      "latitude": -113.325,  
      "longitude": 33.325  
    }  
  }  
}]  
}
```

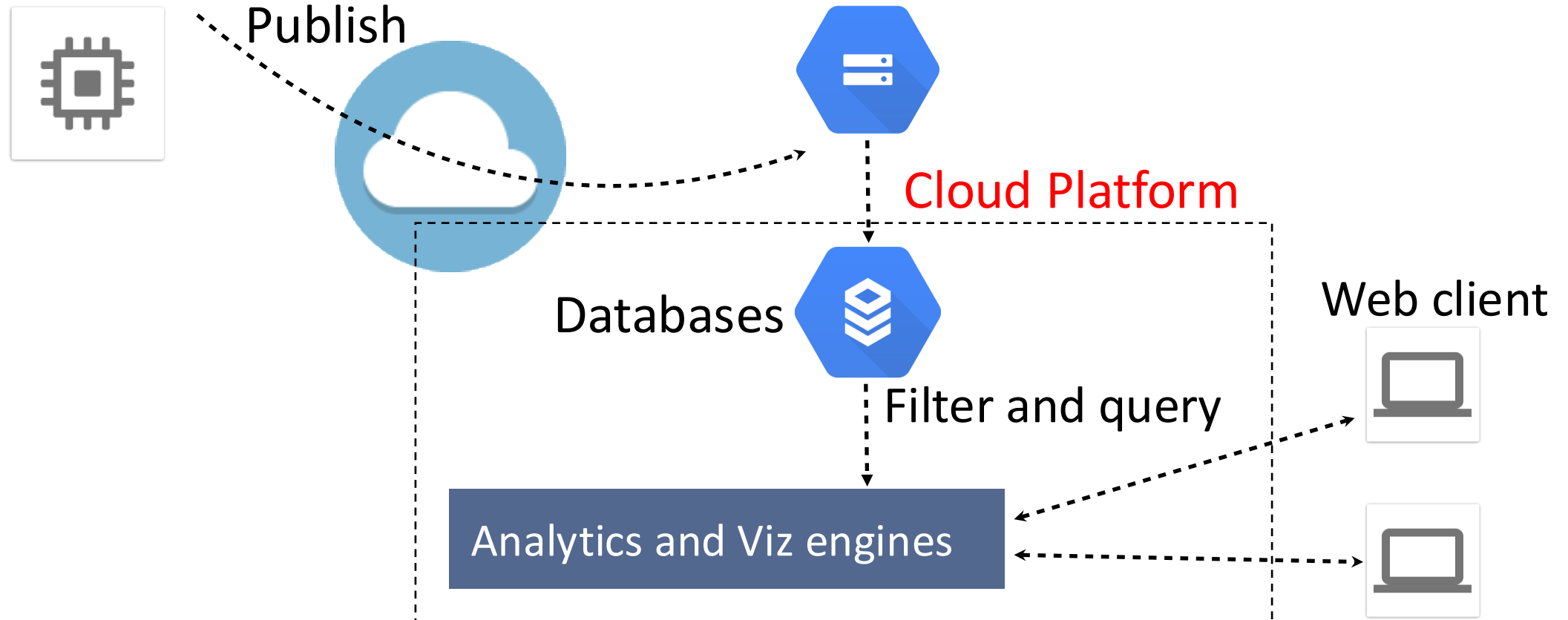
Method5:

Message Queuing Telemetry Transport

IoT nodes with IoT cloud platform using MQTT

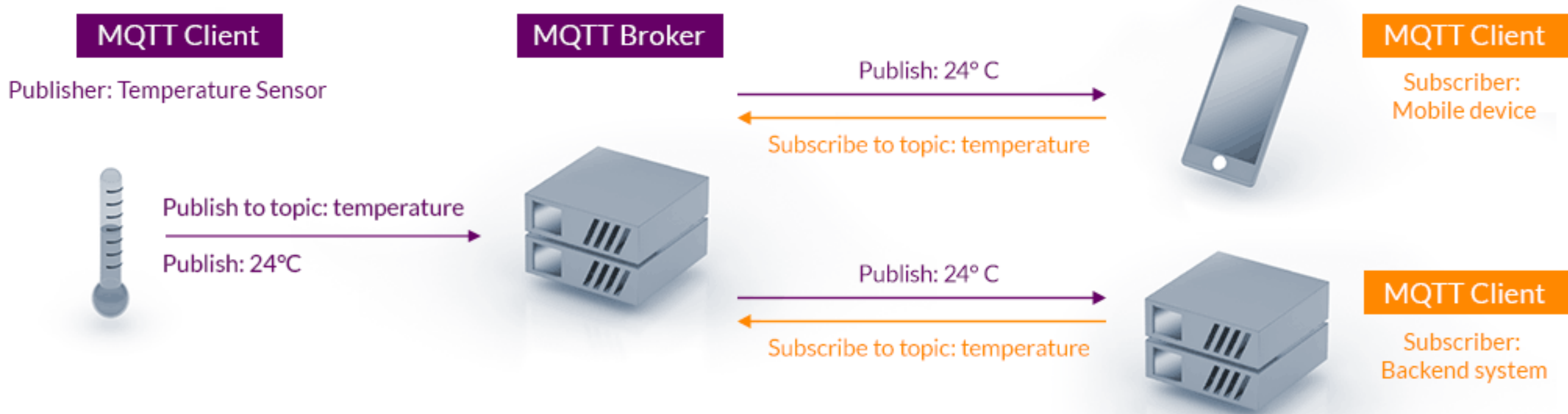
IoT Node with sensor
(MQTT Publisher)

Gateway
(MQTT Broker + Subscriber)



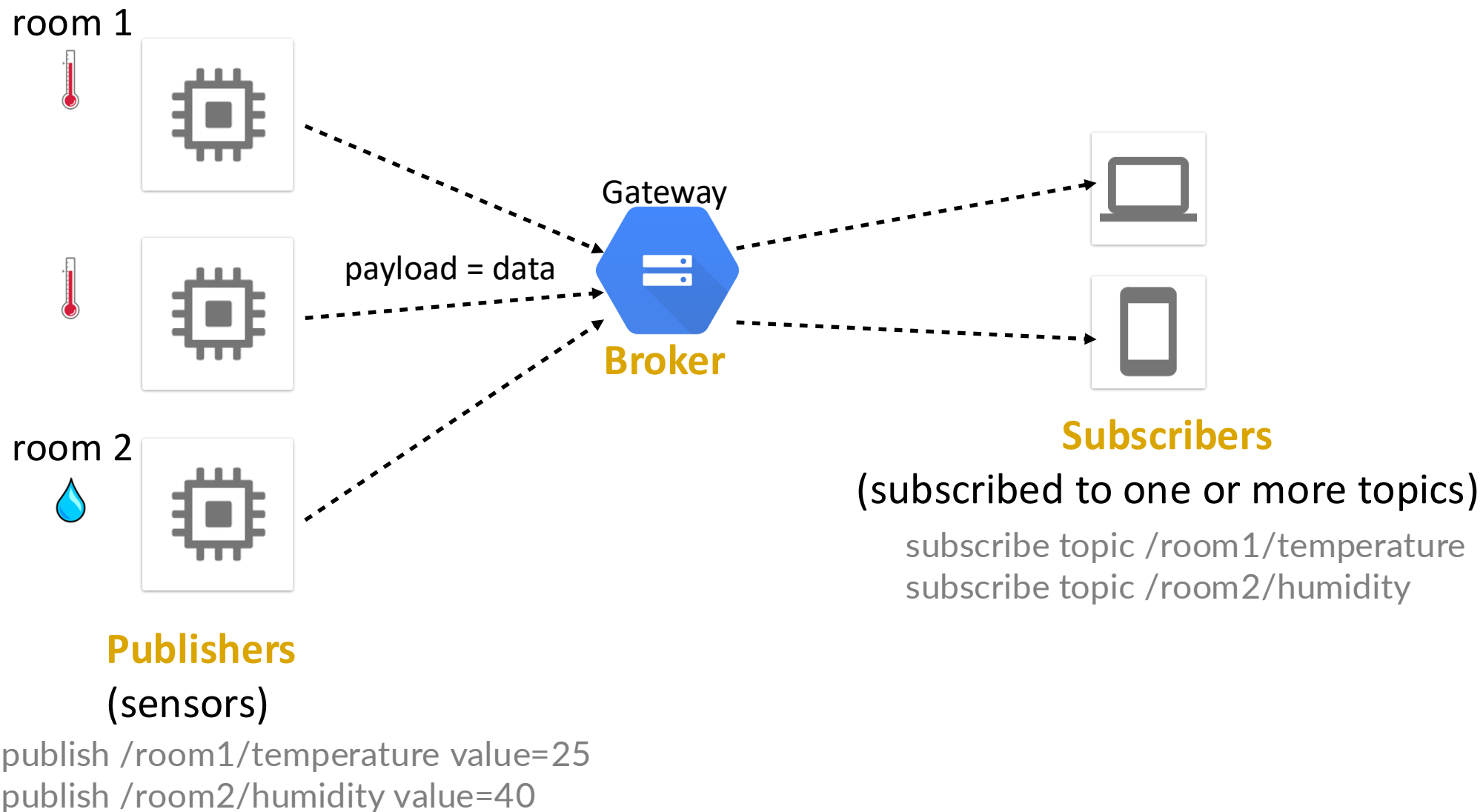
MQTT: The Standard for IoT Messaging

MQTT is an OASIS standard messaging protocol for the Internet of Things (IoT). It is designed as an extremely lightweight publish/subscribe messaging transport that is ideal for connecting remote devices with a small code footprint and minimal network bandwidth. MQTT today is used in a wide variety of industries, such as automotive, manufacturing, telecommunications, oil and gas, etc.



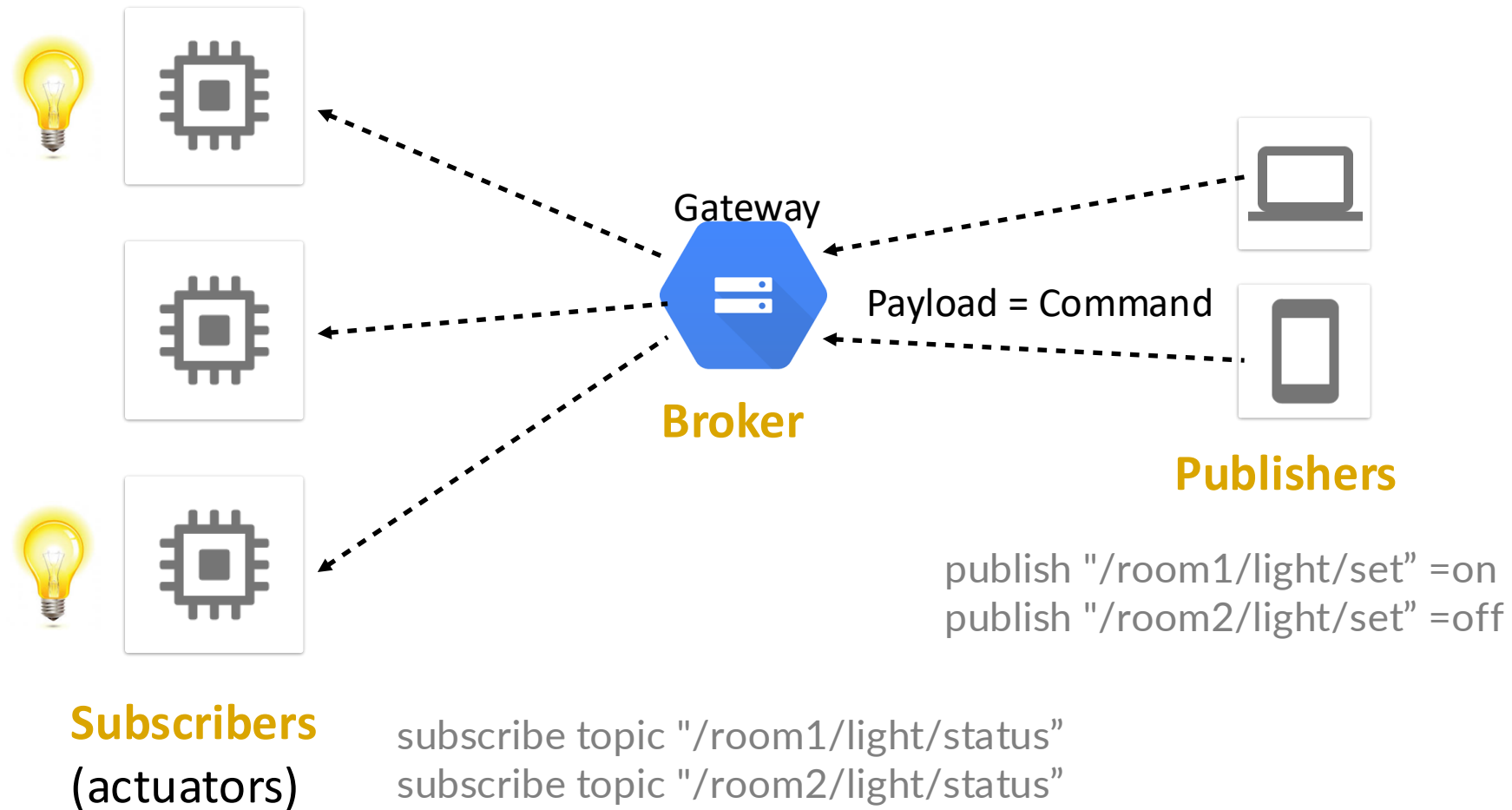
MQTT App Model — Telemetry

Message Queuing Telemetry Transport



MQTT App Model — Remote Control

Message Queuing Telemetry Transport



MQTT-QoS (Quality of Service)

- Depending on your type of data and the underlying transport
- **More reliability means more data transmissions, and more code management**
- managed by the broker, but the client can request certain QoS
 - QoS-0
 - QoS-1
 - QoS-2

QoS 0 - Fire and Forget

- The simplest QoS
- the broker/client will deliver the message once, with no confirmation
- This is best used when your underlying transport is very reliable, say TCP/IP, or when you want to conserve power and data at the trade of consistency



For example, say you have a GPS vehicle tracker that sends location data over cellular every 30 minutes.

If your tracker is running over a long period of time it might be OK if once in a while, a GPS location is 'dropped'. You just want to get most messages so you have an idea of where your truck is.

It's more important that you conserve battery because the cellular network has reliability and re-transmission built in - if you are outside range you don't want to keep re-trying, you'll use up your battery!

QoS 1 - Delivered at least once

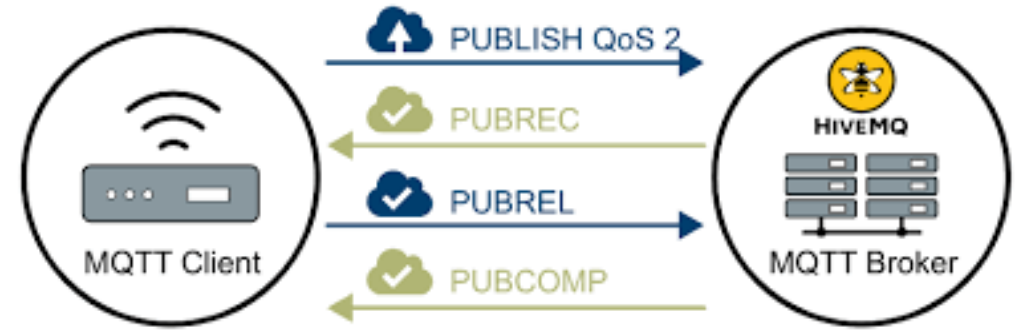
- The broker/client will deliver the message at least once, with confirmation required
- **ACK** are gotten for every publication, subscription



For example, you have an IoT AC unit that is monitored and controlled remotely. When you send a message for it to turn on you want to make sure it turns on! If you send a message and the transport fails, you can just keep sending messages over until you get a reply from the broker.

QoS 2 - Deliver only once

- the greatest of the QoS
- the broker/client will deliver the message **exactly once** by using a four-step handshake.



For example, you have a robotic tractor that you send commands like “Move forward 50 meters” - you want to make sure that you don’t accidentally send two messages and it turns out it moved 100 meters! Since QoS 2 is a bit of a pain, you may want to change this to, instead, use two topics: one that sends the movement command from controller to tractor, and then another one from the tractor to controller that gives the current location.

MQTT Topics

- The topic is referred to an UTF-8 string that the broker uses to filter messages for each connected client
- The topic consists of one or more topic levels
- Each topic level is separated by a forward slash (topic level separator).

Example:



The diagram illustrates the structure of an MQTT topic. It shows the topic string "myhome / groundfloor / livingroom / temperature" in green. Above the first slash, the text "topic level separator" is written in red, with a red arrow pointing down to the slash. Below the string, two green curly braces are used to group "myhome" and "groundfloor", each with the label "topic level" written below it in green.

topic level separator

↓

myhome / groundfloor / livingroom / temperature

topic level topic level

MQTT Wildcards

- Clients can subscribe to the exact topic of a published message
- or use **wildcards** to subscribe to multiple topics at the same time
- A wildcard can only be used to subscribe to topics, not to publish a message
- There are two different kinds of wildcards:
 - *single-level*
 - *multi-level*

Single Level: +

- A single-level wildcard replaces one topic level
- The plus symbol (+) represents a single-level wildcard in a topic.

Example:

myhome / groundfloor / + / temperature

single-level
wildcard
↓
only one level

Question: Which subscribed topic will be referred?

- a) myhome/groundfloor/livingroom/temperature
- b) myhome/groundfloor/kitchen/temperature
- c) myhome/groundfloor/kitchen/temperature
- d) myhome/firstfloor/kitchen/temperature
- e) myhome/groundfloor/kitchen/fidge/temperature

Multi Level:

- The multi-level wildcard covers many topic levels
- The hash symbol (#) represents the multi-level wild card in the topic
- For the broker to determine which topics match, the multi-level wildcard must be placed as the last character in the topic and preceded by a forward slash

Example:

multi-level
wildcard

↓

myhome / groundfloor / #

only at the end
multiple topic levels

Question: Which subscribed topic will be referred?

- a) myhome/groundfloor/livingroom/temperature
- b) myhome/groundfloor/kitchen/temperature
- c) myhome/groundfloor/kitchen/brightness
- d) myhome/firstfloor/kitchen/temperature



Paho is an



MQTT and MQTT-SN are lightweight publish/subscribe messaging transports for TCP/IP and connectionless protocols (such as UDP) respectively. The Eclipse Paho project provides open source, mainly client side, [implementations](#) of MQTT and MQTT-SN in a variety of programming languages.

```
1) To install paho  
$ pip install paho-mqtt
```

```
2) Using paho-mqtt  
>>>import paho.mqtt.client as mqtt
```

Publisher

```
import paho.mqtt.client as mqtt
MQTT_HOST = "broker.mqttdashboard.com"
MQTT_PORT = 1883
MQTT_KEEPALIVE_INTERVAL = 60 #60 seconds
MQTT_TOPIC = "CPE_HOUSE/room1"

def on_publish(self,client,userdata):
    print("Message Published...")

client = mqtt.Client()
client.on_publish = on_publish
client.connect(MQTT_HOST,MQTT_PORT,MQTT_KEEPALIVE_INTERVAL)
```

Subscriber

```
import paho.mqtt.client as mqtt
MQTT_HOST = "broker.mqttdashboard.com"

def on_connect(self, client, userdata, rc):
    print("MQTT Connected.")
    self.subscribe("CPE_HOUSE/room1/#")
    self.subscribe("CPE_HOUSE/room2/#")

def on_message(client, userdata, msg):
    print("message received...")
    print(msg.topic + " " + str(msg.qos) + " " + str(msg.payload.decode("utf-8", "strict")))

client = mqtt.Client()
client.on_connect = on_connect
client.on_message = on_message
client.connect(MQTT_HOST)
client.loop_forever()
```

IoT decisions require low-latency processing

IoT devices produce an ongoing sequence—not a completed dataset.

- Examples

- Detect overheating before equipment fails.
 - • Update room occupancy and energy control continuously.
 - • Identify abnormal vibration while a machine is running.
- Trigger an alert when measurements exceed a threshold.
 - Stream processing turns each new event into an updated result with bounded delay.

Two processing models answer different questions

Batch processing

What happened over a complete, bounded dataset?

Example: summarize yesterday's sensor history.

Stream processing

What is happening as new events arrive?

Example: detect overheating while equipment is running.

The right model is determined mainly by data boundaries and required response time.

Batch processing: bounded data, finite jobs

Input

A known dataset such as one hour, one day or one stored partition.

Execution

A job starts, scans the available data, produces a result and finishes.

Optimization goal

High throughput and efficient use of compute; latency is usually minutes to hours.

IoT uses

Daily reports, historical ETL, backfills, trend analysis and model training.

Stream processing: unbounded data, active queries

Input

An ongoing sequence of events with no natural end.

Execution

A query remains active and incrementally processes newly arrived data.

Optimization goal

Keep end-to-end delay within the application's decision deadline.

IoT uses

Live monitoring, anomaly detection, alerts, dashboards and control decisions.

Spark supports both

One programming model

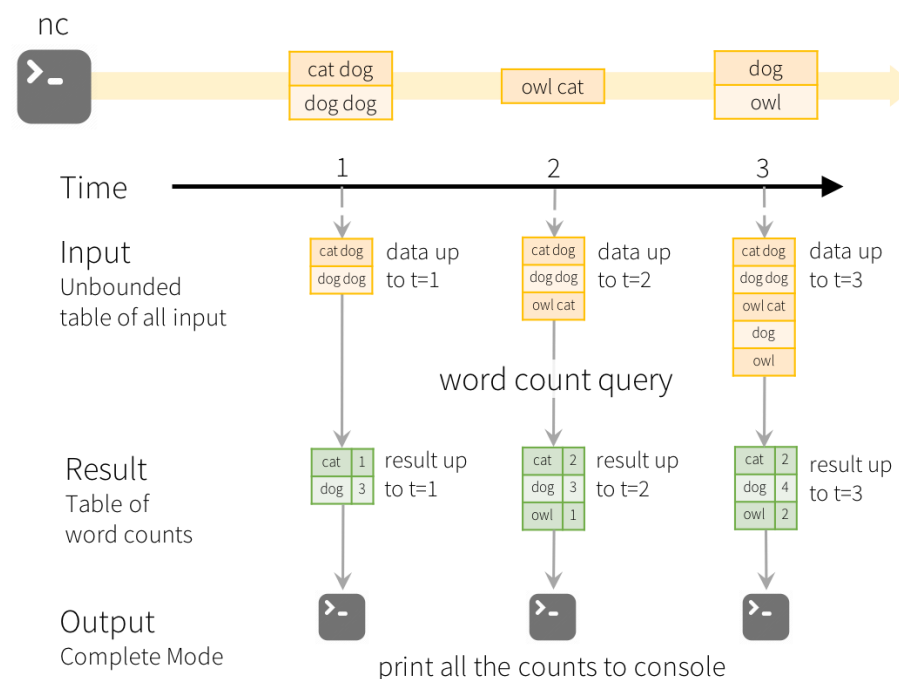
Use DataFrame and SQL operations for both bounded and unbounded data.

Incremental execution

Structured Streaming updates the result table whenever new input arrives.

Default engine

Spark normally executes the query as a sequence of micro-batches.



Model of the Quick Example

One temperature dataset, two useful products

Batch path — report later

At 00:05, read all readings from the previous day.

Compute min, max, average and missing-device counts.

Store one daily report for analysis and planning.

Stream path — act now

As readings arrive, compute a one-minute rolling room average.

If $\text{avg_temp} > 35^{\circ}\text{C}$, emit an alert immediately.

Keep event-time state only as long as needed for late events.

A real IoT platform commonly uses both paths for different decisions.

sensor reading is an event

A useful event carries both measurement and context.

Required fields

- device_id — stable producer identity
 - event_time — when the device observed the value
 - metric and value — what was measured
- location or group key — how events will be aggregated
 - event_id — supports deduplication

The event should be self-contained enough to process without querying the device again.

Stable schemas prevent silent corruption

Example JSON event

- {"event_id":"e-1042", "device_id":"sensor-01",
▪ "room":"room1", "event_time":"2026-08-31T09:31:22Z",
"temperature":36.4, "unit":"C"}
- Validation rules
- Parse event_time as TIMESTAMP—not as an arbitrary string.
 - • Store measurements as numeric types.
 - • Reject impossible values and unknown units.
- Version the schema when fields change.

Spark needs an MQTT ingestion bridge

- Spark Structured Streaming has no built-in MQTT source.
 - **An MQTT subscriber receives each JSON message.**
 - The bridge writes messages into a Spark-readable source.
-
- Classroom pipeline
 - MQTT → Python subscriber → new JSON files → Spark
-
- Keep ingestion and stream-processing responsibilities separate.

Use files for the first Spark pipeline

MQTT → subscriber → landing directory → Spark

- • The subscriber writes each message as a new JSON file.
- • Rename the completed temporary file atomically into the landing directory.
- Spark processes newly arrived files as an unbounded stream.
 - Why this works well for teaching
- MQTT and Spark responsibilities remain visible.
 - • No extra distributed messaging platform is required.
 - • Students can inspect the raw events.
- Limitation: file arrival is micro-batch oriented and not designed for very high event rates.

Streams behave like growing tables

Input table

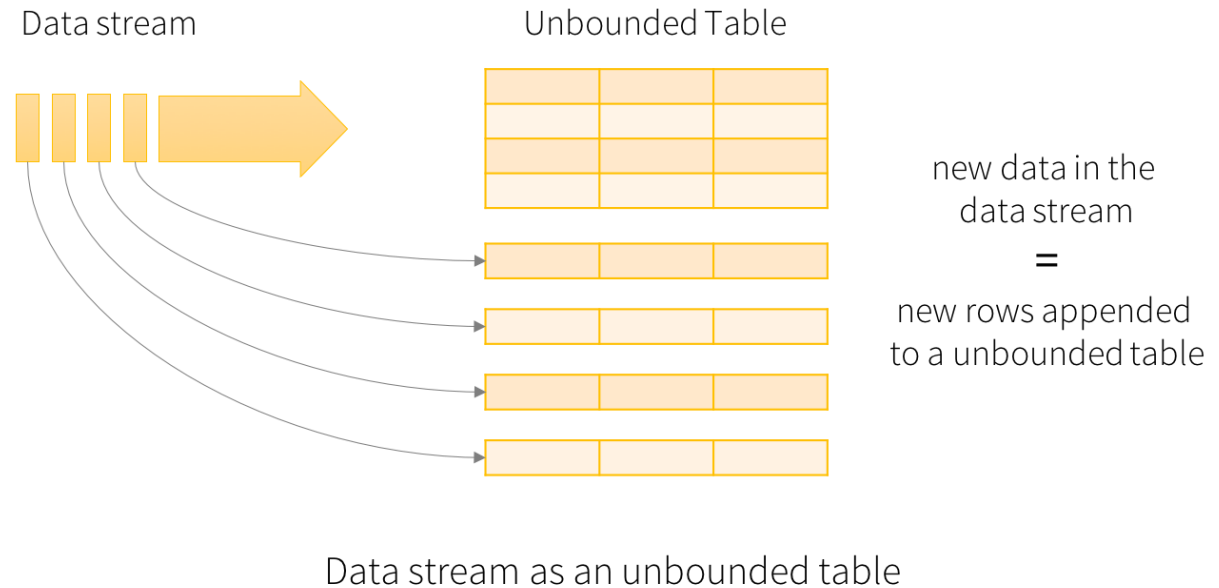
Every arriving IoT event behaves like a newly appended row.

Query

Spark applies familiar select, filter, join and groupBy operations.

Result table

Only the state needed to update the result is retained.



Four parts of a streaming query

☐ 1 Source

- ☐ New JSON landing files.

☐ 2 Transform

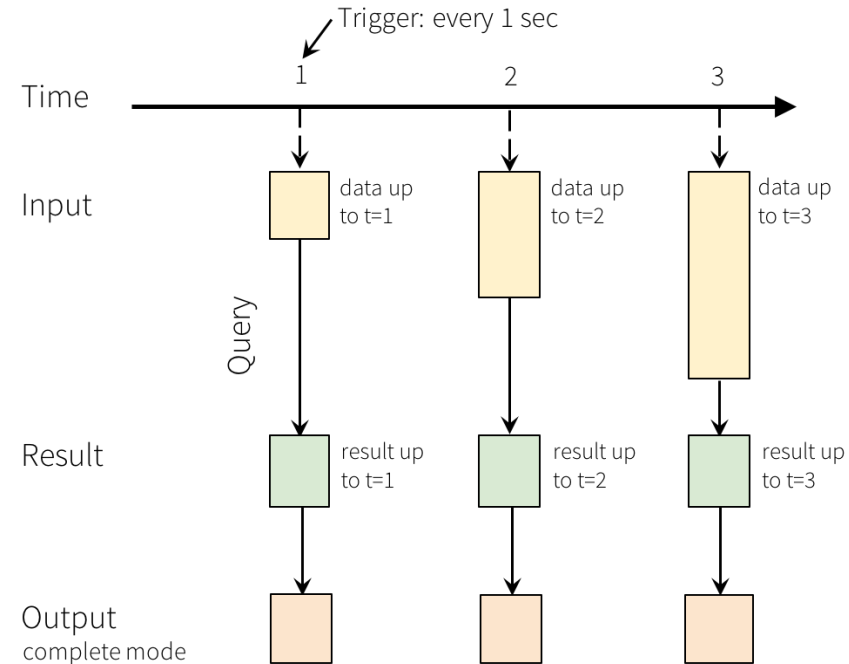
- ☐ Parse, validate, enrich and aggregate.

☐ 3 Trigger + output mode

- ☐ Decide when work runs and which rows are emitted.

☐ 4 Sink + checkpoint

- ☐ Write results and preserve recovery progress.



Programming Model for Structured Streaming

Sources and sinks define system boundaries

Sources for this lecture

- • JSON files arriving in a landing directory
- • Rate source for testing Spark logic
- Socket source only for a simple, non-fault-tolerant demonstration
- Common sinks
- Console or memory for development only
 - • Parquet/Delta-style storage for analysis
 - • foreachBatch for databases and custom logic
- Select a sink that supports the chosen output mode and failure behavior.

Build a clear transformation pipeline

- 1 Receive JSON from MQTT.
 - 2 Write each complete event as a new landing file.
 - 3 Read new files using an explicit schema.
- 4 Select typed event fields.
 - 5 Validate ranges and required values.
 - 6 Add reference data such as building or device type.
- 7 Assign event-time windows.
 - 8 Aggregate by window and key.
 - 9 Apply alert logic.
- 10 Write results with a checkpoint.

Bridge MQTT messages to landing files

```
import os, uuid
from pathlib import Path
import paho.mqtt.client as mqtt

LANDING = Path("/data/iot_landing")
LANDING.mkdir(parents=True, exist_ok=True)

def on_message(client, userdata, msg):
    event_id = uuid.uuid4().hex
    temporary = LANDING / f"{event_id}.tmp"
    completed = LANDING / f"{event_id}.json"
    temporary.write_bytes(msg.payload)
    os.replace(temporary, completed)

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.on_message = on_message
client.connect("localhost", 1883)
client.subscribe("CPE371/house/+/temperature")
client.loop_forever()
```

Read new JSON files as a stream

```
from pyspark.sql.functions import col
from pyspark.sql.types import *

schema = StructType([
    StructField("event_id", StringType()),
    StructField("device_id", StringType()),
    StructField("room", StringType()),
    StructField("event_time", TimestampType()),
    StructField("temperature", DoubleType()),
    StructField("unit", StringType())
])

events = (spark.readStream
    .schema(schema)
    .option("maxFilesPerTrigger", 100)
    .json("/data/iot_landing")
    .filter(col("event_id").isNotNull()))
```

Validate data before it enters state

Reject or quarantine

- Missing event_id, device_id or event_time
- Temperature outside the sensor's physical range
- Unsupported unit or schema version
 - Malformed JSON

Normalize

- Convert Fahrenheit to Celsius
- Map device_id to room and building
- Standardize timestamps to UTC
 - Deduplicate
 - Use event_id with a watermark so Spark does not keep unlimited state.

Use event time for IoT windows

- Event time: recorded by the device; represents the real measurement time.
 - **Processing time: when Spark handles the event; affected by delay and load.**
 - Ingestion time: when the platform receives the event.
-
- Use event time for IoT windows. Use ingestion time only when device clocks are unreliable.

State enables history-aware results

Stateless operations

- Select columns, filter rows and calculate a value from one event.

Stateful operations

- Windowed counts/averages, streaming deduplication and stream-stream joins.
- Spark must remember information across triggers.
- Control state growth
 - • Use watermarks for event-time state.
- Choose appropriate keys and window sizes.
 - • Avoid unbounded distinct groups.
 - • Monitor state-store rows, memory and processing time.

Choose an output mode

- Append: write only new final rows.
 - Update: write rows changed since the previous trigger.
 - Complete: write the entire result table every trigger.
-
- The query and sink must support the selected mode. Output mode alone does not guarantee end-to-end exactly-once behavior.

Checkpoints make a query recoverable

A checkpoint stores

- Which landing files have already been processed
- Query metadata
- State required by stateful operators
 - Operational rules
- Use durable storage—not /tmp in production.
 - Give each query a dedicated checkpoint directory.
 - Keep query structure compatible when restarting from a checkpoint.
- Make custom sinks idempotent when retries could repeat writes.
 - Recovery semantics depend on both the source and sink.

Worked example: detect a hot room

■ Input IoT events

Goal: calculate the one-minute average temperature per room and raise ALERT when $\text{avg_temp} > 35^{\circ}\text{C}$.

Source: MQTT devices bridged to a JSON landing directory

Example event:

```
{"room":"room1",  
  "event_time":"09:31:22",  
  "temperature":36.4}
```

WINDOW STATUS	ROOM	AVG_TEMP	COUNT
09:30–09:31	room1	34.8	27 OK
09:31–09:32	room1	36.1	30 ALERT

Processing path

parse and validate JSON

→ convert event_time

→ watermark 2 minutes

→ one-minute window

→ group by room

→ average temperature

→ apply threshold

→ write result / alert

Aggregate by event-time window

```
from pyspark.sql.functions import avg, count, window
```

```
aggregated = (events  
    .withWatermark("event_time", "2 minutes")  
    .groupBy(  
        window("event_time", "1 minute"),  
        "room"  
    )  
    .agg(  
        avg("temperature").alias("avg_temp"),  
        count("*").alias("event_count")  
    ))
```

```
results = aggregated.select(  
    "window.start", "window.end", "room",  
    "avg_temp", "event_count"  
)
```

Write updates and create alerts

```
from pyspark.sql.functions import col, when
```

```
output = results.withColumn(  
    "status",  
    when(col("avg_temp") > 35, "ALERT")  
    .otherwise("OK")  
)
```

```
query = (output.writeStream  
    .outputMode("update")  
    .format("console")  
    .option("truncate", "false")  
    .option("checkpointLocation",  
            "/checkpoints/room-temperature")  
    .trigger(processingTime="10 seconds")  
    .start())
```

```
query.awaitTermination()
```

Monitor whether Spark keeps up

☐ Throughput

- ☐ Compare input rows/s with processed rows/s.

☐ Trigger latency

- ☐ Check whether each micro-batch finishes before the next trigger.

☐ Backlog

- ☐ Watch the number and age of unprocessed landing files.

☐ State health

- ☐ Monitor state rows, memory, failed batches and sink errors.

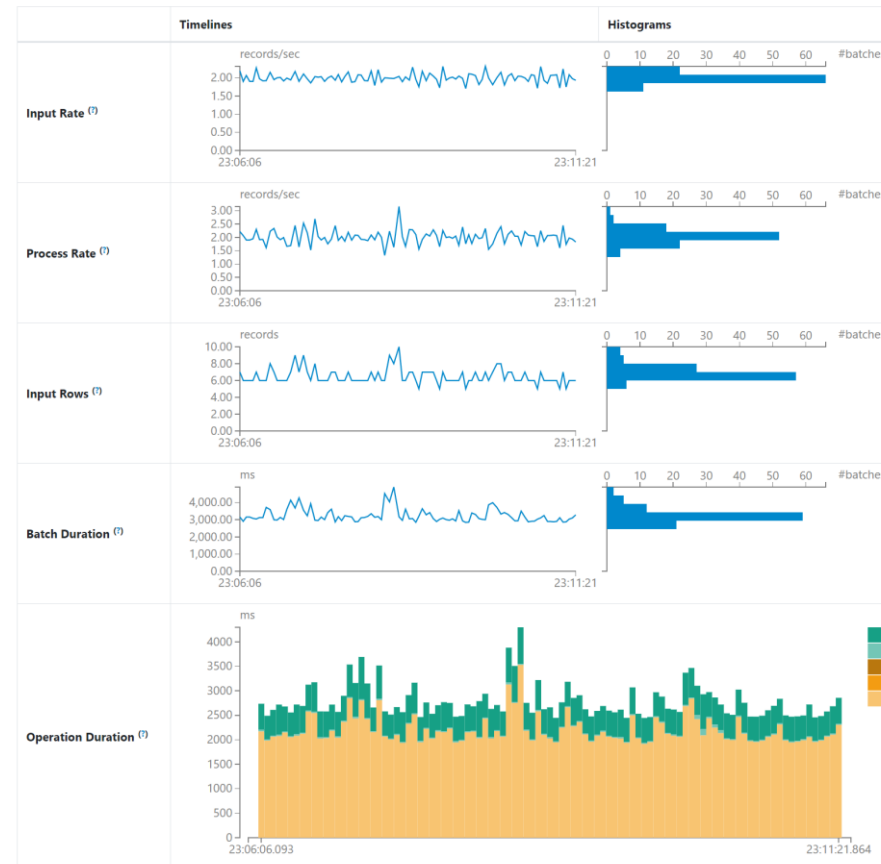
Streaming Query Statistics

Running batches for 26 minutes 56 seconds since 2020/05/21 22:44:28 (505 completed batches)

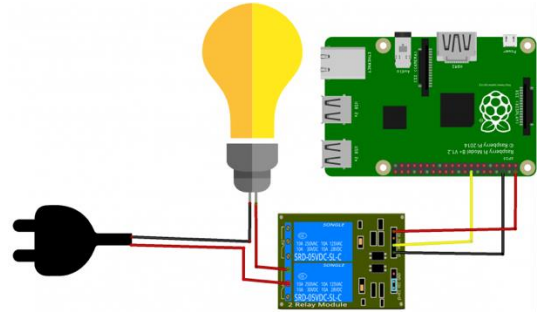
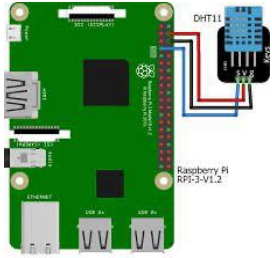
Name: myQuery

Id: 9d0dd219-995d-423e-b97d-36931110bb27

RunId: 04e101e6-8bf8-4165-9099-9104c951ba3b



■ Temperature/ Humidity Sensor



■ LED Controller



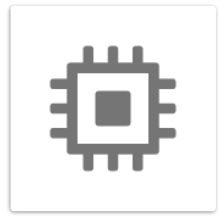
Activity:

Activity: MQTT-to-Spark Stream Processing

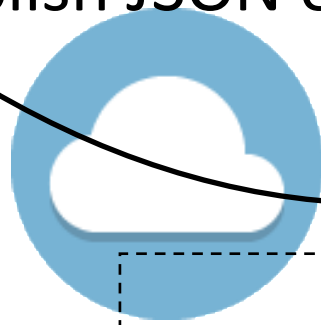
1. Publish the provided sensor data to the MQTT broker.
2. Subscribe to the topic and write each event to the JSON landing directory.
3. Process newly arriving files with Spark Structured Streaming.
4. Calculate windowed averages (10 sec) for:
 - Indoor temperature
 - Indoor humidity
 - Outdoor temperature
 - Outdoor humidity
5. Display the updated results.

From MQTT devices to continuous results

IoT devices / gateways
(MQTT publishers)



Publish JSON events



MQTT broker
(device ingestion)



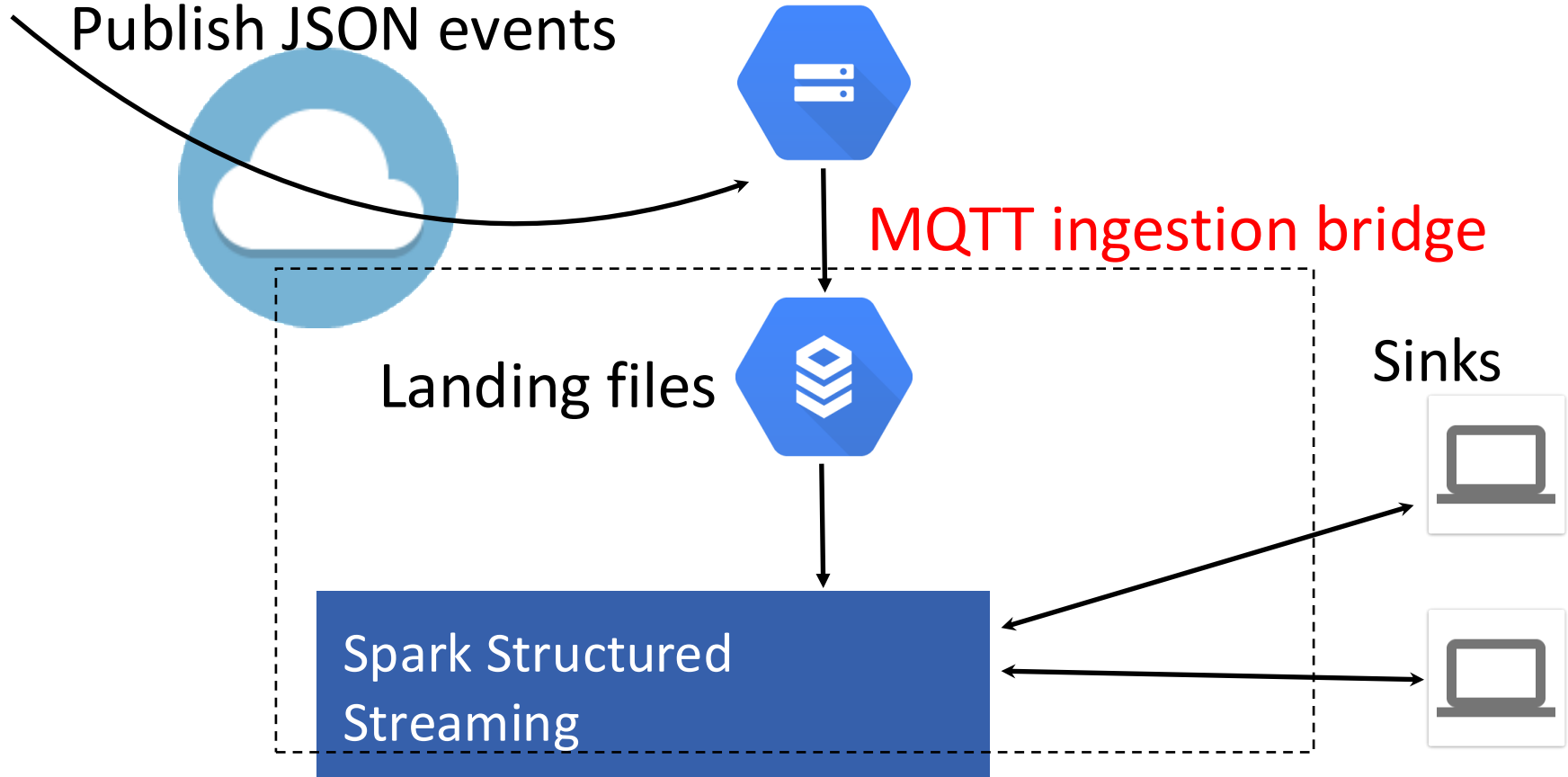
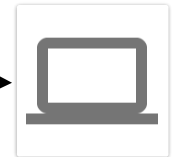
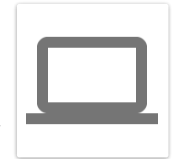
MQTT ingestion bridge

Landing files



Spark Structured Streaming

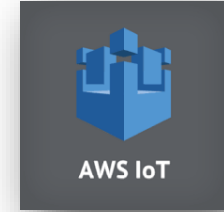
Sinks



Some IoT Platforms

- Available functionalities

- IoT data transport
- Cloud storage, visualization, dashboards
- Some with SDKs, APIs and GUIs to create advanced apps.
- Based on both proprietary and open protocols (Free vs. Commercial) — 40+



Predix



creator



ThingWorx
A PTC Business



carriots

Issues in Real-World Deployment

Power consumption

- Node would last a few hours if running non-stop.
- Need sleep mode, ultra low-power CPU and networking, etc.

Network coverage

- Sensor nodes may be far from gateway
- Required multi-hop networking or long-range networking technologies

Security

- Does telemetry data come from valid nodes?
- Who can send commands to control IoT actuator node

Management

- Gateway and resource discovery
- Remote status monitoring
- Firmware upgrades and key assignment for 10k+ nodes?

Summary

- MQTT as the key application layer protocol for IoT applications
- Customable gateway to collect data from IoT nodes allows for more flexibility
 - Data directly forwarded to public cloud for visualization
 - Data stored in database can be retrieved for later analytics
- Only touch a surface of largely available IoT technology options
 - Choices determined by requirements and constraints
 - Real-world deployment must also consider energy consumption, cost, manageability, security, etc.